

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Muhammad Rizki Ramadhan

NIM. 2310817310008

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rizki Ramadhan

NIM : 2310817310008

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Eka Setya Wijaya, S.T., M.Kom.
NIP. 198205082008011010

Daftar Isi

LEMBAR PENGESAHAN	i
Daftar Isi.....	ii
Daftar Gambar.....	iv
Daftar Tabel	v
MODUL 1: Android Basic with Kotlin	1
SOAL 1	1
A. Source Code	4
B. Output Program.....	12
C. Pembahasan.....	15
D. Tautan Git.....	18
MODUL 2: Android Layout with Compose	19
SOAL 1	19
E. Source Code	21
F. Output Program.....	35
G. Pembahasan.....	38
SOAL 2	43
H. Pembahasan.....	43
I. Tautan Git.....	43
MODUL 3: Build a Scrollable List.....	44
SOAL 1	44
J. Source Code	46
K. Output Program.....	69
L. Pembahasan.....	71
SOAL 2	79
M. Pembahasan.....	79
N. Tautan Git.....	79
MODUL 4: ViewModel and Debugging	80
SOAL 1	80
O. Source Code	81
P. Output Program.....	106
Q. Pembahasan.....	108
SOAL 2	122

R. Pembahasan.....	122
S. Tautan Git.....	122
MODUL 5: Connect to the Internet	123
SOAL 1	123
T. Source Code	123
U. Output Program.....	144
V. Pembahasan.....	145
W. Tautan Git.....	161

Daftar Gambar

Gambar 1. Tampilan Awal Aplikasi	1
Gambar 2. Tampilan Dadu Setelah Diroll.....	2
Gambar 3. Tampilan Roll Dadu Double	3
Gambar 4. Tampilan Awal Aplikasi XML	12
Gambar 5. Tampilan Dadu Setelah Di-Roll XML	12
Gambar 6. Tampilan Roll Dadu Double XML	13
Gambar 7. Tampilan Awal Aplikasi Jetpack Compose	13
Gambar 8. Tampilan Dadu Setelah Di-Roll Jetpack Compose.....	14
Gambar 9. Tampilan Roll Dadu Double Jetpack Compose	14
Gambar 10. Tampilan Awal Aplikasi	19
Gambar 11. Tampilan Dadu Setelah Diroll.....	20
Gambar 12. Tampilan Roll Dadu Double	20
Gambar 13. Tampilan Awal Aplikasi XML	35
Gambar 14. Tampilan Pilihan Persentase Tip XML	35
Gambar 15. Tampilan Aplikasi Setelah Dijalankan XML.....	36
Gambar 16. Tampilan Awal Aplikasi Jetpack Compose	36
Gambar 17. Tampilan Pilihan Persentase Tip Jetpack Compose.....	37
Gambar 18. Tampilan Aplikasi Setelah Dijalankan Jetpack Compose.....	37
Gambar 19. UI List	45
Gambar 20. UI Detail.....	45
Gambar 21. UI List (XML).....	69
Gambar 22. UI Detail (XML)	69
Gambar 23. UI List (Jetpack Compose).....	70
Gambar 24. UI Detail (Jetpack Compose)	70
Gambar 25. UI List (XML).....	106
Gambar 26. UI Detail (XML)	106
Gambar 27. UI List (Jetpack Compose).....	107
Gambar 28. UI Detail (Jetpack Compose)	107
Gambar 29. UI List	144
Gambar 30. UI Detail.....	144
Gambar 31. UI IMDb.....	145

Daftar Tabel

Table 1. Source Code Jawaban Soal 1 XML	4
Table 2. Source Code Jawaban Soal 1 XML	5
Table 3. Source Code Jawaban Soal 1 Jetpack Compose	7
Table 4. Source Code Jawaban Soal 1 XML	21
Table 5. Source Code Jawaban Soal 1 XML	27
Table 6. Source Code Jawaban Soal 1 Jetpack Compose	27
Table 7. Source Code Jawaban Soal 1 XML	46
Table 8. Source Code Jawaban Soal 1 XML	47
Table 9. Source Code Jawaban Soal 1 XML	48
Table 10. Source Code Jawaban Soal 1 XML	50
Table 11. Source Code Jawaban Soal 1 XML	52
Table 12. Source Code Jawaban Soal 1 XML	53
Table 13. Source Code Jawaban Soal 1 XML	55
Table 14. Source Code Jawaban Soal 1 XML	57
Table 15. Source Code Jawaban Soal 1 Jetpack Compose	57
Table 16. Source Code Jawaban Soal 1 Jetpack Compose	60
Table 17. Source Code Jawaban Soal 1 Jetpack Compose	63
Table 18. Source Code Jawaban Soal 1 Jetpack Compose	65
Table 19. Source Code Jawaban Soal 1 Jetpack Compose	66
Table 20. Source Code Jawaban Soal 1 Jetpack Compose	66
Table 21. MainActivity.kt.....	81
Table 22. Chocolate.kt	82
Table 23. ChocolateProvider.kt.....	83
Table 24. ChocolateAdapter.kt	84
Table 25. DetailActivity.kt.....	86
Table 26. ChocolateViewModel.kt	87
Table 27. ChocolateViewModelFactory.kt.....	88
Table 28. activity_main.xml	89
Table 29. activity_detail.xml.....	89
Table 30. row_chocolate.xml.....	91
Table 31. MainActivity.kt.....	92

Table 32. Chocolate.kt	95
Table 33. ChocolateProvider.kt.....	95
Table 34. ChocolateItem.kt.....	97
Table 35. ChocolateListScreen.kt	101
Table 36. DetailScreen.kt.....	102
Table 37. ChocolateViewModel.kt	104
Table 38. MainActivity.kt.....	123
Table 39. MovieDao.kt	128
Table 40. MovieDatabase.kt	129
Table 41. MovieEntity.kt	129
Table 42. MovieListResponse.kt.....	130
Table 43. ApiService.kt.....	131
Table 44. MovieRepositoryImpl.kt.....	131
Table 45. Mapper.kt	133
Table 46. Movie.kt	134
Table 47. MovieRepository.kt.....	134
Table 48. GetPopularMoviesUseCase.kt	134
Table 49. MovieItem.kt.....	135
Table 50. MovieDetailScreen.kt	138
Table 51. MovieListScreen.kt.....	140
Table 52. MovieListState.kt.....	141
Table 53. MovieViewModel.kt.....	142
Table 54. MovieViewModelFactory.kt.....	143

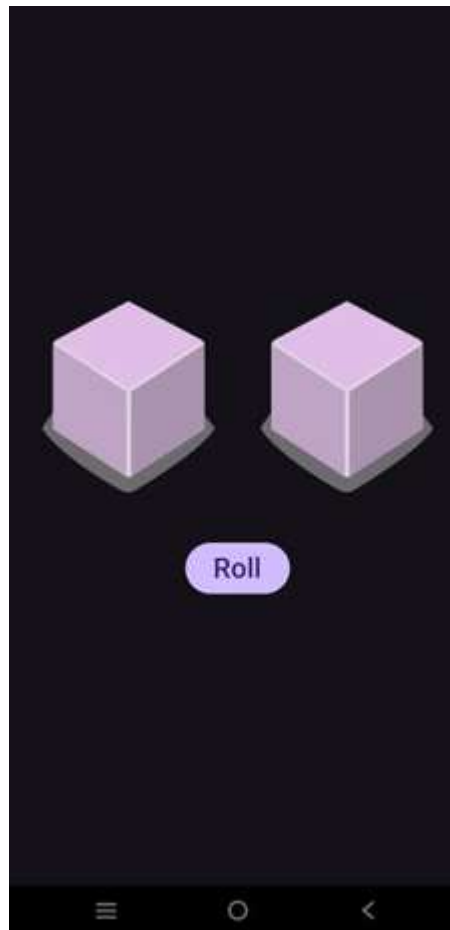
MODUL 1: Android Basic with Kotlin

SOAL 1

Soal Praktikum:

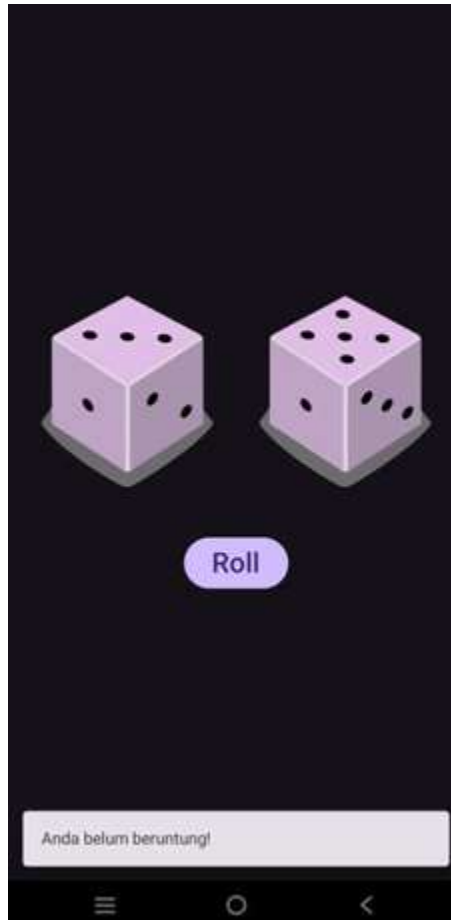
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



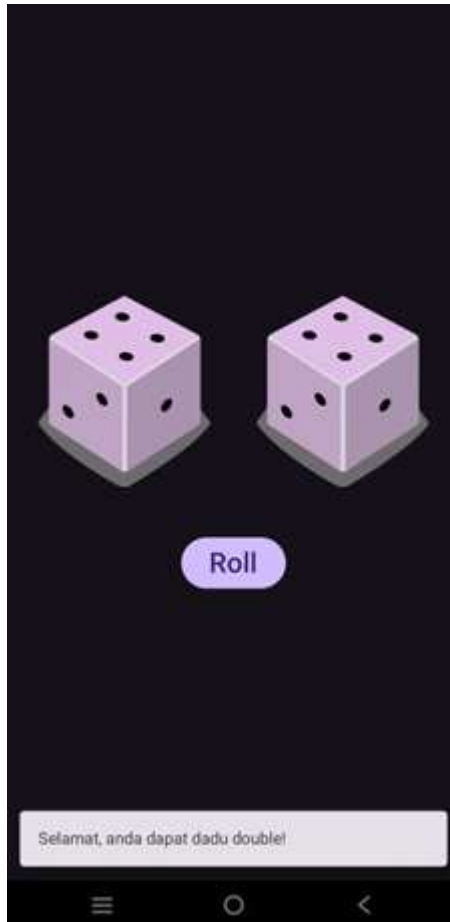
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Diroll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 1 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

XML:

MainActivity.kt

Table 1. Source Code Jawaban Soal 1 XML

1	package com.example.diceroll
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import com.google.android.material.snackbar.Snackbar
7	import androidx.appcompat.app.AppCompatActivity
8	import com.example.diceroll2.R
9	
10	class MainActivity : AppCompatActivity() {
11	
12	private lateinit var diceImage1: ImageView
13	private lateinit var diceImage2: ImageView
14	private lateinit var rollButton: Button
15	private lateinit var coordinatorLayout:
16	androidx.coordinatorlayout.widget.CoordinatorLayout
17	private val diceImages = listOf(
18	R.drawable.dice_0,
19	R.drawable.dice_1,
20	R.drawable.dice_2,
21	R.drawable.dice_3,
22	R.drawable.dice_4,
23	R.drawable.dice_5,
24	R.drawable.dice_6
25	)
26	
27	override fun onCreate(savedInstanceState: Bundle?) {
28	super.onCreate(savedInstanceState)

29	setContentView(R.layout.activity_main)	
30		
31	coordinatorLayout	=
	findViewById(R.id.coordinatorLayout)	
32	diceImage1 = findViewById(R.id.dice_image1)	
33	diceImage2 = findViewById(R.id.dice_image2)	
34	rollButton = findViewById(R.id.roll_button)	
35		
36	rollButton.setOnClickListener {	
37	val result1 = (1..6).random()	
38	val result2 = (1..6).random()	
39		
40	diceImage1.setImageResource(diceImages[result1])	
41	diceImage2.setImageResource(diceImages[result2])	
42		
43	val message = if (result1 == result2) {	
44	"Selamat, anda dapat dadu double!"	
45	} else {	
46	"Anda belum beruntung!"	
47	}	
48		
49	Snackbar.make(coordinatorLayout,	message,
	Snackbar.LENGTH_SHORT).show()	
50	}	
51	}	
52	}	

activity_main.xml

Table 2. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:id="@+id/coordinatorLayout"
5	android:layout_width="match_parent"

```

6      android:layout_height="match_parent"
7      android:padding="24dp">
8
9      <LinearLayout
10         android:orientation="vertical"
11         android:gravity="center"
12         android:layout_width="match_parent"
13         android:layout_height="match_parent">
14
15         <LinearLayout
16             android:orientation="horizontal"
17             android:layout_width="wrap_content"
18             android:layout_height="wrap_content"
19             android:layout_marginBottom="16dp">
20
21             <ImageView
22                 android:id="@+id/dice_image1"
23                 android:layout_width="150dp"
24                 android:layout_height="match_parent"
25                 android:src="@drawable/dice_0" />
26
27             <ImageView
28                 android:id="@+id/dice_image2"
29                 android:layout_width="150dp"
30                 android:layout_height="170dp"
31                 android:src="@drawable/dice_0" />
32         </LinearLayout>
33
34         <Button
35             android:id="@+id/roll_button"
36             android:layout_width="wrap_content"
37             android:layout_height="wrap_content"
38             android:text="Roll" />
39     </LinearLayout>

```

40	</androidx.coordinatorlayout.widget.CoordinatorLayout>
----	--

Jetpack Compose:

MainActivity.kt

Table 3. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.diceroll
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.Image
8	import androidx.compose.foundation.layout.Arrangement
9	import androidx.compose.foundation.layout.Column
10	import androidx.compose.foundation.layout.Row
11	import androidx.compose.foundation.layout.Spacer
12	import androidx.compose.foundation.layout.fillMaxSize
13	import androidx.compose.foundation.layout.height
14	import androidx.compose.foundation.layout.padding
15	import androidx.compose.foundation.layout.wrapContentSize
16	import androidx.compose.material3.Button
17	import androidx.compose.material3.Scaffold
18	import androidx.compose.material3.SnackbarDuration
19	import androidx.compose.material3.SnackbarHost
20	import androidx.compose.material3.SnackbarHostState
21	import androidx.compose.material3.Text
22	import androidx.compose.runtime.Composable
23	import androidx.compose.runtime.mutableStateOf
24	import androidx.compose.runtime.remember
25	import androidx.compose.runtime.getValue
26	import androidx.compose.runtime.rememberCoroutineScope
27	import androidx.compose.runtime.setValue
28	import androidx.compose.ui.Alignment
29	import androidx.compose.ui.Modifier

```

30 import androidx.compose.ui.res.painterResource
31 import androidx.compose.ui.res.stringResource
32 import androidx.compose.ui.tooling.preview.Preview
33 import androidx.compose.ui.unit.dp
34 import com.example.diceroll.ui.theme.DiceRollTheme
35 import kotlinx.coroutines.Job
36 import kotlinx.coroutines.launch
37
38 class MainActivity : ComponentActivity() {
39     override fun onCreate(savedInstanceState: Bundle?) {
40         super.onCreate(savedInstanceState)
41         enableEdgeToEdge()
42         setContent {
43             DiceRollTheme {
44                 DiceRollerApp()
45             }
46         }
47     }
48
49     @Preview
50     @Composable
51     fun DiceRollerApp() {
52         DiceWithButtonAndImage()
53     }
54
55     @Composable
56     fun DiceWithButtonAndImage(
57         modifier: Modifier = Modifier
58             .fillMaxSize()
59             .wrapContentSize(Alignment.Center)
60     ) {
61         var dismissJob by remember { mutableStateOf<Job?>(null) }
62

```

63	var result1 by remember { mutableStateOf(0) }
64	var result2 by remember { mutableStateOf(0) }
65	val snackbarHostState = remember { SnackbarHostState() }
66	val coroutineScope = rememberCoroutineScope()
67	
68	val dice1 = when (result1) {
69	0 -> R.drawable.dice_0
70	1 -> R.drawable.dice_1
71	2 -> R.drawable.dice_2
72	3 -> R.drawable.dice_3
73	4 -> R.drawable.dice_4
74	5 -> R.drawable.dice_5
75	else -> R.drawable.dice_6
76	}
77	
78	val dice2 = when (result2) {
89	0 -> R.drawable.dice_0
80	1 -> R.drawable.dice_1
81	2 -> R.drawable.dice_2
82	3 -> R.drawable.dice_3
83	4 -> R.drawable.dice_4
84	5 -> R.drawable.dice_5
85	else -> R.drawable.dice_6
86	}
87	
88	Scaffold(
89	snackbarHost = { SnackbarHost(snackbarHostState) }
90	
91	) { padding ->
92	Column(
93	modifier = modifier.padding(padding),
94	horizontalAlignment
	Alignment.CenterHorizontally

95	) {	
96	Row(	
97	horizontalArrangement	=
	Arrangement.spacedBy(16.dp),	
98	verticalAlignment	=
	Alignment.CenterVertically	
99	) {	
100	Image(	
101	painter = painterResource(dice1),	
102	contentDescription	=
	result1.toString()	
103	)	
104	Image(	
105	painter = painterResource(dice2),	
	contentDescription	=
106	result2.toString()	
107	)	
108	}	
109		
110	Spacer(modifier = Modifier.height(10.dp))	
111		
112	Button(onClick = {	
113	result1 = (1..6).random()	
114	result2 = (1..6).random()	
115		
116	val message = if (result1 == result2) {	
117	"Selamat, anda dapat dadu double!"	
118	} else {	
119	"Anda belum beruntung!"	
120	}	
121		
122	coroutineScope.launch	{
123	snackbarHostState.currentSnackbarData?.dismiss()	
124		

125	snackbarHostState.showSnackbar(	
126	message = message,	
127	duration	=
	SnackbarDuration.Indefinite	
128	)	
129	}	
130		
131	dismissJob?.cancel()	
132		
133	dismissJob = coroutineScope.launch {	
134	kotlinx.coroutines.delay(2000)	
135	snackbarHostState.currentSnackbarData?.dismiss()	
136	}	
137	)) {	
138	Text(stringResource(R.string.roll))	
139	}	
140	}	
141	}	
142	}	
143	}	

B. Output Program



Gambar 4. Tampilan Awal Aplikasi XML



Gambar 5. Tampilan Dadu Setelah Di-Roll XML



Gambar 6. Tampilan Roll Dadu Double XML



*Gambar 7. Tampilan Awal Aplikasi
Jetpack Compose*



*Gambar 8. Tampilan Dadu Setelah Di-Roll
Jetpack Compose*



Gambar 9. Tampilan Roll Dadu Double Jetpack Compose

C. Pembahasan

MainActivity.kt (XML):

1. Pada line [1], syntax `package com.example.diceroll` merupakan deklarasi nama package file Kotlin.
2. Pada line [3] sampai line [8], `import android.os.Bundle, import android.widget.Button, import android.widget.ImageView, import com.google.android.material.snackbar.Snackbar, import androidx.appcompat.app.AppCompatActivity, import com.example.diceroll.R` merupakan program import class dan library yang diperlukan, seperti penyimpanan data sementara, mengatur komponen UI, menampilkan pesan pada layer, dan mengakses resource layout, drawable, dan lain-lain.
3. Pada line [10], syntax `class MainActivity : AppCompatActivity() {}` merupakan deklarasi kelas bernama MainActivity yang extend dari AppCompatActivity.
4. Pada line [12] sampai line [15], mereferensikan komponen UI dari layout XML, dan lateinit berarti variable akan diinisialisasi nanti.
5. Pada line [17] sampai line [25], deklarasi list variabel `diceImage`.
6. Pada line [27] sampai line [29], fungsi yang digunakan untuk menetapkan layout XML sebagai UI tampilan utama.
7. Pada line [31] sampai line [34], menghubungkan variable Kotlin dengan komponen XML menggunakan Id.
8. Pada line [36] sampai line [38], logika saat tombol ditekan program akan dijalankan dan menghasilkan angka acak dari 1 sampai 6 pada dua dadu.
9. Pada line [40] sampai line [41], deklarasi untuk mengubah gambar dadu sesuai angka hasil random.
10. Pada line [43] sampai line [47], logika untuk menentukan pesan, jika kedua dadu sama, maka akan menampilkan pesan tertentu, dan sebaliknya.
11. Pada line [49], syntax `Snackbar.make(coordinatorLayout, pesan, Snackbar.LENGTH_SHORT).show()` merupakan deklarasi program untuk menampilkan snackbar di bawah layer dengan pesan yang ditentukan.

activity_main.xml:

1. Pada line [1], syntax `<?xml version="1.0" encoding="utf-8"?>` sebagai deklarasi XML.
2. Pada line [2] sampai [7], merupakan root layout, coordinator layout. Dimana coordinatorLayout merupakan layout fleksibel yang bisa bekerja sama dengan elemen-elemen seperti snackbar dan lain-lain. Kemudian ada `id = "coordinatorlayout"` sebagai anchor untuk snackbar di MainActivity.kt. terakhir `padding = "24dp"` sebagai pengasih jarak dari sisi layer.
3. Pada line [9] sampai line [13], syntax `<LinearLayout` digunakan untuk Menyusun elemen dari atas ke bawah dan `android:gravity = "center"` untuk membuat konten terpusat secara horizontal.
4. Pada line [15] sampai line [19], syntax `<LinearLayout` digunakan untuk menyusun dua buah dadu secara horizontal dan `marginBottom` agar dadu tidak menempel ke tombol.
5. Pada line [21] sampai line [26], syntax `<ImageView` digunakan untuk menampilkan gambar dadu awal (`dice_0`) dengan ukuran tetap 100dp, dan `marginEnd` untuk mengasih jarak ke dadu kedua.
6. Pada line [28] sampai line [32], syntax `<ImageView` digunakan untuk menampilkan gambar dadu kedua dengan menampilkan gambar dadu awal (`dice_0`), namun tanpa `marginEnd`.
7. Pada line [35] sampai line [39], syntax `<Button` digunakan untuk menampilkan tombol yang akan melempar dua dadu ketika diklik yang dihubungkan dengan `rollButton` di file Kotlin.

MainActivity.kt (Jetpack Compose):

1. Pada line [1], syntax `package com.example.diceroll` merupakan deklarasi nama package file Kotlin.
2. Pada line [3] sampai line [36], merupakan program import class dan library yang diperlukan, seperti `compose` (layout, button, image, dan sebagainya), `coroutines` untuk snackbar, dan lain-lain.

3. Pada line [38] sampai line [47], syntax `class MainActivity : ComponenttActivity() {}` merupakan activity untuk Jetpack Compose yang berisi program UI penuh di layer, mengatur UI dengan compose, dan program untuk tema dan memanggil composable utama.
4. Pada line [49] sampai line [53], merupakan fungsi utama composable dan dapat dipreview di Android Studio dengan adanya `@Preview`.
5. Pada line [63] sampai line [64], merupakan fungsi utama UI yang berfungsi untuk menampilkan dua dadu dan satu tbutton roll.
6. Pada line [59] sampai line [60], merupakan program untuk menyimpan hasil dadu sehingga Compose tahu kapan UI harus di-recompose.
7. Pada line [65] sampai line [66], syntax `val snackbarHostState` merupakan variable yang digunakan untuk mengelola notifikasi (snackbar), kemudian `val coroutineScope` digunakan untuk menampilkan snackbar secara asynchronous.
8. Pada line [68] sampai line [86], merupakan program untuk menentukan gambar mana yang akan ditampilkan berdasarkan angka `result1` dan `result2`.
9. Pada line [88] sampai line [110], syntax `Sacffold()` merupakan kerangka UI Compose, `Row()` merupakan program untuk menaruh duan gambar dadu secara horizontal, `Spacer()` program untuk memberi jarak vertikal, dan `Button()` program untuk button yang ketika diklik akan mengacak angka secara random dan menampilkan Snackbar dengan pesan.
10. Pada line [128] sampai line [137], merupakan fungsi button yang berisi program ketika button diklik untuk mengacak dua angka dadu dari 1-6, kemudian nilai akan disimpan ke dalam `result1` dan `result2` yang akan digunakan untuk menentukan gambar dadu. Kemudian, menentukan pesan yang sesuai dengan kondisi apakah angka berbeda atau sama (double) yang nilainya akan disipan ke dalam variable `message`. `coroutineScope.lunch` merupakan program untuk membuat coroutine baru yang bisa berjalan secara asynchronous tanpa menghambat UI. Di dalamnya terdapat program agar Ketika masih ada snackbar yang muncul sebelumnya akan ditutup terlebih dahulu agar tidak tumpeng tindih, Ketika ada `dismissJob` sebelumnya yang aktif akan dibatalkan terlebih dahulu untuk mencegah banyak `delay()`, membuat

coroutine baru dan menyimpannya ke dalam dismissJob yang akan memrogram waktu selama 2 detik sebelum lanjut ke baris berikutnya, dan akan ditutup secara otomatis.

11. Pada line [138], syntax `Text (stringResource (R.string.roll) )` digunakan untuk menampilkan teks tombol dari strings.xml .

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/rizkiirr/PraktikumMobile/tree/main/Modul%201>

MODUL 2: Android Layout with Compose

SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 10. Tampilan Awal Aplikasi

Calculate Tip

 Bill Amount
60

 Tip Percentage
15%

15%

18%

20%

Gambar 11. Tampilan Dadu Setelah Diroll

Calculate Tip

 Bill Amount
60

 Tip Percentage
18%

Round up tip? ☒

Tip Amount:
\$11.00

Gambar 12. Tampilan Roll Dadu Double

E. Source Code

XML:

MainActivity.kt

Table 4. Source Code Jawaban Soal 1 XML

1	<code>package</code>	<code>com.example.tipcalculatorxml</code>
2		
3	<code>import</code>	<code>android.icu.text.NumberFormat</code>
4	<code>import</code>	<code>android.os.Bundle</code>
5	<code>import</code>	<code>android.widget.AdapterView</code>
6	<code>import</code>	<code>android.widget.AutoCompleteTextView</code>
7	<code>import</code>	<code>androidx.appcompat.app.AppCompatActivity</code>
8	<code>import</code>	<code>com.example.tipcalculatorxml.databinding.ActivityMainBinding</code>
9	<code>import</code>	<code>androidx.core.widget.addTextChangedListener</code>
10	<code>import</code>	<code>java.util.Locale</code>
11	<code>import</code>	<code>kotlin.math.ceil</code>
12		
13	<code>class</code>	<code>MainActivity : AppCompatActivity() {</code>
14		
15	<code>private lateinit var</code>	<code>binding: ActivityMainBinding</code>
16	<code>private val</code>	<code>tipPercentage = listOf ("15%", "18%", "20%")</code>
17	<code>private var</code>	<code>selectedTipPercentage = 15.0</code>
18	<code>private var</code>	<code>roundUp = false</code>
19		
20	<code>override fun onCreate(savedInstanceState: Bundle?)</code>	<code>{</code>
21	<code>super.onCreate(savedInstanceState)</code>	
22	<code>binding = ActivityMainBinding.inflate(layoutInflater)</code>	
23	<code>setContentView(binding.root)</code>	
24		
25	<code>setupTipPercentageInput()</code>	
26	<code>setupBillInput()</code>	
27	<code>setupRoundUpTip()</code>	

```

28     }
29
30     private fun setupTipPercentageInput() {
31         val adapter = ArrayAdapter(this,
32             android.R.layout.simple_dropdown_item_1line, tipPercentage)
33
34         binding.tipPercentageInput.setAdapter(adapter)
35
36         binding.tipPercentageInput.setOnFocusChangeListener {
37             view, hasFocus ->
38             if (hasFocus) {
39                 (view as?
40                     AutoCompleteTextView)?.showDropDown()
41             }
42         }
43
44         binding.tipPercentageInput.setOnItemClickListener { _,
45             position, _ ->
46             selectedTipPercentage =
47             tipPercentage[position].dropLast(1).toDouble()
48             val billAmount =
49             binding.billAmountInput.text.toString().toDoubleOrNull() ?:
50             0.0
51             updateCalculatedTip(billAmount)
52         }
53     }
54
55     private fun setupBillInput() {
56         binding.billAmountInput.addTextChangedListener { editable ->
57             val billAmount =
58             editable?.toString()?.toDoubleOrNull() ?: 0.0
59             updateCalculatedTip(billAmount)
60         }
61     }

```

54	
55	<code>private fun setupRoundUpTip() {</code>
56	<code>binding.roundUpSwitch.setOnCheckedChangeListener {_,</code>
57	<code>isChecked</code>
57	<code>roundUp = isChecked</code>
58	<code>val billAmount =</code>
	<code>binding.billAmountInput.text.toString().toDoubleOrNull() ?:</code>
	<code>0.0</code>
59	<code>updateCalculatedTip(billAmount)</code>
60	<code>}</code>
61	<code>}</code>
62	
63	<code>private fun updateCalculatedTip(billAmount: Double) {</code>
64	<code>var tip = selectedTipPercentage / 100 * billAmount</code>
65	<code>if (roundUp) {</code>
	<code>tip = ceil(tip)</code>
	<code>}</code>
66	<code>val formattedTip =</code>
	<code>NumberFormat.getCurrencyInstance(Locale.US).format(tip)</code>
67	<code>binding.calculatedTip.text =</code>
	<code>getString(R.string.tip_amount) + " " + formattedTip</code>
68	<code>}</code>
69	<code>}</code>

activity_main.xml

1	<code><?xml version="1.0" encoding="utf-8"?></code>
2	<code><androidx.constraintlayout.widget.ConstraintLayout</code>
3	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4	<code>xmlns:app="http://schemas.android.com/apk/res-auto"</code>
5	<code>xmlns:tools="http://schemas.android.com/tools"</code>
6	<code>android:id="@+id/main"</code>
7	<code>android:layout_width="match_parent"</code>
8	<code>android:layout_height="match_parent"</code>

9	
10	tools:context=".MainActivity">
11	
12	<TextView
13	android:id="@+id/title_text"
14	android:layout_width="wrap_content"
15	android:layout_height="wrap_content"
16	android:layout_marginTop="40dp"
17	android:text="@string/calculate_tip"
18	android:textSize="20sp"
19	app:layout_constraintBottom_toBottomOf="parent"
20	app:layout_constraintEnd_toEndOf="parent"
21	app:layout_constraintHorizontal_bias="0.109"
22	app:layout_constraintStart_toStartOf="parent"
23	app:layout_constraintTop_toTopOf="parent"
24	app:layout_constraintVertical_bias="0.034" />
25	
26	<com.google.android.material.textfield.TextInputLayout
27	android:id="@+id/bill_amount_layout"
28	android:layout_width="0dp"
29	android:layout_height="wrap_content"
30	android:layout_marginTop="20dp"
31	android:layout_marginStart="32dp"
32	android:layout_marginEnd="32dp"
33	app:layout_constraintTop_toBottomOf="@+id/title_text"
34	app:layout_constraintStart_toStartOf="parent"
35	app:layout_constraintEnd_toEndOf="parent"
36	app:startIconDrawable="@drawable/money">
37	
38	
39	<com.google.android.material.textfield.TextInputEditText
40	android:id="@+id/billAmountInput"
41	android:layout_width="match_parent"
42	android:layout_height="wrap_content"

```

43         android:hint="@string/bill_amount"
44         android:importantForAccessibility="yes"
45         android:paddingStart="50dp"
46         android:inputType="numberDecimal"                                />
47     </com.google.android.material.textfield.TextInputLayout>
48
49     <com.google.android.material.textfield.TextInputLayout
50         android:id="@+id/tip_percentage_layout"
51         android:layout_width="0dp"
52         android:layout_height="wrap_content"
53         android:layout_marginTop="20dp"
54         android:layout_marginStart="32dp"
55         android:layout_marginEnd="32dp"
56         android:hint="@string/tip_option"
57         app:startIconDrawable="@drawable/baseline_percent_24"
58
59         app:layout_constraintTop_toBottomOf="@id/bill_amount_layout"
60         app:layout_constraintStart_toStartOf="parent"
61         app:layout_constraintEnd_toEndOf="parent">
62
63         <AutoCompleteTextView
64             android:id="@+id/tipPercentageInput"
65             android:layout_width="match_parent"
66             android:layout_height="55dp"
67             android:inputType="none"
68             android:focusable="true"
69             android:cursorVisible="false"
70             android:clickable="true"
71             android:dropDownHeight="wrap_content"
72             android:dropDownVerticalOffset="10dp"
73             android:importantForAccessibility="yes"
74             android:paddingStart="50dp"                                />
75     </com.google.android.material.textfield.TextInputLayout>
76     <LinearLayout

```


77	android:id="@+id/round_up_row"
78	android:layout_width="0dp"
79	android:layout_height="wrap_content"
80	android:orientation="horizontal"
81	
82	app:layout_constraintTop_toBottomOf="@id/tip_percentage_layout"
83	app:layout_constraintStart_toStartOf="parent"
84	app:layout_constraintEnd_toEndOf="parent"
85	android:layout_marginTop="32dp"
86	android:paddingStart="16dp"
87	android:paddingEnd="16dp"
88	android:gravity="center_vertical">
89	
90	<TextView
91	android:id="@+id/round_up_text"
92	android:layout_width="139dp"
93	android:layout_height="match_parent"
94	android:layout_gravity="start"
95	android:layout_marginStart="15dp"
96	android:text="@string/round_up_tip"
97	android:textSize="18sp" />
98	
99	<Switch
100	android:id="@+id/roundUpSwitch"
101	android:layout_width="wrap_content"
102	android:layout_height="wrap_content"
103	android:minWidth="48dp"
104	android:minHeight="48dp"
105	android:layout_marginStart="150dp"
106	android:importantForAccessibility="yes" />
107	</LinearLayout>
108	
109	<TextView
110	android:id="@+id/calculated_tip"

111	android:layout_width="wrap_content"
112	android:layout_height="wrap_content"
113	android:layout_marginTop="36dp"
114	android:text="@string/tip_amount"
115	android:textSize="25sp"
116	app:layout_constraintEnd_toEndOf="parent"
117	app:layout_constraintHorizontal_bias="0.181"
118	app:layout_constraintStart_toStartOf="parent"
119	app:layout_constraintTop_toBottomOf="@id/round_up_row"
120	/>
121	
122	</androidx.constraintlayout.widget.ConstraintLayout>

Table 5. Source Code Jawaban Soal 1 XML

Jetpack Compose:

MainActivity.kt

Table 6. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.tipcalculatorcompose
2	
3	import androidx.compose.material3.ExperimentalMaterial3Api
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.enableEdgeToEdge
8	import androidx.compose.material.icons.Icons
9	import androidx.annotation.StringRes
10	import androidx.compose.foundation.layout.Arrangement
11	import androidx.compose.foundation.layout.Column
12	import androidx.compose.foundation.layout.Row
13	import androidx.compose.foundation.layout.Spacer
14	import androidx.compose.foundation.layout.fillMaxSize
15	import androidx.compose.foundation.layout.fillMaxWidth
16	import androidx.compose.foundation.layout.height

17	import androidx.compose.foundation.layout.padding
18	import androidx.compose.foundation.rememberScrollState
19	import androidx.compose.foundation.text.KeyboardOptions
20	import androidx.compose.foundation.verticalScroll
21	import androidx.compose.material.icons.filled.Money
22	import androidx.compose.material.icons.filled.Percent
23	import androidx.compose.material3.DropdownMenuItem
24	import androidx.compose.material3.ExposedDropdownMenuBox
25	import androidx.compose.material3.ExposedDropdownMenuDefaults
26	import androidx.compose.material3.Icon
27	import androidx.compose.material3.MaterialTheme
28	import androidx.compose.material3.Surface
29	import androidx.compose.material3.Switch
30	import androidx.compose.material3.Text
31	import androidx.compose.material3.TextField
32	import androidx.compose.runtime.Composable
33	import androidx.compose.runtime.getValue
34	import androidx.compose.runtime.mutableStateOf
35	import androidx.compose.runtime.remember
36	import androidx.compose.runtime.setValue
37	import androidx.compose.ui.Alignment
38	import androidx.compose.ui.Modifier
39	import androidx.compose.ui.res.stringResource
40	import androidx.compose.ui.text.input.ImeAction
41	import androidx.compose.ui.text.input.KeyboardType
42	import androidx.compose.ui.tooling.preview.Preview
43	import androidx.compose.ui.unit.dp
44	import
	com.example.tipcalculatorcompose.ui.theme.TipCalculatorComposeTheme
45	import java.text.NumberFormat
46	
47	class MainActivity : ComponentActivity() {
48	override fun onCreate(savedInstanceState: Bundle?) {
49	super.onCreate(savedInstanceState)

```

50         enableEdgeToEdge()
51         setContent {
52             TipCalculatorComposeTheme {
53                 Surface (
54                     modifier = Modifier.fillMaxSize(),
55                 ) {
56                     TipCalculatorLayout()
57                 }
58             }
59         }
60     }
61 }
62 @OptIn(ExperimentalMaterial3Api::class)
63 @Composable
64 fun TipCalculatorLayout() {
65     var amountInput by remember { mutableStateOf("") }
66     var calculatedTip by remember { mutableStateOf("") }
67     var roundUp by remember { mutableStateOf(false) }
68     var expanded by remember { mutableStateOf(false) }
69
70     val optionsTipPercentage = listOf("15%", "18%", "20%")
71     var selectedOption by remember {
72         mutableStateOf(optionsTipPercentage[0]) }
73
74     Column(
75         modifier = Modifier
76             .padding(horizontal = 40.dp)
77             .verticalScroll(rememberScrollState()),
78         horizontalAlignment = Alignment.CenterHorizontally,
79         verticalArrangement = Arrangement.Center
80     ) {
81         Text(
82             text = stringResource(R.string.calculate_tip),
83             modifier = Modifier

```

```

83         .padding(bottom = 16.dp, top = 40.dp)
84         .align(alignment = Alignment.Start)
85     )
86
87     EditNumberField(
88         label = R.string.bill_amount,
89         value = amountInput,
90         onValueChanged = {
91             amountInput = it
92             calculatedTip = calculateTipInput(it, selectedOption,
roundUp)
93         },
94         leadingIcon = { Icon(Icons.Filled.Money,
contentDescription = null) },
95         keyboardOptions = KeyboardOptions(
96             keyboardType = KeyboardType.Number,
97             imeAction = ImeAction.Next
98         ),
99         modifier = Modifier
100             .padding(bottom = 32.dp)
101             .fillMaxWidth()
102     )
103
104     ExposedDropdownMenuBox(
105         expanded = expanded,
106         onExpandedChange = { expanded = !expanded }
107     ) {
108         TextField(
109             readOnly = true,
110             value = selectedOption,
111             onValueChange = {},
112             label = { Text(stringResource(R.string.tip_option))
},
113

```

114	leadingIcon = { Icon(Icons.Default.Percent,
	contentDescription = null)},
115	trailingIcon = {
116	
117	ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded)
118	},
119	colors =
	ExposedDropdownMenuDefaults.textFieldColors(),
120	modifier = Modifier.menuAnchor().fillMaxWidth()
121	)
122	
123	ExposedDropdownMenu(
124	expanded = expanded,
125	onDismissRequest = { expanded = false }
126	) {
127	optionsTipPercentage.forEach { selectionOption ->
128	DropdownMenuItem(
129	text = { Text(text = selectionOption) },
130	onClick = {
131	selectedOption = selectionOption
132	expanded = false
133	calculatedTip =
	calculateTipInput(amountInput, selectedOption, roundUp)
134	}
135	)
136	}
137	}
138	}
139	
140	RoundTheTipRow(
141	roundUp = roundUp,
142	onRoundUpChanged = {
143	roundUp = it
144	

```

145         calculatedTip      =      calculateTipInput (amountInput,
selectedOption, it)
146     },
147     modifier = Modifier.padding(bottom = 32.dp)
148 )
149
150     Text (
151         text      =      stringResource(R.string.tip_amount,
calculatedTip),
152         style = MaterialTheme.typography.displaySmall
153     )
154
155     Spacer(modifier = Modifier.height(150.dp))
156 }
157 }
158
159 private fun calculateTipInput(amountInput: String, selectedTipText:
160 String, roundUp: Boolean): String {
161     val amount = amountInput.toDoubleOrNull() ?: 0.0
162     val tipPercent = selectedTipText.dropLast(1).toDoubleOrNull() ?:
15.0
163     return calculateTip(amount, tipPercent, roundUp)
164 }
165
166 @Composable
167 fun EditNumberField(
168     value: String,
169     @StringRes label: Int,
170     leadingIcon: @Composable (() -> Unit)? = null,
171     onValueChanged: (String) -> Unit,
172     keyboardOptions: KeyboardOptions,
173     modifier: Modifier = Modifier
174 ) {
175

```

```

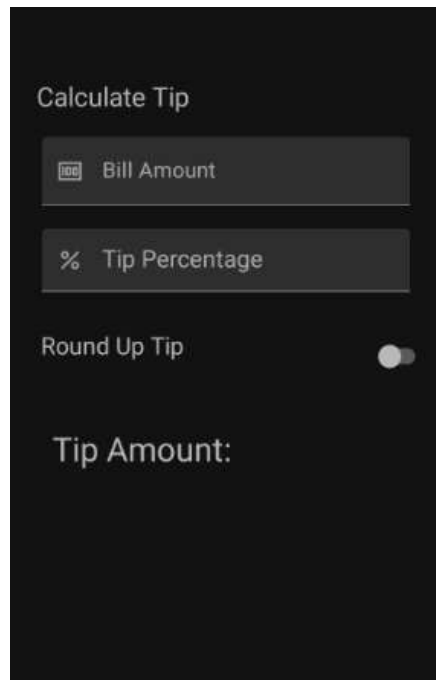
176     TextField(
177         value = value,
178         singleLine = true,
179         modifier = modifier,
180         onValueChange = onValueChanged,
181         label = { Text(stringResource(label)) },
182         leadingIcon = leadingIcon,
183
184         keyboardOptions = keyboardOptions
185     )
186 }
187
188 @Composable
189 fun RoundTheTipRow(
190     roundUp: Boolean,
191     onRoundUpChanged: (Boolean) -> Unit,
192     modifier: Modifier = Modifier
193 ) {
194     Row(
195         modifier = modifier
196             .fillMaxWidth()
197             .height(48.dp),
198         verticalAlignment = Alignment.CenterVertically,
199         horizontalArrangement = Arrangement.SpaceBetween
200     ) {
201         Text(
202             text = stringResource(R.string.round_up_tip),
203             modifier = Modifier.weight(1f)
204         )
205         Switch(
206             checked = roundUp,
207             onCheckedChange = onRoundUpChanged,
208         )
209     }

```

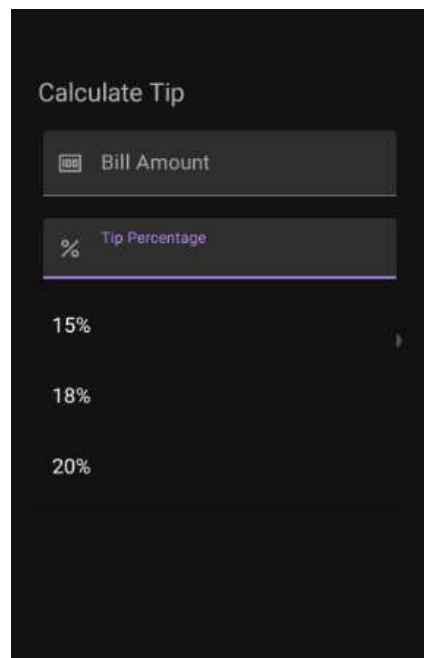


```
210 }
211
212 private fun calculateTip(amount: Double, tipPercent: Double = 15.0,
    roundUp: Boolean): String{
213     var tip = tipPercent / 100 * amount
214     if (roundUp){
215         tip = kotlin.math.ceil(tip)
216     }
217
218     return
219     NumberFormat.getCurrencyInstance(java.util.Locale.US).format(tip)
220 }
221
222 @Preview(showBackground = true)
223 @Composable
224 fun TipCalculatorLayoutPreview(){
225     TipCalculatorComposeTheme{
226         TipCalculatorLayout()
227     }
228 }
```

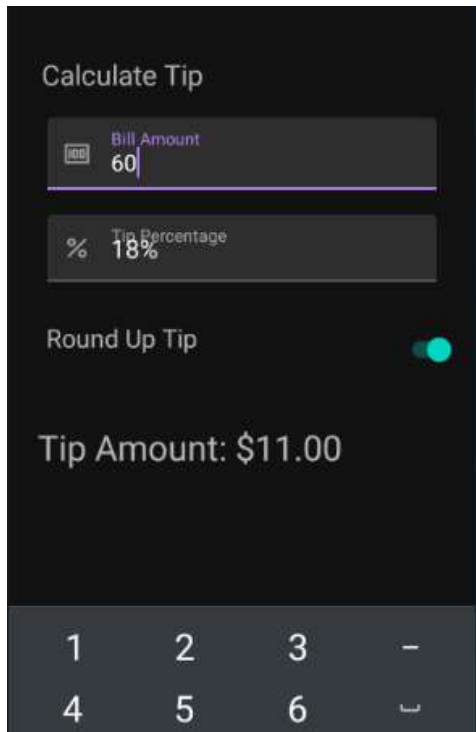
F. Output Program



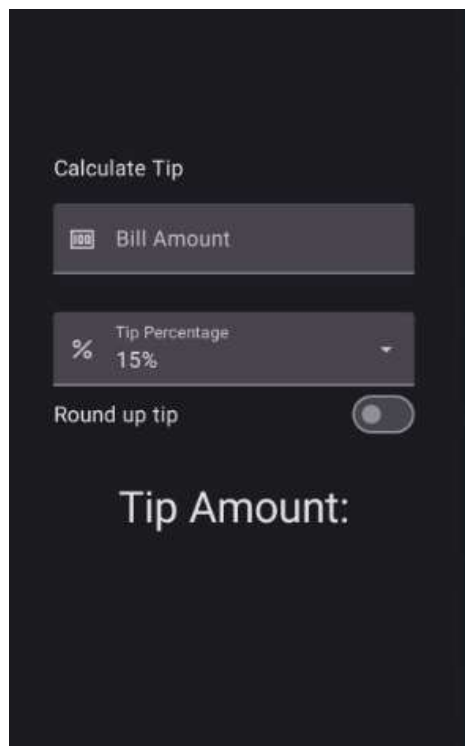
Gambar 13. Tampilan Awal Aplikasi XML



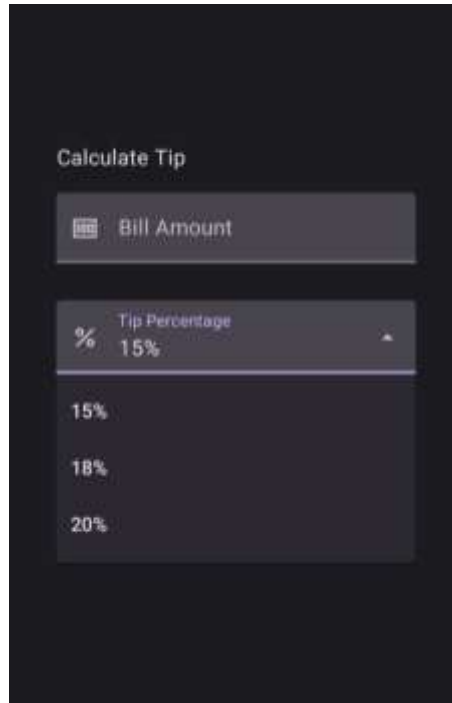
Gambar 14. Tampilan Pilihan Persentase Tip XML



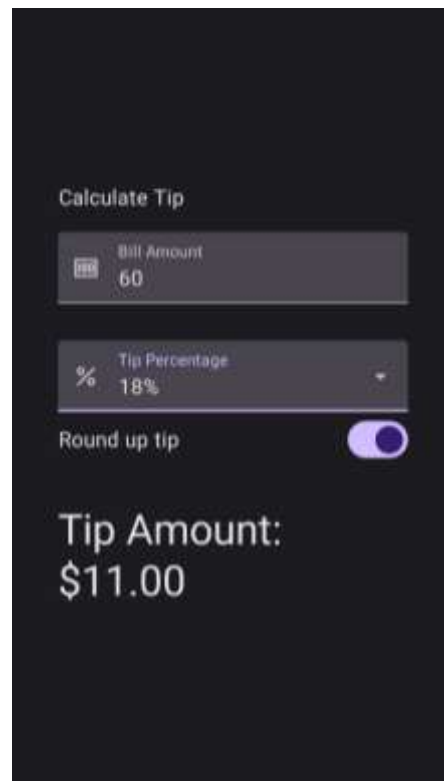
Gambar 15. Tampilan Aplikasi Setelah Dijalankan XML



Gambar 16. Tampilan Awal Aplikasi Jetpack Compose



Gambar 17. Tampilan Pilihan Persentase Tip Jetpack Compose



Gambar 18. Tampilan Aplikasi Setelah Dijalankan Jetpack Compose

G. Pembahasan

MainActivity.kt (XML):

1. Pada line [1], syntax `package package com.example.tipcalculatorxml` merupakan deklarasi nama package file Kotlin.
2. Pada line [3] sampai line [11],
`import android.icu.text.NumberFormat, import android.os.Bundle,`
`import android.widget.AdapterView, import`
`android.widget.AutoCompleteTextView, import`
`androidx.appcompat.app.AppCompatActivity, import`
`com.example.tipcalculatorxml.databinding.ActivityMainBinding,`
`import`
`androidx.core.widget.addTextChangedListener, import`
`java.util.Locale, import kotlin.math.ceil` merupakan program import class dan library yang diperlukan, seperti format mata uang, menangani lifecycle onCreate, dropdown, base class activity, viewbinding, perubahan teks, format lokal, dan pembulatan ke atas.
3. Pada line [14], syntax `class MainActivity : AppCompatActivity()`
`{}` merupakan deklarasi kelas bernama MainActivity yang extend dari AppCompatActivity.
4. Pada line [16], `private lateinit var binding:`
`ActivityMainBinding` mendeklarasikan objek binding untuk mengakses view dari XML via viewBinding.
5. Pada line [17], `private val tipPercentage = listOf ("15%",`
`"18%", "20%")` merupakan list opsi persentase tip dalam bentuk string.
6. Pada line [18], `private var selectedTipPercentage = 15.0`
`private var roundUp = false` merupakan program untuk menyimpan persentase tip terpilih dan status pembulatan tip.
7. Pada line [21] sampai line [29], `override fun onCreate` program untuk menginisialisasi binding, menampilkan UI, dan setup fungsi input.

8. Pada line [31] sampai line [48], private fun setupTipPercentageInput() { merupakan fungsi untuk menangani dropdown tip.
9. Pada line [33] sampai line [34], val adapter = ArrayAdapter(this, android.R.layout.simple_dropdown_item_1line, tipPercentage) binding.tipPercentageInput.setAdapter(adapter) membuat dan menetapkan adapter ke AutoCompleteTextView agar tampil sebagai dropdown.
10. Pada line [43] sampai line [48], binding.tipPercentageInput.setOnItemClickListener { _, _, position, _ -> selectedTipPercentage = tipPercentage[position].dropLast(1).toDouble() val billAmount = binding.billAmountInput.text.toString().toDoubleOrNull() ?: 0.0 updateCalculatedTip(billAmount) } berfungsi saat user memilih item, simpan nilai persentase, ambil bill saat ini, lalu update hasil tip.
11. Pada line [51] sampai line [56], private fun setupBillInput() { a. binding.billAmountInput.addTextChangedListener { editable -> val billAmount = editable?.toString()?.toDoubleOrNull() ?: 0.0 updateCalculatedTip(billAmount) } } berfungsi saat user mengetik angka pada bill, ambil nilainya dan update hasil tip.
12. Pada line [58] sampai line [64], private fun setupRoundUpTip() { binding.roundUpSwitch.setOnCheckedChangeListener { _, isChecked -> roundUp = isChecked val billAmount = binding.billAmountInput.text.toString().toDoubleOrNull() ?: 0.0 updateCalculatedTip(billAmount) } } berfungsi saat switch round-up diubah, simpan statusnya dan hitung ulang tip.
13. Pada line [66] sampai line [72], private fun updateCalculatedTip(billAmount: Double) { var tip = selectedTipPercentage / 100 * billAmount if (roundUp) {

```
tip = ceil(tip) } val formattedTip =
NumberFormat.getCurrencyInstance(Locale.US).format(tip)
binding.calculatedTip.text =
getString(R.string.tip_amount) + " " + formattedTip}
berfungsi untuk menghitung tip, mengalikan bill dengan persentase, jika perlu
dibulatkan, gunakan ceil(), dan memformat ke mata uang dan tampilkan ke UI.
```

activity_main.xml:

1. Pada line [1], syntax `<?xml version="1.0" encoding="utf-8"?>` sebagai deklarasi XML.
2. Pada line [2], `<androidx.constraintlayout.widget.ConstraintLayout` merupakan Layout utama menggunakan `ConstraintLayout`, memungkinkan penempatan komponen berdasarkan hubungan antar elemen.
3. Pada line [5] sampai line [8], digunakan untuk ID dari layout utama, ukuran layout memenuhi seluruh layar (`match_parent`), dan hanya untuk Android Studio agar preview tahu class aktivitas.
4. Pada line [10] sampai line [22], syntax berfungsi untuk judul layar: **"Calculate Tip"**, memberi ruang dari atas, ukuran teks sedang, menyematkan posisi ke atas, kiri, kanan.
5. Pada line [24], syntax berfungsi untuk membungkus untuk `TextInputEditText`, memberikan fitur Material seperti ikon, label, animasi input.
6. Pada line [36] sampai line [43], syntax berfungsi untuk membuat kolom input angka desimal untuk jumlah tagihan, hint tampil saat input kosong, dan memunculkan keyboard angka dengan titik desimal.
7. Pada line [47] sampai line [55], sama seperti sebelumnya, tapi untuk dropdown tip dan menambahkan ikon persen di sisi kiri.
 - a. Pada line [60] sampai line [72], `AutoCompleteTextView` digunakan untuk memilih dari persentase tip yang tersedia, agar terlihat seperti dropdown, bukan input biasa.
8. Pada line [75] sampai line [86], berfungsi sebagai baris horizontal yang berisi label dan Switch.
9. Pada line [88] sampai line [95], berfungsi untuk membuat label: **"Round Up Tip"**.

10. Pada line [99] sampai line [106], berfungsi untuk membuat tombol Switch untuk memilih apakah tip dibulatkan.
11. Pada line [111] sampai line [121], berfungsi untuk menampilkan hasil kalkulasi tip, misalnya: Tip Amount: \$3.45.
12. Pada line [123], menutup root layout constraintLayout.

MainActivity.kt (Jetpack Compose):

1. Pada line [1], `package com.example.tipcalculatorcompose` merupakan deklarasi nama package file Kotlin.
2. Pada line [3] sampai line [45], merupakan program import class dan library yang diperlukan, seperti layout, material icons, state management, dan sebagainya.
3. Pada line [47] sampai line [61], `class MainActivity : ComponenttActivity() {}` merupakan activity untuk Jetpack Compose yang berisi program UI penuh di layer, mengatur UI dengan compose, dan program untuk tema dan memanggil composable utama.
4. Pada line [48], `override fun onCreate(savedInstanceState: Bundle?) {` merupakan override fungsi onCreate untuk menginisialisasi UI saat activity dibuat.
5. Pada line [50], `enableEdgeToEdge()` merupakan program untuk memanfaatkan seluruh layar.
6. Pada line [51] sampai line [59], `setContent{` untuk meneentukan isis dari UI yang menggunakan Jetpack Compose.
7. Pada line [52] sampai line [58], `TipCalculatorComposeTheme {` merupakan tema kustom aplikasi.
8. Pada line [53] sampai line [55], `Surface( modifier = Modifier)` merupakan program untuk menampilkan elemen latar belakang yang mengisi seluruh layar.
9. Pada line [56], `TipCalculatorLayout()` berfungsi untuk menampilkan layout utama kalkulator tip.

10. Pada line [65] sampai line [155], merupakan fungsi composable utama untuk menampilkan UI kalkulator tip.
11. Pada line [66] sampai line [72], merupakan program untuk menyimpan input bill amount, menyimpan hasil perhitungan tip, menyimpan status pembulatan ke atas, menyimpan status apakah dropdown tip terbuka atau tidak, dan opsi persentase tip.
12. Pada line [74] sampai line [79], syntax merupakan membuat layout kolom yang scrollable dan rata tengah..
13. Pada line [81] sampai line [86], merupakan fungsi untuk menampilkan teks judul “Calculate Tip”
14. Pada line [88] sampai line [103], memanggil composable untuk input angka atau jumlah bill.
15. Pada line [105] sampai line [137], membuat dropdown untuk memilih tip percentage.
16. Pada line [109] sampai line [120], menampilkan textfield read-only sebagai anchor dropdown.
17. Pada line [122] sampai line [136], berfungsi untuk menampilkan pilihan dalam dropdown.
18. Pada line [139] sampai line [146], berfungsi untuk memanggil composable switch untuk mengaktifkan atau menonaktifkan pembulatan.
19. Pada line [148] sampai line [151], berfungsi untuk menampilkan hasil kalkulasi tip
20. Pada line [153], berfungsi untuk memberi jarak di bagian bawah.
21. Pada line [157] sampai line [161], fungsi pembantu untuk menghitung tip dari input string.
22. Pada line [163] sampai line [171], merupakan reusable composable untuk menginput angka dengan label dan icon.
23. Pada line [185] sampai line [190], menampilkan teks dan switch untuk pembulatan ke atas tip.
24. Pada line [209] sampai line [216], untuk menghitung besarnya tip dan mengubahnya ke format mata uang (USD)
25. Pada line [218] sampai line [223], menampilkan pratinjau layout utama di android studio preview.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

H. Pembahasan

Perbedaan antara implemetasi XML dan Jetpack Compose adalah terletak pada struktur pemrogramannya. Pada XML pembuatan antarmuka atau tampilan aplikasi berada di file.xml, sedangkan untuk pembuatan logika program ada di file.kt yang berbasis Kotlin. Kelebihan dari XML ini adalah karena telah digunakan sejak lama XML lebih matang dalam syntax pemrograman dan sudah banyak didokumentasi. Kemudian, XML juga akan lebih mudah dalam pengembangan aplikasi karena file terpisah antara program tampilan dan logika (jika tampilan tidak harus berubah secara real-time). Kekurangannya, sulit jika harus mengubah tampilan yang dinamis dan harus selalu berpindah-pindah antara file tampilan dan logika.

Sebaliknya, pada Jetpack Compose pembuatan program tampilan dan logika aplikasi tergabung di dalam satu file berbasis Kotlin. Kelebihannya adalah program lebih singkat, mudah digunakan jika memerlukan perubahan tampilan yang dinamis, dan terintegrasi dengan fitur-fitur modern. Namun kekurangannya adalah karena masih baru jadi tidak semua library disupport, dan tidak semua fitur klasik android tersedia.

I. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

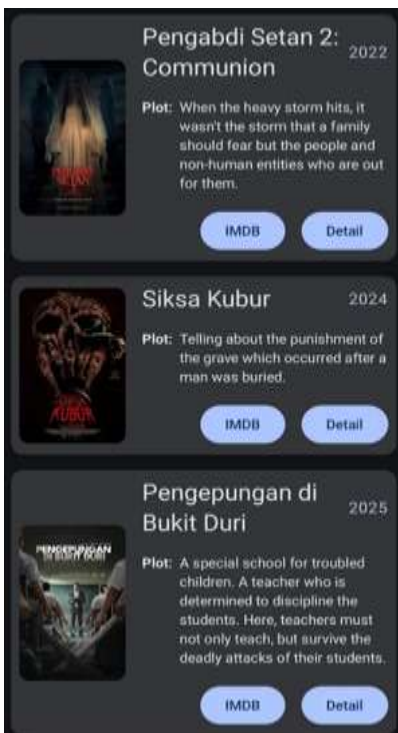
<https://github.com/rizkiirr/PemrogramanMobile.git>

MODUL 3: Build a Scrollable List

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding



Gambar 19. UI List



Gambar 20. UI Detail

J. Source Code

XML:

MainActivity.kt

Table 7. Source Code Jawaban Soal 1 XML

1	package com.example.scrollablelistxml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.recyclerview.widget.LinearLayoutManager
6	import androidx.recyclerview.widget.RecyclerView
7	
8	class MainActivity : AppCompatActivity() {
9	override fun onCreate(savedInstanceState: Bundle?) {
10	super.onCreate(savedInstanceState)
11	setContentView(R.layout.activity_main)
12	
13	val recyclerView: RecyclerView =
14	findViewById(R.id.recyclerView)
15	recyclerView.layoutManager =
16	LinearLayoutManager(this)
17	recyclerView.adapter = ChocolateAdapter(this,
18	DataProvider.chocolateList)
19	}
20	}

Chocolate.kt

Table 8. Source Code Jawaban Soal 1 XML

1	package com.example.scrollablelistxml
2	
3	data class Chocolate(
4	val id: Int,
5	val name: String,
6	val description: String,
7	val chocolateImageId: Int,
8	val btnLink: String
9	)

ChocolateAdapter.kt

Table 9. Source Code Jawaban Soal 1 XML

1	package com.example.scrollablelistxml
2	
3	import android.content.Context
4	import android.content.Intent
5	import android.net.Uri
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import android.widget.*
10	import androidx.recyclerview.widget.RecyclerView
11	
12	class ChocolateAdapter(private val context: Context, private
13	val chocolateList: List<Chocolate>) :
14	RecyclerView.Adapter<ChocolateAdapter.ViewHolder>() {
15	
16	class ViewHolder(view: View) :
	RecyclerView.ViewHolder(view) {
17	val image: ImageView =
	view.findViewById(R.id.imageChocolate)
18	val name: TextView = view.findViewById(R.id.textName)
	val description: TextView =
19	view.findViewById(R.id.textDescription)
	val btnImdb: Button =
20	view.findViewById(R.id.buttonImdb)
	val btnDetail: Button =
21	view.findViewById(R.id.buttonDetail)
	}
22	
23	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): ViewHolder {
24	

```

25         val view =
26     LayoutInflater.from(context).inflate(R.layout.row_coklat,
    parent, false)
27         return ViewHolder(view)
28     }
29
30     override fun getItemCount(): Int = chocolateList.size
31
32     override fun onBindViewHolder(holder: ViewHolder,
    position: Int) {
33         val chocolate = chocolateList[position]
34
35     holder.image.setImageResource(chocolate.chocolateImageId)
36         holder.name.text = chocolate.name
37         holder.description.text = chocolate.description
38
39         holder.btnImdb.setOnClickListener {
40             val intent = Intent(Intent.ACTION_VIEW,
    Uri.parse(chocolate.btnLink))
41             context.startActivity(intent)
42         }
43
44         holder.btnDetail.setOnClickListener {
45             val intent = Intent(context,
    DetailActivity::class.java).apply {
46                 putExtra("id", chocolate.id)
47             }
48             context.startActivity(intent)
49         }
50     }
51 }

```


DataAdapter.kt

Table 10. Source Code Jawaban Soal 1 XML

1	package com.example.scrollablelistxml
2	
3	object DataProvider {
4	val chocolateList = listOf(
5	Chocolate(1, "Dark Chocolate", "Dark chocolate is a form of chocolate made from cocoa solids, cocoa butter and sugar. " +
6	"It has a higher cocoa percentage than white chocolate, milk chocolate, and semisweet chocolate.", R.drawable.darkchocolate, "https://en.wikipedia.org/wiki/Dark_chocolate"),
7	Chocolate(2, "Milk Chocolate", "Milk chocolate is a form of solid chocolate containing cocoa, sugar and milk. " +
8	"Milk chocolate contains smaller amounts of cocoa solids than dark chocolates do, and (as with white chocolate) contains milk solids.", R.drawable.milkchocolate, "https://en.wikipedia.org/wiki/Milk_chocolate"),
9	Chocolate(3, "White Chocolate", "White chocolate is a chocolate made from cocoa butter, sugar and milk solids. " +
10	"It is ivory in color and lacks the dark appearance of most other types of chocolate as it does not contain the non-fat components of cocoa (cocoa solids).", R.drawable.whitechocolate, "https://en.wikipedia.org/wiki/White_chocolate"),
11	Chocolate(4, "Hazelnut Chocolate", "Hazelnut chocolate adds a new dimension to the flavor by combining the flavor of chocolate with the distinctive hazelnut nut. " +
12	"The hazelnut provides a savory, crunchy, and slightly sweet touch that complements the richness of the chocolate.", R.drawable.hazelnutchocolate, "https://en.wikipedia.org/wiki/Chocolate#Flavors"),
13	

14	Chocolate(5, "Mint Chocolate", "The hazelnut provides a savory, crunchy, and slightly sweet touch that complements the richness of the chocolate. " +
15	"Mint chocolate can be found in a wide variety of confectionery items, such as candy, mints, cookies, mint chocolate chip ice cream, hot chocolate, and others. ", R.drawable.mintchocolate,
16	"https://en.wikipedia.org/wiki/Chocolate#Flavors")
17	)
	}

DetailActivity.kt

Table 11. Source Code Jawaban Soal 1 XML

```
1 package com.example.scrollablelistxml
2
3 import android.os.Bundle
4 import android.widget.ImageView
5 import android.widget.TextView
6 import androidx.appcompat.app.AppCompatActivity
7
8 class DetailActivity : AppCompatActivity() {
9     override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         setContentView(R.layout.activity_detail)
12
13         val id = intent.getIntExtra("id", -1)
14         val chocolate = DataProvider.chocolateList.find { it.id ==
15 id }
16
17         chocolate?.let {
18
19 findViewById<ImageView>(R.id.imageChocolateDetail).setImageResourc
20 e(it.chocolateImageId)
21         findViewById<TextView>(R.id.textTitleDetail).text =
22 it.name
23
24 findViewById<TextView>(R.id.textDescriptionDetail).text =
25 it.description
26         }
27     }
28 }
```

activity_detail.kt

Table 12. Source Code Jawaban Soal 1 XML

1	<ScrollView
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:padding="16dp">
7	
8	<androidx.constraintlayout.widget.ConstraintLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<ImageView
13	android:id="@+id/imageChocolateDetail"
14	android:layout_width="0dp"
15	android:layout_height="240dp"
16	android:scaleType="centerCrop"
17	app:layout_constraintTop_toTopOf="parent"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintEnd_toEndOf="parent" />
20	
21	<TextView
22	android:id="@+id/textTitleDetail"
23	android:layout_marginTop="16dp"
24	android:textSize="24sp"
25	android:textStyle="bold"
26	android:layout_width="0dp"
27	android:layout_height="wrap_content"
28	
29	app:layout_constraintTop_toBottomOf="@id/imageChocolateDetail"
30	app:layout_constraintStart_toStartOf="parent"
31	app:layout_constraintEnd_toEndOf="parent" />

```

32
33         <TextView
34             android:id="@+id/textLabelPlot"
35             android:textStyle="bold"
36             android:layout_marginTop="8dp"
37             android:layout_width="wrap_content"
38             android:layout_height="wrap_content"
39
40 app:layout_constraintTop_toBottomOf="@id/textTitleDetail"
41             app:layout_constraintStart_toStartOf="parent" />
42
43         <TextView
44             android:id="@+id/textDescriptionDetail"
45             android:layout_width="0dp"
46             android:layout_height="wrap_content"
47             android:layout_marginTop="4dp"
48
49 app:layout_constraintTop_toBottomOf="@id/textLabelPlot"
50             app:layout_constraintStart_toStartOf="parent"
51             app:layout_constraintEnd_toEndOf="parent"
52             app:layout_constraintBottom_toBottomOf="parent"
53 />
54     </androidx.constraintlayout.widget.ConstraintLayout>
55 </ScrollView>

```

Table 13. Source Code Jawaban Soal 1 XML

1	<androidx.cardview.widget.CardView
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:card_view="http://schemas.android.com/apk/res-auto"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	card_view:cardCornerRadius="12dp">
9	
10	<androidx.constraintlayout.widget.ConstraintLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:padding="8dp">
14	
15	
16	<com.google.android.material.imageview.ShapeableImageView
17	android:id="@+id/imageChocolate"
18	android:layout_width="100dp"
19	android:layout_height="140dp"
20	android:scaleType="centerCrop"
21	android:layout_marginEnd="8dp"
22	app:layout_constraintTop_toTopOf="parent"
23	app:layout_constraintBottom_toBottomOf="parent"
24	app:layout_constraintStart_toStartOf="parent"
25	
26	app:shapeAppearanceOverlay="@style/RoundedImageStyle" />
27	
28	
29	<LinearLayout
30	android:id="@+id/infoLayout"
31	android:orientation="vertical"

32	android:layout_width="0dp"
33	android:layout_height="wrap_content"
34	android:layout_marginStart="8dp"
35	
36	app:layout_constraintStart_toEndOf="@id/imageChocolate"
37	
38	app:layout_constraintTop_toTopOf="@id/imageChocolate"
39	
40	app:layout_constraintBottom_toBottomOf="@id/imageChocolate"
41	app:layout_constraintEnd_toEndOf="parent">
42	
43	<TextView
44	android:id="@+id/textName"
45	android:layout_width="wrap_content"
46	android:layout_height="wrap_content"
47	android:textStyle="bold"
48	android:textSize="18sp" />
49	
50	<TextView
51	android:id="@+id/textDescription"
52	android:layout_width="wrap_content"
53	android:layout_height="wrap_content"
54	android:layout_marginTop="4dp" />
55	
56	<LinearLayout
57	android:layout_width="wrap_content"
58	android:layout_height="wrap_content"
59	android:orientation="horizontal"
60	android:layout_marginTop="8dp">
61	
62	<Button
63	android:id="@+id/buttonImdb"
64	android:layout_width="wrap_content"
65	android:layout_height="wrap_content"

66	android:text="IMDB" />
67	
68	<Button
69	android:id="@+id/buttonDetail"
70	android:layout_width="wrap_content"
71	android:layout_height="wrap_content"
72	android:text="Detail"
73	android:layout_marginStart="8dp" />
74	</LinearLayout>
75	</LinearLayout>
76	
77	</androidx.constraintlayout.widget.ConstraintLayout>
78	</androidx.cardview.widget.CardView>

activity_main.xml

Table 14. Source Code Jawaban Soal 1 XML

1	<LinearLayout
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:orientation="vertical"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent">
6	
7	<androidx.recyclerview.widget.RecyclerView
8	android:id="@+id/recyclerView"
9	android:layout_width="match_parent"
10	android:layout_height="match_parent"/>
11	</LinearLayout>

Jetpack Compose:

MainActivity.kt

Table 15. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.scrollablelistcompose
2	
3	import android.os.Bundle


```

4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.padding
7 import
  androidx.compose.material3.Scaffold
  ld
8 import androidx.compose.runtime.Composable
9 import androidx.compose.runtime.remember
10 import androidx.compose.ui.Modifier
11 import androidx.navigation.NavType
12 import androidx.navigation.compose.NavHost
13 import androidx.navigation.compose.composable
14 import androidx.navigation.compose.rememberNavController
15 import androidx.navigation.navArgument
16 import com.example.scrollablelistcompose.data.DataProvider
17 import
  com.example.scrollablelistcompose.ui.theme.ScrollAbleListCompose
  eTheme
18
19 class MainActivity : ComponentActivity() {
20     override fun onCreate(savedInstanceState: Bundle?) {
21         super.onCreate(savedInstanceState)
22         setContent {
23             ScrollAbleListComposeTheme {
24                 val navController = rememberNavController()
25                 MyApp(navController = navController)
26             }
27         }
28     }
29 }
30
31 @Composable
32 fun MyApp(navController: androidx.navigation.NavHostController)
33 {

```

```

34     val chocolates = remember { DataProvider.chocolateList }
35
36     Scaffold { innerPadding ->
37         NavHost(
38             navController = navController,
39             startDestination = "home",
40             modifier = Modifier.padding(innerPadding)
41         ) {
42             composable("home") {
43                 ScrollableListHomeContent(
44                     onDetailClick = { selected ->
45
46 navController.navigate("detail/${selected.no}")
47                     }
48                 )
49             }
50             composable(
51                 "detail/{id}",
52                 arguments = listOf(navArgument("id") { type =
NavType.IntType })
53             ) { backStackEntry ->
54                 val id = backStackEntry.arguments?.getInt("id")
55                 val chocolate = chocolates.find { it.no == id }
56                 chocolate?.let {
57                     DetailScreen(it)
58                 }
59             }
60         }
61     }
62 }

```

ChocolateListItem.kt

Table 16. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.scrollablelistcompose
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.Column
7	import androidx.compose.foundation.layout.Row
8	import androidx.compose.foundation.layout.Spacer
9	import androidx.compose.foundation.layout.fillMaxWidth
10	import androidx.compose.foundation.layout.height
11	import androidx.compose.foundation.layout.padding
12	import androidx.compose.foundation.layout.width
13	import androidx.compose.foundation.shape.RoundedCornerShape
14	import androidx.compose.material3.Button
15	import androidx.compose.material3.Card
16	import androidx.compose.material3.MaterialTheme
17	import androidx.compose.material3.Text
18	import androidx.compose.runtime.Composable
19	import androidx.compose.ui.Alignment
20	import androidx.compose.ui.Modifier
21	import androidx.compose.ui.draw.clip
22	import androidx.compose.ui.layout.ContentScale
23	import androidx.compose.ui.platform.LocalContext
24	import androidx.compose.ui.res.painterResource
25	import androidx.compose.ui.unit.dp
26	import com.example.scrollablelistcompose.data.Chocolate
27	
28	@Composable
29	fun ChocolateListItem(chocolate: Chocolate, onDetailClick:
30	(Chocolate) -> Unit) {
31	val context = LocalContext.current
32	

```

33     Card(
34         modifier = Modifier
35             .padding(8.dp)
36             .fillMaxWidth(),
37         shape = RoundedCornerShape(16.dp)
38     ) {
39         Row(
40             modifier = Modifier.padding(8.dp),
41             verticalAlignment = Alignment.CenterVertically
42         ) {
43
44             Image(
45                 painter = painterResource(id =
chocolate.chocolateImageId),
46                 contentDescription = null,
47                 contentScale = ContentScale.Crop,
48                 modifier = Modifier
49                     .width(100.dp)
50                     .height(140.dp)
51                     .clip(RoundedCornerShape(12.dp))
52             )
53
54             Spacer(modifier = Modifier.width(12.dp))
55
56             Column(
57                 modifier = Modifier
58                     .padding(16.dp)
59                     .weight(1f)
60             ) {
61                 Text(text = "${chocolate.title}")
62                 Text(text = "No: ${chocolate.no}")
63
64
65

```

66	Text(
67	text = "\${chocolate.description}",
68	style = MaterialTheme.typography.bodySmall,
69	modifier = Modifier.padding(top = 4.dp)
70	)
71	
72	Row(modifier = Modifier.padding(top = 8.dp)) {
73	Button(onClick = {
74	val intent = Intent(Intent.ACTION_VIEW,
75	Uri.parse(chocolate.btnLink))
76	context.startActivity(intent)
77	}) {
78	Text("IMDB")
79	}
80	Spacer(modifier =
81	Modifier.padding(horizontal = 4.dp))
82	Button(onClick = {
83	onDetailClick(chocolate)
84	}) {
85	Text("Detail")
86	}
87	}
88	}
89	}
90	}
91	}
92	}
93	
94	@Composable
95	fun ChocolateImage(chocolate: Chocolate, modifier: Modifier =
96	Modifier) {
97	Image(

98	<pre> painter = painterResource(id = chocolate.chocolateImageId), 99 contentDescription = null, 100 contentScale = ContentScale.Crop, 101 modifier = Modifier 102 .padding(8.dp) 103 .width(120.dp) 104 .height(150.dp) 105 .clip(RoundedCornerShape(16.dp)) 106) 107 }</pre>
----	--

DetailScreen.kt

Table 17. Source Code Jawaban Soal 1 Jetpack Compose

1	<pre>package com.example.scrollablelistcompose</pre>
2	
3	<pre>import androidx.compose.foundation.Image</pre>
4	<pre>import androidx.compose.foundation.layout.Column</pre>
5	<pre>import androidx.compose.foundation.layout.Row</pre>
6	<pre>import androidx.compose.foundation.layout.Spacer</pre>
7	<pre>import androidx.compose.foundation.layout.fillMaxWidth</pre>
8	<pre>import androidx.compose.foundation.layout.height</pre>
9	<pre>import androidx.compose.foundation.layout.padding</pre>
10	<pre>import androidx.compose.foundation.shape.RoundedCornerShape</pre>
11	<pre>import androidx.compose.material3.MaterialTheme</pre>
12	<pre>import androidx.compose.material3.Text</pre>
13	<pre>import androidx.compose.runtime.Composable</pre>
14	<pre>import androidx.compose.ui.Alignment</pre>
15	<pre>import androidx.compose.ui.Modifier</pre>
16	<pre>import androidx.compose.ui.draw.clip</pre>
17	<pre>import androidx.compose.ui.layout.ContentScale</pre>
18	<pre>import androidx.compose.ui.res.painterResource</pre>
19	<pre>import androidx.compose.ui.unit.dp</pre>
20	<pre>import com.example.scrollablelistcompose.data.Chocolate</pre>

```

21
22 @Composable
23 fun DetailScreen(chocolate: Chocolate) {
24     Column(modifier = Modifier
25         .padding(16.dp)
26         .fillMaxWidth()
27     ) {
28         Image(
29             painter = painterResource(id =
chocolate.chocolateImageId),
30             contentDescription = null,
31             contentScale = ContentScale.Crop,
32             modifier = Modifier
33                 .fillMaxWidth()
34                 .height(240.dp)
35                 .clip(RoundedCornerShape(12.dp))
36         )
37
38         Spacer(modifier = Modifier.height(16.dp))
39
40         Row(
41             modifier = Modifier.fillMaxWidth(),
42             verticalAlignment = Alignment.CenterVertically
43         ) {
44             Text(
45                 text = chocolate.title,
46                 style = MaterialTheme.typography.titleLarge,
47                 modifier = Modifier.weight(1f)
48             )
49         }
50
51         Spacer(modifier = Modifier.height(8.dp))
52
53         Text(

```

54	text = chocolate.description,
55	style = MaterialTheme.typography.bodyMedium
56	)
57	}
58	}

ScrollableListHome.kt

Table 18. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.scrollablelistcompose
2	
3	import androidx.compose.foundation.lazy.LazyColumn
4	import androidx.compose.foundation.lazy.items
5	import androidx.compose.runtime.remember
6	import androidx.compose.runtime.Composable
7	import com.example.scrollablelistcompose.data.DataProvider
8	import androidx.compose.ui.Modifier
9	import com.example.scrollablelistcompose.data.Chocolate
10	
11	
12	@Composable
13	fun ScrollableListHomeContent(modifier: Modifier = Modifier, onDetailClick: (Chocolate) -> Unit = {}) {
14	val chocolates = remember { DataProvider.chocolateList }
15	LazyColumn(modifier = modifier) {
16	items(chocolates) { chocolate ->
17	ChocolateListItem(chocolate, onDetailClick =
	{onDetailClick(it)})
18	}
19	}
20	}

Chocolate.kt

Table 19. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.scrollablelistcompose.data
2	
3	data class Chocolate(4 val no: Int, 5 val title: String, 6 val description: String, 7 val chocolateImageId: Int, 8 val btnLink: String 9)

DataProvider.kt

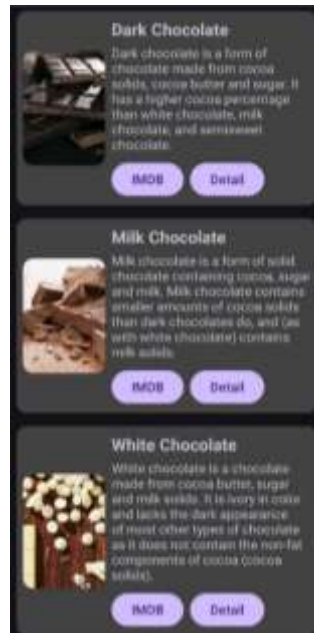
Table 20. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.scrollablelistcompose.data
2	
3	import com.example.scrollablelistcompose.R
4	
5	object DataProvider { 6 val chocolateList = listOf(7 Chocolate(8 no = 1, 9 title = "Dark Chocolate", 10 description = "Dark chocolate is a form of chocolate made from cocoa solids, cocoa butter and sugar. " + 11 "It has a higher cocoa percentage than white chocolate, milk chocolate, and semisweet chocolate.", 12 chocolateImageId = R.drawable.darkchocolate, 13 btnLink = "https://en.wikipedia.org/wiki/Dark_chocolate" 14), 15 Chocolate(16 no = 2, 17 title = "Milk Chocolate",

18	description = "Milk chocolate is a form of solid chocolate containing cocoa, sugar and milk. " +
19	"Milk chocolate contains smaller amounts of cocoa solids than dark chocolates do, and (as with white chocolate) contains milk solids.",
20	chocolateImageId = R.drawable.milkchocolate,
21	btnLink = "https://en.wikipedia.org/wiki/Milk_chocolate"
22	),
23	Chocolate(no = 3, title = "White Chocolate", description = "White chocolate is a chocolate made from cocoa butter, sugar and milk solids. " + "It is ivory in color and lacks the dark appearance of most other types of chocolate as it does not contain the non-fat components of cocoa (cocoa solids).", chocolateImageId = R.drawable.whitechocolate, btnLink = "https://en.wikipedia.org/wiki/White_chocolate"
30	),
31	Chocolate(no = 4, title = "Hazelnut Chocolate", description = "Hazelnut chocolate adds a new dimension to the flavor by combining the flavor of chocolate with the distinctive hazelnut nut. " + "The hazelnut provides a savory, crunchy, and slightly sweet touch that complements the richness of the chocolate.", chocolateImageId = R.drawable.huzelnutchocolate, btnLink = "https://kioskcokelat.com/blogs/news/perbedaan-coklat-dan-coklat-hazelnut"

38	),
39	Chocolate(
40	no = 5,
41	title = "Mint Chocolate",
42	description = "Mint chocolate (or chocolate mint) is a popular flavor of chocolate, made by adding a mint flavoring, such as peppermint, spearmint, or crème de menthe, to chocolate. " +
43	"Mint chocolate can be found in a wide variety of confectionery items, such as candy, mints, cookies, mint chocolate chip ice cream, hot chocolate, and others.",
44	chocolateImageId = R.drawable.mintchocolate,
45	btnLink = "https://en.wikipedia.org/wiki/Chocolate#Flavors"
46	)
47	)
48	}

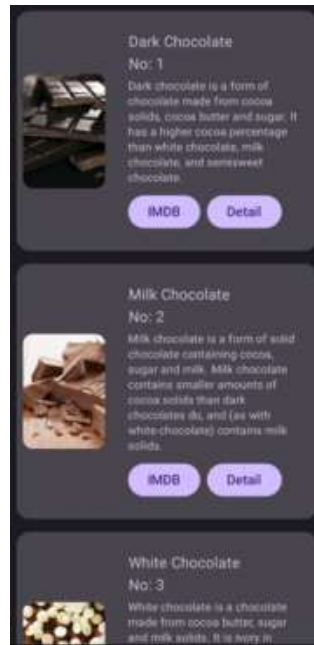
K. Output Program



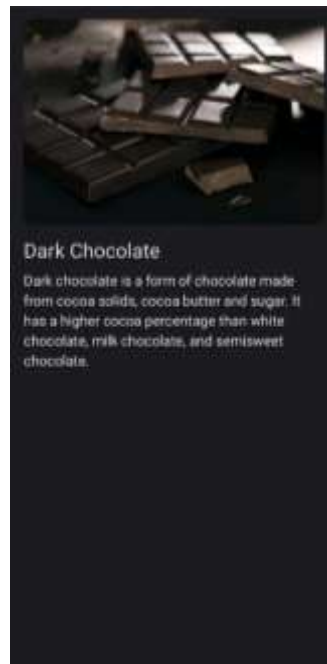
Gambar 21. UI List (XML)



Gambar 22. UI Detail (XML)



Gambar 23. UI List (Jetpack Compose)



Gambar 24. UI Detail (Jetpack Compose)

L. Pembahasan

MainActivity.kt (XML):

1. Syntax `package com.example.scrollablelistxml` merupakan deklarasi nama package file Kotlin.
2. Syntax `class MainActivity : AppCompatActivity() {}` merupakan deklarasi kelas bernama MainActivity yang extend dari AppCompatActivity.
3. Syntax `private lateinit var binding: ActivityMainBinding` mendeklarasikan objek binding untuk mengakses view dari XML via viewBinding.
4. Syntax `override fun onCreate(savedInstanceState: Bundle?) {}` merupakan lifecycle method pertama yang dipanggil saat Activity dibuat.
5. Syntax `super.onCreate(savedInstanceState)` merupakan program untuk memanggil method onCreate() dari superclass untuk menjalankan proses inisialisasi default dan wajib dipanggil agar activity dapat berjalan dengan benar.
6. Syntax `setContentView(R.layout.activity_main)` program untuk menetapkan layout XML yang digunakan sebagai UI untuk activity ini.
7. Syntax `val recyclerView: RecyclerView = findViewById(R.id.recyclerView)` merupakan fungsi untuk mendeklarasikan dan menginisialisasi variabel recyclerView dari layout XML, findViewById mencari view berdasarkan ID `@+id/recyclerView` yang telah didefinisikan di XML.
8. Syntax `recyclerView.layoutManager = LinearLayoutManager(this)` berfungsi untuk mengatur adapter untuk RecyclerView, adapter ChocolateAdapter bertanggung jawab untuk menghubungkan data (chocolateList) dengan tampilan list, this mengacu pada konteks activity, dan DataProvider.chocolateList adalah sumber data.

Chocolate.kt (XML):

1. Syntax `package com.example.scrollablelistxml` merupakan deklarasi package tempat file ini berada, yaitu com.example.scrollablelistxml.

2. Syntax `data class Chocolate(` merupakan definisi data class bernama `Chocolate`, data class secara otomatis menyediakan fungsi-fungsi penting, class ini biasanya dipakai untuk menyimpan data sederhana, seperti model atau entitas.
3. Syntax `val id: Int, val name: String, val description: String, val chocolateImageId: Int, val btnLink: String` mendeklarasikan variabel dengan tipe datanya.

ChocolateAdapter.kt (XML):

1. Syntax `class ChocolateAdapter(...)` :
`RecyclerView.Adapter<ChocolateAdapter.ViewHolder>()`
membuat class `ChocolateAdapter`, turunan dari `RecyclerView.Adapter`. `Adapter` ini digunakan untuk menjembatani data `Chocolate` dengan tampilan di `RecyclerView`.
2. Syntax `class ViewHolder(view: View)` :
`RecyclerView.ViewHolder(view)` merupakan wadah (wrapper) untuk setiap item yang akan ditampilkan di `RecyclerView`.
3. Syntax `val image: ImageView` =
`view.findViewById(R.id.imageChocolate)` menghubungkan view dari XML (`R.id.imageChocolate`) ke variabel `image`.
4. Syntax `val name: TextView` =
`view.findViewById(R.id.textName)` merupakan program untuk menghubungkan `TextView` dengan cokelat.
5. Syntax `val description: TextView` =
`view.findViewById(R.id.textDescription)` merupakan program untuk untuk menyambungkan `TextView` deskripsi cokelat.
6. Syntax `val btnImdb: Button` =
`view.findViewById(R.id.buttonImdb)` program untuk menghubungkan tombol IMDb atau link ke variabel `btnImdb`.
7. Syntax `val btnDetail: Button` =
`view.findViewById(R.id.buttonDetail)` merupakan fungsi untuk menghubungkan tombol Detail ke variabel `btnDetail`.

8. Syntax `override fun onCreateViewHolder(...)` berfungsi untuk mengubah `row_coklat.xml` menjadi objek view dan membuat ViewHolder dari tampilan tersebut.
9. Syntax `val chocolate = chocolateList[position]` merupakan program untuk mengambil objek Chocolate berdasarkan indeks.
10. Syntax
`holder.image.setImageResource(chocolate.chocolateImageId)`
) menetapkan gambar dari resource ID ke ImageView.
11. Syntax `holder.name.text = chocolate.name` menampilkan nama coklat.
12. Syntax `holder.btnImdb.setOnClickListener { ... }` merupakan program untuk menetapkan aksi saat tombol IMDb diklik dan membuat intent dengan `ACTION_VIEW` untuk membuka `chocolate.btnLink` di browser.
13. Syntax `holder.btnDetail.setOnClickListener { ... }` merupakan aksi untuk tombol Detail, membuat Intent untuk membuka `DetailActivity`, menyisipkan id dari coklat sebagai data ekstra (`putExtra("id", chocolate.id)`).

DataProvider.kt (XML):

1. Syntax `object DataProvider { }` merupakan program singleton object bernama `DataProvider`.
2. Syntax `val chocolateList = listOf(...)` mendeklarasikan properti `chocolateList` bertipe `List` dari objek `Chocolate`, menggunakan `val` karena list-nya bersifat immutable (tidak dapat diganti dengan list baru), dan fungsi `listOf(...)` digunakan untuk membuat list berisi data-data `Chocolate`.

DetailActivity.kt (XML):

1. Syntax `package com.example.scrollablelistxml` merupakan deklarasi nama package file Kotlin.
2. Syntax `class DetailActivity : AppCompatActivity() { }` merupakan deklarasi kelas bernama `DetailActivity` yang extend dari `AppCompatActivity`.

3. Syntax `override fun onCreate(savedInstanceState: Bundle?)`
`{}` mendeklarasikan fungsi `onCreate` dipanggil saat activity dibuat pertama kali, parameter menyimpan state sebelumnya jika ada.
4. Syntax `super.onCreate(savedInstanceState)` merupakan fungsi untuk memanggil implementasi `onCreate` dari superclass (`AppCompatActivity`), wajib dilakukan agar lifecycle Android tetap berjalan normal.
5. Syntax `setContentView(R.layout.activity_detail)` menetapkan layout XML `activity_detail.xml` sebagai tampilan utama activity ini.
6. Syntax `val id = intent.getIntExtra("id", -1)` mengambil data id yang dikirim dari `ChocolateAdapter` via `Intent`, jika id tidak ditemukan, default-nya adalah `-1`.
7. Syntax `val chocolate = DataProvider.chocolateList.find { it.id == id }` merupakan Mencari objek `Chocolate` dalam `DataProvider.chocolateList` yang ID-nya cocok dengan data id dari intent, menggunakan fungsi `find { ... }`, yang akan mengembalikan elemen pertama yang cocok atau null jika tidak ditemukan.
8. Syntax `chocolate?.let { ... }` berfungsi untuk mengecek apakah `chocolate` tidak null, jika ya, maka blok `let` akan dijalankan untuk menampilkan data.
9. Syntax
`findViewById<ImageView>(R.id.imageChocolateDetail).setImageResource(it.chocolateImageId)` berfungsi untuk mengatur gambar pada `ImageView` di layout detail berdasarkan gambar dari objek `Chocolate`.
10. Syntax `findViewById<TextView>(R.id.textTitleDetail).text = it.name` berfungsi untuk menampilkan nama cokelat ke `TextView` dengan ID `textTitleDetail`.
11. Syntax
`findViewById<TextView>(R.id.textDescriptionDetail).text = it.description` berfungsi untuk menampilkan deskripsi cokelat ke `TextView` dengan ID `textDescriptionDetail`.

activity_detail.xml

1. Syntax `package ScrollView` merupakan komponen yang memungkinkan isi layout dapat digulir secara vertikal jika kontennya lebih tinggi dari layar.
2. Syntax `ConstraintLayout` merupakan layout fleksibel dan powerful yang memungkinkan kita mengatur posisi elemen UI berdasarkan hubungan (constraint) antar elemen atau dengan parent.
3. Syntax `ImageView` untuk menampilkan gambar.
4. Syntax `TextView` merupakan program untuk menampilkan nama coklat, deskripsi.

activity_main.xml

1. Syntax `<LinearLayout>` merupakan layout linear yang menyusun elemen-elemen UI secara vertikal (satu di atas yang lain) karena `orientation="vertical"`.
2. Syntax `RecyclerView` merupakan komponen yang digunakan untuk menampilkan daftar item (seperti daftar coklat) secara efisien dan bisa discroll. Syntax `ImageView` untuk menampilkan gambar.

MainActivity.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablelistcompose` merupakan deklarasi nama package file Kotlin.
2. Syntax `class MainActivity : ComponenttActivity() {}` merupakan activity untuk Jetpack Compose yang berisi program UI penuh di layer, mengatur UI dengan compose, dan program untuk tema dan memanggil composable utama.
3. Syntax `override fun onCreate(savedInstanceState: Bundle?) {super.onCreate(savedInstanceState)}` merupakan override fungsi `onCreate` untuk menginisialisasi UI saat activity dibuat.
4. Syntax `ScrollAbleListComposeTheme{}` merupakan program untuk `(screen)`.
5. Syntax `MyApp(navController = navController)` untuk menjalankan composable `MyApp()` dan meneruskan `navController`.

6. Syntax `@Composable fun MyApp(navController: androidx.navigation.NavHostController) {` merupakan fungsi composable utama yang menerima NavController untuk navigasi antar layar.
7. Syntax `Scaffold { innerPadding ->` merupakan program untuk menyediakan struktur dasar seperti padding otomatis, top bar, FAB, dsb. Kemudian `innerPadding` adalah padding otomatis agar konten tidak tertutup oleh elemen UI sistem seperti status bar atau navigation bar.
8. Syntax `NavHost() {}` merupakan program untuk mengelola semua screen dan navigation berdasarkan navController. Dan `startDestination` adalah layar pertama yang akan muncul, yaitu "home".
9. Syntax `composable("home") {}` merupakan program untuk jika item di-klik, akan berpindah ke screen "detail/{id}" dengan mengirimkan no dari objek yang diklik.
10. Syntax `composable()` merupakan program untuk mendefinisikan rute dengan argumen: "detail/{id}", di mana id adalah Int.

ChocolateListItem.kt (Jetpack Compose):

1. Syntax `package package com.example.scrollablelistcompose` merupakan deklarasi nama package file Kotlin.
2. Syntax `@Composable fun ChocolateListItem(chocolate: Chocolate, onDetailClick: (Chocolate) -> Unit) {` merupakan program untuk menandai fungsi sebagai composable.
3. Syntax `Card () {` merupakan program untuk latar belakang.
4. Syntax `Row () {` merupakan row utama untuk mengatur konsen dengan berbagai ketentuan.
5. Syntax `Image ()` merupakan fungsi untuk menampilkan gambar dengan resource ID.
6. Syntax `Column () {}` merupakan program untuk mengatur teks dan tombol pada aplikasi.

7. Syntax `@Composable fun ChocolateImage(chocolate: Chocolate, modifier: Modifier = Modifier) {` merupakan program untuk menampilkan gambar.

ScrollAbleListHome.kt (Jetpack Compose):

1. Syntax `package package com.example.scrollablelistcompose` merupakan deklarasi nama package file Kotlin.
2. Syntax `@Composable fun ScrollAbleListHomeContent(modifier: Modifier = Modifier, onDetailClick: (Chocolate) -> Unit = {})` {merupakan program untuk menandai fungsi sebagai composable dan menerima dua parameter untuk styling dan klik tombol.
3. Syntax `val chocolates = remember {` `DataProvider.chocolateList }` merupakan program untuk mengambil daftar coklat dari `DataProvider` dan agar data tidak diambil ulang setiap kali recomposition terjadi.
4. Syntax `LazyColumn(modifier = modifier) {` merupakan list scrollable vertikal yang hanya memuat item yang terlihat di layar (efisien).

Chocolate.kt (Jetpack Compose):

1. Syntax `package package com.example.scrollablelistcompose` merupakan deklarasi nama package file Kotlin.
2. Syntax `data class Chocolate(`
`val no: Int,`
`val title: String,`
`val description: String,`
`val chocolateImageId: Int,`
`val btnLink: String`

merupakan kelas untuk mendefinisikan kelas yang terutama digunakan untuk menyimpan data. Kotlin secara otomatis menyediakan beberapa fungsi tambahan seperti `toString()`, `equals()`, `hashCode()`, dan `copy()` untuk kelas ini.

DataProvider.kt (Jetpack Compose):

1. Syntax `package package com.example.scrollablelistcompose` merupakan deklarasi nama package file Kotlin.
2. Syntax `object DataProvider {}` merupakan object Kotlin, yang berfungsi sebagai singleton (hanya ada satu instance). Objek ini digunakan untuk menyediakan data statis, dalam hal ini adalah daftar coklat.
3. Syntax `val chocolateList = listOf()` merupakan properti yang berisi daftar objek Chocolate. Menggunakan `listOf()` untuk membuat list dari beberapa objek Chocolate.
4. Syntax `Chocolate()`, merupakan objek Chocolate yang mendefinisikan sebuah coklat lengkap dengan ID, nama, deskripsi, gambar, dan link.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

M. Pembahasan

Karena RecyclerView masih kompatibel saat ini, terlebih keterbatasan LazyColumn untuk kasus-kasus spesifik, seperti pagination yang kompleks, data binding tingkat lanjut, multi-view types, dan item click listeners. Kemudian RecyclerView juga dapat memberikan lebih banyak kontrol terkait optimasi memori dengan fitur seperti ViewPool, yang memungkinkan daftar item tampilan yang sudah ada untuk meningkatkan kinerja. Selain itu, perpindahan dari XML ke Jetpack Compose juga memerlukan waktu dan perubahan besar dalam struktur aplikasi apalagi aplikasi yang dari awal dikembangkan dengan XML yang membutuhkan pemahaman dan penguasaan baru untuk bisa berpindah menggunakan Jetpack Compose.

N. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rizkiirr/Praktikum_Mobile

MODUL 4: ViewModel and Debugging

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a) Log saat data item masuk ke dalam list
 - b) Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c) Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - d) Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.
- e. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

O. Source Code

XML:

MainActivity.kt

Table 21. MainActivity.kt

1	package com.example.scrollablexml
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import androidx.activity.viewModels
7	import androidx.appcompat.app.AppCompatActivity
8	import androidx.lifecycle.LifecycleScope
9	import androidx.recyclerview.widget.LinearLayoutManager
10	import androidx.recyclerview.widget.RecyclerView
11	import com.example.scrollablexml.ui.ChocolateAdapter
12	import com.example.scrollablexml.ui.DetailActivity
13	import com.example.scrollablexml.viewmodel.ChocolateViewModel
14	import
	com.example.scrollablexml.viewmodel.ChocolateViewModelFactory
15	import kotlinx.coroutines.launch
16	import timber.log.Timber
17	
18	class MainActivity : AppCompatActivity() {
19	private val viewModel: ChocolateViewModel by viewModels {
20	ChocolateViewModelFactory("MainActivity")
21	}
22	
23	override fun onCreate(savedInstanceState: Bundle?) {
24	super.onCreate(savedInstanceState)
25	Timber.plant(Timber.DebugTree())
26	setContentView(R.layout.activity_main)
27	
28	

29	val recyclerView: RecyclerView =
	findViewById(R.id.recyclerView)
30	recyclerView.layoutManager = LinearLayoutManager(this)
31	
32	lifecycleScope.launch {
33	viewModel.chocolateList.collect { chocolate ->
	recyclerView.adapter =
34	ChocolateAdapter(this@MainActivity, chocolate,
35	onDetailClick = {
36	Timber.d("Detail clicked: \$it")
37	viewModel.onChocolateSelected(it)
	val intent = Intent(this@MainActivity,
38	DetailActivity::class.java)
39	intent.putExtra("id", it.id)
40	startActivity(intent)
41	},
42	onLinkClick = {
	Timber.d("IMDB clicked: \${it.btnLink}")
43	val intent = Intent(Intent.ACTION_VIEW,
	Uri.parse(it.btnLink))
44	startActivity(intent)
45	}
46	)
47	}
48	}
49	}
50	}

Chocolate.kt

Table 22. Chocolate.kt

1	package com.example.scrollablexml.data
2	
3	data class Chocolate(
4	val id: Int,

5	val name: String,
6	val description: String,
7	val chocolateImageId: Int,
8	val btnLink: String
9	)

ChocolateProvider.kt

Table 23. ChocolateProvider.kt

1	package com.example.scrollablexml.data
2	
3	import com.example.scrollablexml.R
4	
5	object ChocolateProvider {
6	val chocolateList = listOf(Chocolate(1, "Dark Chocolate", "Dark chocolate is a form of chocolate made from cocoa solids, cocoa butter and sugar. " + "It has a higher cocoa percentage than white chocolate, milk chocolate, and semisweet chocolate.", R.drawable.darkchocolate, "https://en.wikipedia.org/wiki/Dark_chocolate"), Chocolate(2, "Milk Chocolate", "Milk chocolate is a form of solid chocolate containing cocoa, sugar and milk. " + "Milk chocolate contains smaller amounts of cocoa solids than dark chocolates do, and (as with white chocolate) contains milk solids.", R.drawable.milkchocolate, "https://en.wikipedia.org/wiki/Milk_chocolate"), Chocolate(3, "White Chocolate", "White chocolate is a chocolate made from cocoa butter, sugar and milk solids. " + "It is ivory in color and lacks the dark appearance of most other types of chocolate as it does not contain the non-fat components of cocoa (cocoa solids).", R.drawable.whitechocolate, "https://en.wikipedia.org/wiki/White_chocolate"),

12	Chocolate(4, "Hazelnut Chocolate", "Hazelnut chocolate adds a new dimension to the flavor by combining the flavor of chocolate with the distinctive hazelnut nut. " + "The hazelnut provides a savory, crunchy, and slightly sweet touch that complements the richness of the chocolate.", R.drawable.huzelnutchocolate, "https://en.wikipedia.org/wiki/Chocolate#Flavors"),
13	Chocolate(5, "Mint Chocolate", "The hazelnut provides a savory, crunchy, and slightly sweet touch that complements the richness of the chocolate. " + "Mint chocolate can be found in a wide variety of confectionery items, such as candy, mints, cookies, mint chocolate chip ice cream, hot chocolate, and others. ", R.drawable.mintchocolate, "https://en.wikipedia.org/wiki/Chocolate#Flavors")
14	
15	)
16	
17	}

ChocolateAdapter.kt

Table 24. ChocolateAdapter.kt

1	package com.example.scrollablexml.ui
2	
3	import android.content.Context
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import android.widget.Button
8	import android.widget.ImageView
9	import android.widget.TextView
10	import androidx.recyclerview.widget.RecyclerView
11	import com.example.scrollablexml.R
12	import com.example.scrollablexml.data.Chocolate
13	

```

14 class ChocolateAdapter(
15     private val context: Context,
16     private val chocolateList: List<Chocolate>,
17     private val onDetailClick: (Chocolate) -> Unit,
18     private val onLinkClick: (Chocolate) -> Unit
19 ) : RecyclerView.Adapter<ChocolateAdapter.ViewHolder>() {
20
21     inner class ViewHolder(view: View) :
22     RecyclerView.ViewHolder(view) {
23         val image: ImageView =
24         view.findViewById(R.id.imageChocolate)
25         val name: TextView = view.findViewById(R.id.textName)
26         val description: TextView =
27         view.findViewById(R.id.textDescription)
28         val btnImdb: Button = view.findViewById(R.id.buttonImdb)
29         val btnDetail: Button =
30         view.findViewById(R.id.buttonDetail)
31     }
32
33     override fun onCreateViewHolder(parent: ViewGroup, viewType:
34     Int): ViewHolder {
35         val view =
36         LayoutInflater.from(context).inflate(R.layout.row_chocolate,
37         parent, false)
38         return ViewHolder(view)
39     }
40
41     override fun getItemCount(): Int = chocolateList.size
42
43     override fun onBindViewHolder(holder: ViewHolder, position:
44     Int) {
45         val chocolate = chocolateList[position]
46         holder.image.setImageResource(chocolate.chocolateImageId)

```

39	holder.name.text = chocolate.name
40	holder.description.text = chocolate.description
41	
42	holder.btnImdb.setOnClickListener {
43	onLinkClick(chocolate)
44	}
45	
46	holder.btnDetail.setOnClickListener {
47	onDetailClick(chocolate)
48	}
49	}
50	}

DetailActivity.kt

Table 25. DetailActivity.kt

1	package com.example.scrollablexml.ui
2	
3	import android.os.Bundle
4	import android.widget.ImageView
5	import android.widget.TextView
6	import androidx.appcompat.app.AppCompatActivity
7	import com.example.scrollablexml.R
8	import com.example.scrollablexml.data.ChocolateProvider
9	import timber.log.Timber
10	
11	class DetailActivity : AppCompatActivity() {
12	override fun onCreate(savedInstanceState: Bundle?) {
13	super.onCreate(savedInstanceState)
14	Timber.d("DetailActivity opened")
15	
16	setContentView(R.layout.activity_detail)
17	val id = intent.getIntExtra("id", -1)
18	val chocolate = ChocolateProvider.chocolateList.find
	{ it.id == id }

19	
20	chocolate?.let {
21	Timber.d("Displaying chocolate detail: \$it")
22	
23	findViewById<ImageView>(R.id.imageChocolateDetail).setImageResource(it.chocolateImageId)
24	
25	findViewById<TextView>(R.id.textTitleDetail).text = it.name
26	findViewById<TextView>(R.id.textDescriptionDetail).text = it.description
27	}
28	}
29	}

ChocolateViewModel.kt

Table 26. ChocolateViewModel.kt

1	package com.example.scrollablexml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.scrollablexml.data.Chocolate
5	import com.example.scrollablexml.data.ChocolateProvider
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import timber.log.Timber
9	
10	class ChocolateViewModel(private val source: String) : ViewModel() {
11	
12	private val _chocolateList = MutableStateFlow<List<Chocolate>>(emptyList())
13	val chocolateList: StateFlow<List<Chocolate>> get() = _chocolateList
14	

15	private val _selectedChocolate =
16	MutableStateFlow<Chocolate?>(null)
17	init {
18	Timber.d("ViewModel created with source: \$source")
19	_chocolateList.value =
20	ChocolateProvider.chocolateList
21	Timber.d("Chocolate list loaded: \${_chocolateList.value}")
22	}
23	fun onChocolateSelected(chocolate: Chocolate) {
24	_selectedChocolate.value = chocolate
25	Timber.d("Selected chocolate: \$chocolate")
26	}

ChocolateViewModelFactory.kt

Table 27. ChocolateViewModelFactory.kt

1	package com.example.scrollablexml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class ChocolateViewModelFactory(private val source: String) :
7	ViewModelProvider.Factory {
8	override fun <T : ViewModel> create(modelClass: Class<T>):
9	T {
10	return ChocolateViewModel(source) as T
11	}

activity_main.xml

Table 28. activity_main.xml

1	<LinearLayout
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:orientation="vertical"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent">
6	
7	<androidx.recyclerview.widget.RecyclerView
8	android:id="@+id/recyclerView"
9	android:layout_width="match_parent"
10	android:layout_height="match_parent"/>
11	</LinearLayout>

activity_detail.xml

Table 29. activity_detail.xml

1	<ScrollView
2	xmlns:android= http://schemas.android.com/apk/res/android
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:padding="16dp">
7	
8	<androidx.constraintlayout.widget.ConstraintLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<ImageView
13	android:id="@+id/imageChocolateDetail"
14	android:layout_width="0dp"
15	android:layout_height="240dp"
16	android:scaleType="centerCrop"


```

17         app:layout_constraintTop_toTopOf="parent"
18         app:layout_constraintStart_toStartOf="parent"
19         app:layout_constraintEnd_toEndOf="parent" />
20
21     <TextView
22         android:id="@+id/textTitleDetail"
23         android:layout_marginTop="16dp"
24         android:textSize="24sp"
25         android:textStyle="bold"
26         android:layout_width="0dp"
27         android:layout_height="wrap_content"
28
29     app:layout_constraintTop_toBottomOf="@id/imageChocolateDetail"
30         app:layout_constraintStart_toStartOf="parent"
31         app:layout_constraintEnd_toEndOf="parent" />
32
33     <TextView
34         android:id="@+id/textLabelPlot"
35         android:textStyle="bold"
36         android:layout_marginTop="8dp"
37         android:layout_width="wrap_content"
38         android:layout_height="wrap_content"
39
40     app:layout_constraintTop_toBottomOf="@id/textTitleDetail"
41         app:layout_constraintStart_toStartOf="parent" />
42
43     <TextView
44         android:id="@+id/textDescriptionDetail"
45         android:layout_width="0dp"
46         android:layout_height="wrap_content"
47         android:layout_marginTop="4dp"
48         app:layout_constraintTop_toBottomOf="@id/textLabelPlot"
49         app:layout_constraintStart_toStartOf="parent"
50         app:layout_constraintEnd_toEndOf="parent"

```

51	app:layout_constraintBottom_toBottomOf="parent" />
52	</androidx.constraintlayout.widget.ConstraintLayout>
53	</ScrollView>

row_chocolate.xml

Table 30. row_chocolate.xml

1	<ScrollView
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:padding="16dp">
7	
8	<androidx.constraintlayout.widget.ConstraintLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<ImageView
13	android:id="@+id/imageChocolateDetail"
14	android:layout_width="0dp"
15	android:layout_height="240dp"
16	android:scaleType="centerCrop"
17	app:layout_constraintTop_toTopOf="parent"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintEnd_toEndOf="parent" />
20	
21	<TextView
22	android:id="@+id/textTitleDetail"
23	android:layout_marginTop="16dp"
24	android:textSize="24sp"
25	android:textStyle="bold"
26	android:layout_width="0dp"
27	android:layout_height="wrap_content"
28	

29	
30	app:layout_constraintTop_toBottomOf="@id/imageChocolateDetail"
31	app:layout_constraintStart_toStartOf="parent"
32	app:layout_constraintEnd_toEndOf="parent" />
33	
34	<TextView
35	android:id="@+id/textLabelPlot"
36	android:textStyle="bold"
37	android:layout_marginTop="8dp"
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
40	
41	app:layout_constraintTop_toBottomOf="@id/textTitleDetail"
42	app:layout_constraintStart_toStartOf="parent" />
43	
44	<TextView
45	android:id="@+id/textDescriptionDetail"
46	android:layout_width="0dp"
47	android:layout_height="wrap_content"
48	android:layout_marginTop="4dp"
49	app:layout_constraintTop_toBottomOf="@id/textLabelPlot"
50	app:layout_constraintStart_toStartOf="parent"
51	app:layout_constraintEnd_toEndOf="parent"
52	app:layout_constraintBottom_toBottomOf="parent" />
53	</androidx.constraintlayout.widget.ConstraintLayout>
54	</ScrollView>

Jetpack Compose:

MainActivity.kt

Table 31. MainActivity.kt

1	package com.example.scrollablecompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity

```

5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.foundation.lazy.LazyColumn
10 import androidx.compose.foundation.lazy.items
11 import androidx.compose.material3.Scaffold
12 import androidx.compose.runtime.collectAsState
13 import androidx.compose.runtime.getValue
14 import androidx.compose.ui.Modifier
15 import androidx.lifecycle.viewmodel.compose.viewModel
16 import androidx.navigation.compose.NavHost
17 import androidx.navigation.compose.composable
18 import androidx.navigation.compose.rememberNavController
19 import com.example.scrollablecompose.data.ChocolateProvider
20
21 import com.example.scrollablecompose.uiScreen.DetailScreen
22 import com.example.scrollablecompose.ui.theme.ScrollableComposeTheme
23 import com.example.scrollablecompose.uiComponent.ChocolateItem
24 import com.example.scrollablecompose.uiComponent.ListScreen
25 import com.example.scrollablecompose.viewModel.ChocolateViewModel
26 import
    com.example.scrollablecompose.viewModel.ChocolateViewModelFactory
27 import timber.log.Timber
28
29 class MainActivity : ComponentActivity() {
30     override fun onCreate(savedInstanceState: Bundle?) {
31         super.onCreate(savedInstanceState)
32         Timber.plant(Timber.DebugTree())
33         enableEdgeToEdge()
34         setContent {
35             val navController = rememberNavController()
36             val viewModel: ChocolateViewModel = viewModel(
37                 factory = ChocolateViewModelFactory("parameter-mu")

```

```

38         )
39
40         ScrollableComposeTheme {
41             Scaffold(modifier = Modifier.fillMaxSize()) {
42                 innerPadding ->
43                     NavHost(
44                         navController = navController,
45                         startDestination = "list",
46                         modifier = Modifier.padding(innerPadding)
47                     ) {
48                         composable("list") {
49                             val chocolateList by
50                             viewModel.chocolateList.collectAsState()
51                             ListScreen(
52                                 chocolates = chocolateList,
53                                 onDetailClicked = {
54                                     Timber.d("Navigating to Detail of:
55                                     ${it.title}")
56                                     navController.navigate("detail/${it.no}")
57                                 }
58                             )
59                         }
60                         composable("detail/{id}") { backStackEntry ->
61                             val id =
62                             backStackEntry.arguments?.getString("id")?.toIntOrNull()
63                             DetailScreen(id = id, viewModel = viewModel)
64                         }
65                     }
66             }
67
68     }

```

69	}
70	}

Chocolate.kt

Table 32. Chocolate.kt

1	package com.example.scrollablecompose.data
2	
3	data class Chocolate(
4	val no: Int,
5	val title: String,
6	val description: String,
7	val chocolateImageId: Int,
8	val btnLink: String
9	)

ChocolateProvider.kt

Table 33. ChocolateProvider.kt

1	package com.example.scrollablecompose.data
2	
3	import com.example.scrollablecompose.R
4	
5	object ChocolateProvider {
6	val chocolateList = listOf(
7	Chocolate(
8	no = 1,
9	title = "Dark Chocolate",
10	description = "Dark chocolate is a form of
	chocolate made from cocoa solids, cocoa butter and sugar. " +
11	"It has a higher cocoa percentage than
	white chocolate, milk chocolate, and semisweet chocolate.",
12	chocolateImageId = R.drawable.darkchocolate,
13	btnLink =
	"https://en.wikipedia.org/wiki/Dark_chocolate"
14	),

15	Chocolate(
16	no = 2,
17	
18	title = "Milk Chocolate",
19	description = "Milk chocolate is a form of
	solid chocolate containing cocoa, sugar and milk. " +
20	"Milk chocolate contains smaller
	amounts of cocoa solids than dark chocolates do, and (as with
	white chocolate) contains milk solids.",
21	chocolateImageId = R.drawable.milkchocolate,
22	btnLink =
	"https://en.wikipedia.org/wiki/Milk_chocolate"
23	),
24	Chocolate(
25	no = 3,
26	title = "White Chocolate",
27	description = "White chocolate is a chocolate
	made from cocoa butter, sugar and milk solids. " +
28	"It is ivory in color and lacks the
	dark appearance of most other types of chocolate as it does
	not contain the non-fat components of cocoa (cocoa solids).",
29	chocolateImageId = R.drawable.whitechocolate,
30	btnLink =
	"https://en.wikipedia.org/wiki/White_chocolate"
31	),
32	Chocolate(
33	no = 4,
34	title = "Hazelnut Chocolate",
35	description = "Hazelnut chocolate adds a new
	dimension to the flavor by combining the flavor of chocolate
	with the distinctive hazelnut nut. " +
36	"The hazelnut provides a savory,
	crunchy, and slightly sweet touch that complements the
	richness of the chocolate.",

37	chocolateImageId	=
	R.drawable.huzelnutchocolate,	
38	btnLink	=
	"https://kioskcokelat.com/blogs/news/perbedaan-coklat-dan-coklat-hazelnut"	
39	),	
40	Chocolate(	
41	no = 5,	
42	title = "Mint Chocolate",	
43	description = "Mint chocolate (or chocolate mint) is a popular flavor of chocolate, made by adding a mint flavoring, such as peppermint, spearmint, or crème de menthe, to chocolate. " +	
44	"Mint chocolate can be found in a wide variety of confectionery items, such as candy, mints, cookies, mint chocolate chip ice cream, hot chocolate, and others.",	
45	chocolateImageId = R.drawable.mintchocolate,	
46	btnLink	=
	"https://en.wikipedia.org/wiki/Chocolate#Flavors"	
47	)	
48	)	
49	}	

ChocolateItem.kt

Table 34. ChocolateItem.kt

1	package	com.example.scrollablecompose.uiComponent
2		
3	import	android.content.Intent
4	import	android.net.Uri
5	import	androidx.compose.foundation.Image
6	import	androidx.compose.foundation.layout.Column
7	import	androidx.compose.foundation.layout.Row
8	import	androidx.compose.foundation.layout.Spacer

9	<code>import</code>	<code>androidx.compose.foundation.layout.fillMaxWidth</code>
10	<code>import</code>	<code>androidx.compose.foundation.layout.height</code>
11	<code>import</code>	<code>androidx.compose.foundation.layout.padding</code>
12	<code>import</code>	<code>androidx.compose.foundation.layout.width</code>
13	<code>import</code>	<code>androidx.compose.foundation.shape.RoundedCornerShape</code>
14	<code>import</code>	<code>androidx.compose.material3.Button</code>
15	<code>import</code>	<code>androidx.compose.material3.Card</code>
16	<code>import</code>	<code>androidx.compose.material3.MaterialTheme</code>
17	<code>import</code>	<code>androidx.compose.material3.Text</code>
18	<code>import</code>	<code>androidx.compose.runtime.Composable</code>
19	<code>import</code>	<code>androidx.compose.ui.Alignment</code>
20	<code>import</code>	<code>androidx.compose.ui.Modifier</code>
21	<code>import</code>	<code>androidx.compose.ui.draw.clip</code>
22	<code>import</code>	<code>androidx.compose.ui.layout.ContentScale</code>
23	<code>import</code>	<code>androidx.compose.ui.platform.LocalContext</code>
24	<code>import</code>	<code>androidx.compose.ui.res.painterResource</code>
25	<code>import</code>	<code>androidx.compose.ui.tooling.preview.Preview</code>
26	<code>import</code>	<code>androidx.compose.ui.unit.dp</code>
27	<code>import</code>	<code>com.example.scrollablecompose.R</code>
28	<code>import</code>	<code>com.example.scrollablecompose.data.Chocolate</code>
29	<code>import</code>	<code>timber.log.Timber</code>
30		
31	<code>@Composable</code>	
32	<code>fun</code>	<code>ChocolateItem(</code>
33	<code> chocolate:</code>	<code>Chocolate,</code>
34	<code> onDetailClicked:</code>	<code>(Chocolate) -> Unit</code>
35	<code>)</code>	<code>{</code>
36	<code> val</code>	<code>context = LocalContext.current</code>
37		
38	<code> Card(</code>	
39	<code> modifier</code>	<code>= Modifier</code>
40	<code> .padding(8.dp)</code>	
41	<code> .fillMaxWidth(),</code>	
42	<code> shape</code>	<code>= RoundedCornerShape(16.dp)</code>

```

43         )
44         Row(
45             modifier = Modifier.padding(8.dp),
46             verticalAlignment = Alignment.CenterVertically
47         ) {
48
49             Image(
50                 painter = painterResource(id =
chocolate.chocolateImageId),
51                 contentDescription = null,
52                 contentScale = ContentScale.Crop,
53                 modifier = Modifier
54                     .width(100.dp)
55                     .height(140.dp)
56                     .clip(RoundedCornerShape(12.dp))
57             )
58
59             Spacer(modifier = Modifier.width(12.dp))
60
61             Column(
62                 modifier = Modifier
63                     .padding(16.dp)
64                     .weight(1f)
65             ) {
66                 Text(text = chocolate.title)
67                 Text(text = "No: ${chocolate.no}")
68                 Text(
69                     text = chocolate.description,
70                     style = MaterialTheme.typography.bodySmall,
71                     modifier = Modifier.padding(top = 4.dp)
72                 )
73
74                 Row(modifier = Modifier.padding(top = 8.dp)) {
75                     Button(onClick =

```

```

76         Timber.d("Link button clicked:
    ${chocolate.btnLink}")
77         val intent = Intent(Intent.ACTION_VIEW,
    Uri.parse(chocolate.btnLink))
78         context.startActivity(intent)
79     })
80     Text("IMDB")
81 }
82
83     Spacer(modifier =
    Modifier.padding(horizontal = 4.dp))
84
85     Button(onClick = {
86         Timber.d("Detail button clicked:
    ${chocolate.title}")
87         onDetailClicked(chocolate)
88     })
89     {
90         Text("Detail")
91     }
92 }
93 }
94 }
95 }
96 }
97
98
99 @Preview
100 @Composable
101 private fun ChocolateItemPreview() {
102     val chocolate = Chocolate(
103         no = 1,
104         title = "Dark Chocolate",
105         description = "Dark chocolate is a form of chocolate made

```

	from cocoa solids, cocoa butter and sugar. " +
106	"It has a higher cocoa percentage than white
	chocolate, milk chocolate, and semisweet chocolate.",
107	chocolateImageId = R.drawable.darkchocolate,
108	btnLink = "https://en.wikipedia.org/wiki/Dark_chocolate"
109	)
110	ChocolateItem(
111	chocolate= chocolate,
112	onDetailClicked= { }
113	
114	)
115	}

ChocolateListScreen.kt

Table 35. ChocolateListScreen.kt

1	package com.example.scrollablecompose.uiComponent
2	
3	import androidx.compose.foundation.lazy.LazyColumn
4	import androidx.compose.foundation.lazy.items
5	import androidx.compose.runtime.Composable
6	import com.example.scrollablecompose.data.Chocolate
7	
8	@Composable
9	fun ListScreen(
10	chocolates: List<Chocolate>,
11	onDetailClicked: (Chocolate) -> Unit
12	) {
13	LazyColumn {
14	items(chocolates) { chocolate ->
15	ChocolateItem(chocolate = chocolate,
	onDetailClicked = onDetailClicked)
16	}
17	}
18	}

DetailScreen.kt

Table 36. DetailScreen.kt

1	package com.example.scrollablecompose.uiScreen
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.Column
5	import androidx.compose.foundation.layout.Row
6	import androidx.compose.foundation.layout.Spacer
7	import androidx.compose.foundation.layout.fillMaxWidth
8	import androidx.compose.foundation.layout.height
9	import androidx.compose.foundation.layout.padding
10	import androidx.compose.foundation.shape.RoundedCornerShape
11	import androidx.compose.material3.MaterialTheme
12	import androidx.compose.material3.Text
13	import androidx.compose.runtime.Composable
14	import androidx.compose.ui.Alignment
15	import androidx.compose.ui.Modifier
16	import androidx.compose.ui.draw.clip
17	import androidx.compose.ui.layout.ContentScale
18	import androidx.compose.ui.res.painterResource
19	import androidx.compose.ui.unit.dp
20	import
	com.example.scrollablecompose.viewModel.ChocolateViewModel
21	import timber.log.Timber
22	
23	@Composable
24	fun DetailScreen(
25	id: Int?,
26	viewModel: ChocolateViewModel
27	) {
28	val chocolate = id?.let {viewModel.getChocolateById(it)}
29	
30	if (chocolate != null) {
31	

32	Timber.d("Navigating to Detail of: {chocolate.title}")	
33		
34	Column(	
35	modifier = Modifier	
36	.padding(16.dp)	
37	.fillMaxWidth()	
38	) {	
39	Image(	
40	painter = painterResource(id = chocolate.chocolateImageId),	
41	contentDescription = null,	
42	contentScale = ContentScale.Crop,	
43	modifier = Modifier	
44	.fillMaxWidth()	
45	.height(240.dp)	
46	.clip(RoundedCornerShape(12.dp))	
47	)	
48		
49	Spacer(modifier = Modifier.height(16.dp))	
50		
51	Row(	
52	modifier = Modifier.fillMaxWidth(),	
53	verticalAlignment	=
	Alignment.CenterVertically	
54	) {	
55	Text(	
56	text = chocolate.title,	
57	style	=
	MaterialTheme.typography.titleLarge,	
58	modifier = Modifier.weight(1f)	
59	)	
60	}	
61		

62	Spacer(modifier = Modifier.height(8.dp))
63	
64	Text(
65	text = chocolate.description,
66	style = MaterialTheme.typography.bodyMedium
67	)
68	}
69	} else{
70	Timber.e("Chocolate with ID \$id not found")
71	Text("Data tidak ada", modifier =
	Modifier.padding(16.dp))
72	}
73	}

ChocolateViewModel.kt

Table 37. ChocolateViewModel.kt

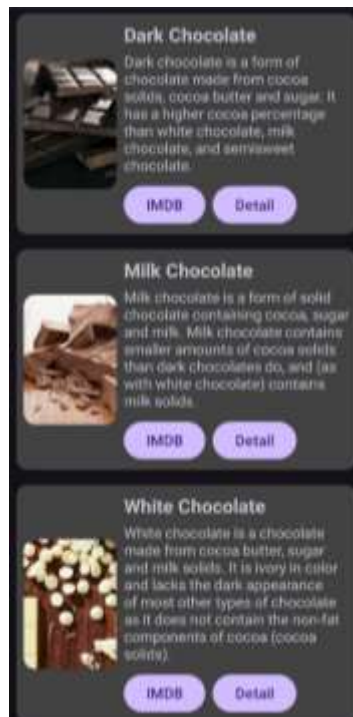
1	package com.example.scrollablecompose.viewModel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.scrollablecompose.data.Chocolate
6	import com.example.scrollablecompose.data.ChocolateProvider
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import timber.log.Timber
10	
11	class ChocolateViewModelFactory(private val param: String) : ViewModelProvider.Factory {
12	override fun <Choco : ViewModel> create(modelClass: Class<Choco>): Choco {
13	if (modelClass.isAssignableFrom(ChocolateViewModel::class.java)) {

```

14 return ChocolateViewModel(param) as Choco
15     }
16     throw IllegalArgumentException("Unknown ViewModel
class")
17     }
18 }
19
20 class ChocolateViewModel(private val param: String) :
    ViewModel() {
21
22     private val _chocolateList =
    MutableStateFlow<List<Chocolate>>(emptyList())
23     val chocolateList: StateFlow<List<Chocolate>> =
    _chocolateList
24
25     init {
26         loadChocolates()
27     }
28
29     private fun loadChocolates() {
30         _chocolateList.value =
    ChocolateProvider.chocolateList
31         Timber.d("Chocolate list loaded:
    ${_chocolateList.value.size} items")
32
33     }
34
35     fun getChocolateById(id: Int): Chocolate? {
36         return _chocolateList.value.find { it.no == id }
37     }
38 }

```

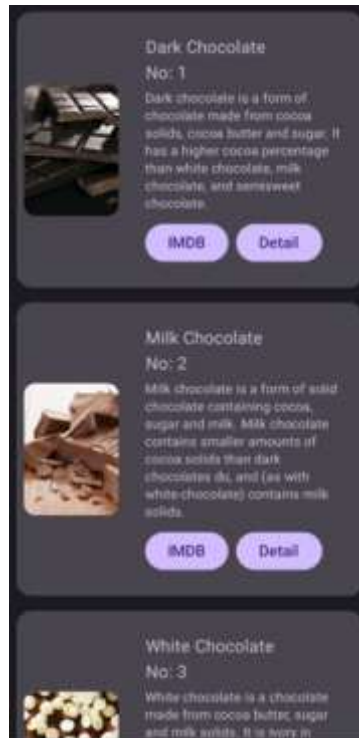

P. Output Program



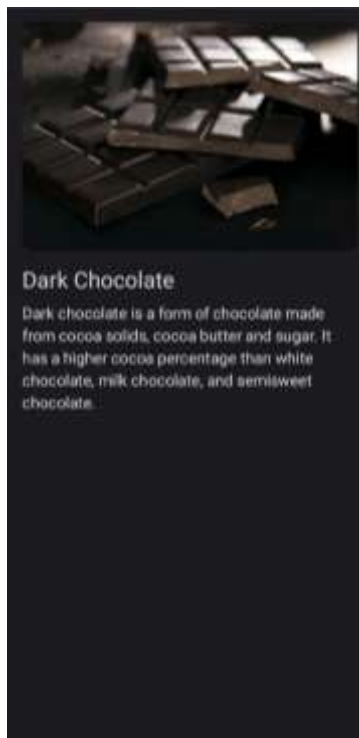
Gambar 25. UI List (XML)



Gambar 26. UI Detail (XML)



Gambar 27. UI List (Jetpack Compose)



Gambar 28. UI Detail (Jetpack Compose)

Q. Pembahasan

MainActivity.kt (XML):

1. Syntax `package package com.example.scrollablexml` merupakan deklarasi nama package file Kotlin.
2. Syntax `class MainActivity : AppCompatActivity() {}` merupakan deklarasi kelas bernama MainActivity yang extend dari AppCompatActivity.
3. Syntax `private val viewModel: ChocolateViewModel by viewModels {ChocolateViewModelFactory("MainActivity")}` membuat instance ChocolateViewModel dengan by viewModels. Menggunakan ChocolateViewModelFactory untuk mengirim parameter "MainActivity" ke ViewModel.
4. Syntax `override fun onCreate(savedInstanceState: Bundle?) {}` merupakan lifecycle method pertama yang dipanggil saat Activity dibuat.
5. Syntax `super.onCreate(savedInstanceState)` merupakan program untuk memanggil method onCreate() dari superclass untuk menjalankan proses inisialisasi default dan wajib dipanggil agar activity dapat berjalan dengan benar.
6. Syntax `Timber.plant(Timber.DebugTree())` program untuk mengatur Timber untuk mencetak log debug selama development.
7. Syntax `setContentView(R.layout.activity_main)` program untuk menetapkan layout XML yang digunakan sebagai UI untuk activity ini.
8. Syntax `val recyclerView: RecyclerView = findViewById(R.id.recyclerView)` merupakan fungsi untuk mendeklarasikan dan menginisialisasi variabel recyclerView dari layout XML, findViewById mencari view berdasarkan IDE `@+id/recyclerView` yang telah didefinisikan di XML.
9. Syntax `recyclerView.layoutManager = LinearLayoutManager(this)` berfungsi untuk mengatur adapter untuk RecyclerView, adapter ChocolateAdapter bertanggung jawab untuk menghubungkan data (chocolateList) dengan tampilan list, this mengacu pada konteks activity, dan DataProvider.chocolateList adalah sumber data.

10. Syntax `lifecycleScope.launch { }` berfungsi untuk meluncurkan coroutine yang mengikuti lifecycle activity (otomatis berhenti saat activity dihancurkan).
11. Syntax `viewModel.chocolateList.collect { chocolate }` -> berfungsi untuk mengumpulkan aliran data chocolateList dari ViewModel (dengan StateFlow atau Flow).
12. Syntax `recyclerView.adapter = ChocolateAdapter(this@MainActivity, chocolate, )` berfungsi untuk menetapkan adapter RecyclerView dengan data chocolate.
13. Syntax `onDetailClick = { Timber.d("Detail clicked: $it") viewModel.onChocolateSelected(it) val intent = Intent(this@MainActivity, DetailActivity::class.java) intent.putExtra("id", it.id) startActivity(intent) },` berfungsi untuk mengatur item diklik untuk detail log data, memanggil fungsi di ViewModel untuk menyimpan yang dipilih, dan mavigasi ke DetailActivity, sambil mengirim ID item.
14. Syntax `onLinkClick = { Timber.d("IMDB clicked:${it.btnLink}") val intent = Intent(Intent.ACTION_VIEW, Uri.parse(it.btnLink)) startActivity(intent) }` berfungsi untuk mengatur tombol link ditekan logging link, membuat Intent untuk membuka link menggunakan browser atau app lain, dan menjalankan intent tersebut.

Chocolate.kt (XML):

1. Syntax `package package com.example.scrollablelistxml` merupakan deklarasi package tempat file ini berada, yaitu package data, `com.example.scrollablexml`.
2. Syntax `data class Chocolate( )` merupakan definisi data class bernama Chocolate, data class secara otomatis menyediakan fungsi-fungsi penting, class ini biasanya dipakai untuk menyimpan data sederhana, seperti model atau entitas.
3. Syntax `val id: Int, val name: String, val description: String, val chocolateImageId: Int, val btnLink: String`

- a. mendeklarasikan variabel dengan tipe datanya.

ChocolateProvider.kt (XML):

1. Syntax `package com.example.scrollablexml.data` merupakan deklarasi package tempat file ini berada, yaitu di package data, com.example.scrollablexml.
2. Syntax `object ChocolateProvider {` digunakan untuk membuat singleton, yaitu satu instance global dan menyimpan data list coklat yang bisa diakses dari mana saja.
3. Syntax `val chocolateList = listOf(` mendeklarasikan properti immutable (val) yang berisi daftar objek Chocolate dan menggunakan `listOf()` untuk membuat list statis.

ChocolateAdapter.kt (XML):

1. Syntax `class ChocolateAdapter(...) : RecyclerView.Adapter<ChocolateAdapter.ViewHolder>()` membuat class `ChocolateAdapter`, turunan dari `RecyclerView.Adapter`. Adapter ini digunakan untuk menjembatani data Chocolate dengan tampilan di `RecyclerView`.
2. Syntax `class ViewHolder(view: View) : RecyclerView.ViewHolder(view)` merupakan wadah (wrapper) untuk setiap item yang akan ditampilkan di `RecyclerView`.
3. Syntax `val image: ImageView = view.findViewById(R.id.imageChocolate)` menghubungkan view dari XML (`R.id.imageChocolate`) ke variabel `image`.
4. Syntax `val name: TextView = view.findViewById(R.id.textName)` merupakan program untuk menghubungkan `TextView` dengan coklat.
5. Syntax `val description: TextView = view.findViewById(R.id.textDescription)` merupakan program untuk menyambungkan `TextView` deskripsi coklat.

6. Syntax `val btnImdb: Button = view.findViewById(R.id.buttonImdb)` program untuk menghubungkan tombol IMDb atau link ke variabel `btnImdb`.
7. Syntax `val btnDetail: Button = view.findViewById(R.id.buttonDetail)` merupakan fungsi untuk menghubungkan tombol Detail ke variabel `btnDetail`.
8. Syntax `override fun onCreateViewHolder(...)` berfungsi untuk mengubah `row_chocolate.xml` menjadi objek view dan membuat ViewHolder dari tampilan tersebut.
9. Syntax `override fun getItemCount(): Int = chocolateList.size` merupakan program untuk mengembalikan jumlah item dalam `chocolateList` dan untuk menentukan berapa banyak item yang ditampilkan.
10. Syntax `override fun onBindViewHolder(holder: ViewHolder, position: Int) {` mengambil data coklat pada posisi `position`.
11. Syntax `holder.name.text = chocolate.name` menampilkan nama coklat.
12. Syntax `holder.btnImdb.setOnClickListener { ... }` merupakan program untuk menetapkan aksi saat tombol IMDb diklik dan membuat intent dengan `ACTION_VIEW` untuk membuka `chocolate.btnLink` di browser.
13. Syntax `holder.btnDetail.setOnClickListener { ... }` merupakan aksi untuk tombol Detail, membuat Intent untuk membuka `DetailActivity`, menyisipkan id dari coklat sebagai data ekstra (`putExtra("id", chocolate.id)`).

DetailActivity.kt (XML):

1. Syntax `package com.example.scrollablexml` merupakan deklarasi nama package file Kotlin.
2. Syntax `class DetailActivity : AppCompatActivity() {}` merupakan deklarasi kelas bernama `DetailActivity` yang extend dari `AppCompatActivity`.
3. Syntax `Timber.d("DetailActivity opened")` merupakan log debug untuk menandai bahwa `DetailActivity` sudah dibuka.

4. Syntax `override fun onCreate(savedInstanceState: Bundle?)`
`{}` mendeklarasikan fungsi `onCreate` dipanggil saat activity dibuat pertama kali, parameter menyimpan state sebelumnya jika ada.
5. Syntax `super.onCreate(savedInstanceState)` merupakan fungsi untuk memanggil implementasi `onCreate` dari superclass (`AppCompatActivity`), wajib dilakukan agar lifecycle Android tetap berjalan normal.
6. Syntax `setContentView(R.layout.activity_detail)` menetapkan layout XML `activity_detail.xml` sebagai tampilan utama activity ini.
7. Syntax `val id = intent.getIntExtra("id", -1)` mengambil data id yang dikirim dari `ChocolateAdapter` via `Intent`, jika id tidak ditemukan, default-nya adalah `-1`.
8. Syntax `val chocolate = DataProvider.chocolateList.find { it.id == id }` merupakan Mencari objek `Chocolate` dalam `DataProvider.chocolateList` yang ID-nya cocok dengan data id dari intent, menggunakan fungsi `find { ... }`, yang akan mengembalikan elemen pertama yang cocok atau null jika tidak ditemukan.
9. Syntax `chocolate?.let { ... }` berfungsi untuk mengecek apakah `chocolate` tidak null, jika ya, maka blok `let` akan dijalankan untuk menampilkan data.
10. Syntax
`findViewById<ImageView>(R.id.imageChocolateDetail).setImageResource(it.chocolateImageId)` berfungsi untuk mengatur gambar pada `ImageView` di layout detail berdasarkan gambar dari objek `Chocolate`.
11. Syntax `findViewById<TextView>(R.id.textTitleDetail).text = it.name` berfungsi untuk menampilkan nama cokelat ke `TextView` dengan ID `textTitleDetail`.
12. Syntax
`findViewById<TextView>(R.id.textDescriptionDetail).text = it.description` berfungsi untuk menampilkan deskripsi cokelat ke `TextView` dengan ID `textDescriptionDetail`.

ChocolateViewModel.kt (XML):

1. Syntax `package com.example.scrollablexml.viewmodel` merupakan deklarasi nama package `viewmodel`.
2. Syntax `class ChocolateViewModel(private val source: String) : ViewModel() {` merupakan `ViewModel` yang memegang data coklat. Menerima parameter `source` untuk menandai asal atau konteks pembuatan `ViewModel`.
3. Syntax `private val _chocolateList = MutableStateFlow<List<Chocolate>>(emptyList())`
4. `val chocolateList: StateFlow<List<Chocolate>> get() = _chocolateList` merupakan fungsi untuk menyimpan list coklat secara mutable.
5. Syntax `private val _selectedChocolate = MutableStateFlow<Chocolate?>(null)` menyimpan coklat yang sedang dipilih, dengan nilai awal `null`.
6. Syntax `init { Timber.d("ViewModel created with source: $source") _chocolateList.value = ChocolateProvider.chocolateList Timber.d("Chocolate list loaded: ${_chocolateList.value}") }` merupakan Blok inisialisasi yang dipanggil saat `ViewModel` dibuat, menampilkan sumber `ViewModel`, mengisi `_chocolateList` dengan data dari `ChocolateProvider`, dan log pesan debug daftar coklat yang sudah dimuat.

ChocolateViewModelFactory.kt (XML):

1. Syntax `package com.example.scrollablexml.viewmodel` merupakan deklarasi nama package `viewmodel`.
2. Syntax `class ChocolateViewModelFactory(private val source: String) : ViewModelProvider.Factory {` merupakan kelas factory `ChocolateViewModelFactory` yang mengimplementasikan interface `ViewModelProvider.Factory`. Factory ini menerima sebuah parameter `source` (tipe `String`) yang akan diteruskan ke `ViewModel`.

3. Syntax `override fun <T : ViewModel> create(modelClass: Class<T>): T {` membuat instance `ViewModel` sesuai kelas yang diminta.
4. Syntax `return ChocolateViewModel(source) as T}` mengembalikan instance `ViewModel` yang sudah dibuat.

activity_detail.xml

1. Syntax `package ScrollView` merupakan komponen yang memungkinkan isi layout dapat digulir secara vertikal jika kontennya lebih tinggi dari layar.
2. Syntax `ConstraintLayout` merupakan layout fleksibel dan powerful yang memungkinkan kita mengatur posisi elemen UI berdasarkan hubungan (constraint) antar elemen atau dengan parent.
3. Syntax `ImageView` untuk menampilkan gambar.
4. Syntax `TextView` merupakan program untuk menampilkan nama coklat, deskripsi.

activity_main.xml

1. Syntax `<LinearLayout>` merupakan layout linear yang menyusun elemen-elemen UI secara vertikal (satu di atas yang lain) karena `orientation="vertical"`.
2. Syntax `RecyclerView` merupakan komponen yang digunakan untuk menampilkan daftar item (seperti daftar coklat) secara efisien dan bisa discroll. Syntax `ImageView` untuk menampilkan gambar.

row_chocolate.xml

1. Syntax `<androidx.cardview.widget.CardView` membuat sebuah card dengan sudut membulat.
2. Syntax `<androidx.constraintlayout.widget.ConstraintLayout` merupakan komponen yang digunakan sebagai container utama di dalam card.
3. Syntax `android:id="@+id/imageChocolate"` merupakan gambar untuk menampilkan foto coklat.

4. Syntax

`<LinearLayout android:id="@+id/infoLayout" android:orientation="vertical"` merupakan container vertikal untuk teks dan tombol di sebelah kanan gambar.

5. Syntax `<TextView android:id="@+id/textName"` merupakan komponen yang digunakan sebagai menampilkan nama coklat..

6. Syntax `<TextView android:id="@+id/textDescription"` merupakan komponen yang digunakan untuk menampilkan deskripsi singkat coklat.

7. Syntax `<Button android:id="@+id/buttonImdb"` `<Button android:id="@+id/buttonDetail"` merupakan tombol untuk membuka link dan detail coklat.

MainActivity.kt (Jetpack Compose):

1. Syntax `package package com.example.scrollablecompose` merupakan deklarasi nama package file Kotlin.

2. Syntax `class MainActivity : ComponenttActivity() {}` merupakan activity untuk Jetpack Compose yang berisi program UI penuh di layer, mengatur UI dengan compose, dan program untuk tema dan memanggil composable utama.

3. Syntax `override fun onCreate(savedInstanceState: Bundle?)` `{super.onCreate(savedInstanceState)}` merupakan override fungsi `onCreate` untuk menginisialisasi UI saat activity dibuat.

4. Syntax `Timber.plant(Timber.DebugTree())` merupakan program untuk enginisialisasi Timber, library logging, agar mencatat log selama pengembangan/debugging.

5. Syntax `enableEdgeToEdge()` untuk mengaktifkan mode Edge-to-Edge UI agar konten aplikasi bisa tampil full screen sampai ke tepi layar.

6. Syntax `setContent {` merupakan fungsi memulai blok Compose UI. Semua UI dibuat di dalam blok ini.

7. Syntax `val NavController = rememberNavController()` merupakan program untuk membuat dan mengingat (mem-remember) instance `NavController` untuk navigasi antar layar Compose.
8. Syntax `val viewModel: ChocolateViewModel = viewModel(factory = ChocolateViewModelFactory("parameter-mu"))` merupakan program untuk mengambil instance `ViewModel` `ChocolateViewModel` dengan bantuan `ChocolateViewModelFactory`.
9. Syntax `ScrollableComposeTheme {` merupakan program membungkus seluruh UI dengan tema aplikasi khusus `ScrollableComposeTheme` yang mengatur warna, typography, dan lain-lain.
10. Syntax `Scaffold(modifier = Modifier.fillMaxSize()) {` `innerPadding ->` merupakan layout dasar yang mendukung struktur UI standar seperti app bar, floating button, dan konten utama. `InnerPadding` adalah padding internal yang `Scaffold` berikan agar konten tidak tertutup oleh elemen UI seperti app bar.
11. Syntax `NavHost() {` merupakan container navigasi yang mengelola switching antar layar berdasarkan rute.
12. Syntax `composable("list") {` merupakan program untuk mendefinisikan route "list" untuk layar daftar cokelat.
13. Syntax `ListScreen() {` merupakan composable `ListScreen` untuk menampilkan daftar cokelat.
14. Syntax `DetailScreen(id = id, viewModel = viewModel)` merupakan program untuk memanggil composable `DetailScreen` untuk menampilkan detail cokelat berdasarkan id.

Chocolate.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablelistxml` merupakan deklarasi package tempat file ini berada, yaitu package data, `com.example.scrollablexml`.

2. Syntax `data class Chocolate(` merupakan definisi data class bernama `Chocolate`, data class secara otomatis menyediakan fungsi-fungsi penting, class ini biasanya dipakai untuk menyimpan data sederhana, seperti model atau entitas.
3. Syntax `val id: Int, val name: String, val description: String, val chocolateImageId: Int, val btnLink: String` mendeklarasikan variabel dengan tipe datanya.

ChocolateProvider.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablexml.data` merupakan deklarasi package tempat file ini berada, yaitu di package `data`, `com.example.scrollablexml`.
2. Syntax `object ChocolateProvider {` digunakan untuk membuat singleton, yaitu satu instance global dan menyimpan data list coklat yang bisa diakses dari mana saja.
3. Syntax `val chocolateList = listOf(` mendeklarasikan properti immutable (`val`) yang berisi daftar objek `Chocolate` dan menggunakan `listOf()` untuk membuat list statis.

ChocolateListItem.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablecompose` merupakan deklarasi nama package file Kotlin.
2. Syntax `@Composable fun ChocolateListItem(chocolate: Chocolate, onDetailClick: (Chocolate) -> Unit) {` merupakan program untuk menandai fungsi sebagai composable.
3. Syntax `Card () {` merupakan program untuk latar belakang.
4. Syntax `Row () {` merupakan row utama untuk mengatur konsen dengan berbagai ketentuan.
5. Syntax `Image ()` merupakan fungsi untuk menampilkan gambar dengan resource ID.
6. Syntax `Spacer(modifier = Modifier.width(12.dp))` merupakan fungsi untuk memberi jarak horizontal 12dp antara gambar dan konten teks.

7. Syntax `Column() {}` merupakan program untuk mengatur teks dan tombol pada aplikasi.
8. Syntax `Text(text = chocolate.title), Text(text = "No: ${chocolate.no}"), Text(text = chocolate.description, style = MaterialTheme.typography.bodySmall, modifier = Modifier.padding(top = 4.dp))` merupakan program untuk menampilkan judul, nomor, dan deskripsi item.
9. Syntax `Button(onClick = {Timber.d("Link button clicked: ${chocolate.btnLink}") val intent = Intent(Intent.ACTION_VIEW, Uri.parse(chocolate.btnLink)) context.startActivity(intent)}) {Text("IMDB")}` merupakan program untuk mencatat log dan membuka browser ke URL yang ada di `chocolate.btnLink` menggunakan `Intent.ACTION_VIEW`.
10. Syntax `Button(onClick = {Timber.d("Detail button clicked: ${chocolate.title}") onDetailClicked(chocolate)}) {Text("Detail")}` merupakan program untuk mencatat mencatat log dan memanggil callback `onDetailClicked` dengan objek coklat.

ChocolateListScreen.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablecompose.uiComponent` merupakan deklarasi nama package `UIComponent`.
2. Syntax `@Composable fun ListScreen(chocolates: List<Chocolate>, onDetailClicked: (Chocolate) -> Unit) {` merupakan program untuk mendeklarasikan fungsi composable `ListScreen`.
3. Syntax `LazyColumn {` merupakan program untuk membuat list vertikal yang bisa di-scroll dengan performa optimal. Berbeda dengan `Column`, `LazyColumn` hanya membuat dan menampilkan item yang terlihat saja.
4. Syntax `items(chocolates) { chocolate -> }` merupakan program untuk membuat elemen list untuk setiap item di dalam daftar `chocolates`.

5. Syntax `ChocolateItem(chocolate = chocolate, onDetailClicked = onDetailClicked)` merupakan program untuk memanggil composable `ChocolateItem` untuk setiap item di list.

DetailScreen.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablecompose.uiScreen` merupakan deklarasi nama package `uiScreen`.
2. Syntax `@Composable fun DetailScreen(id: Int?, viewModel: ChocolateViewModel) {` merupakan program untuk mendefinisikan fungsi composable `DetailScreen`.
3. Syntax `val chocolate = id?.let { viewModel.getChocolateById(it) }` merupakan program untuk mengambil data `Chocolate` berdasarkan ID menggunakan `ViewModel`.
4. Syntax `if (chocolate != null) {` merupakan program untuk mengecek apakah data `chocolate` ditemukan.
5. Syntax `Timber.d("Navigating to Detail of: ${chocolate.title}")` merupakan program untuk menampilkan log debug jika data ditemukan.
6. Syntax `Column(modifier = Modifier.padding(16.dp).fillMaxWidth()) {` merupakan program untuk mengatur layout vertikal (atas ke bawah) dengan padding dan lebar penuh.
7. Syntax `Image()` merupakan program untuk menampilkan gambar dari resource berdasarkan ID dari objek `chocolate`.
8. Syntax `Text(text = chocolate.title, style = MaterialTheme.typography.titleLarge, modifier = Modifier.weight(1f))` merupakan program untuk menampilkan judul `chocolate` dalam satu baris (Row).
9. Syntax `Text(text = chocolate.description, style = MaterialTheme.typography.bodyMedium)` merupakan program untuk menampilkan deskripsi `chocolate` menggunakan gaya teks dari `MaterialTheme`.

10. Syntax `} else {Timber.e("Chocolate with ID $id not found")Text("Data tidak ada", modifier =Modifier.padding(16.dp))}` merupakan program Jika chocolate null (tidak ditemukan): Log error menggunakan Timber dan menampilkan teks “Data tidak ada”.

DetailScreen.kt (Jetpack Compose):

1. Syntax `package com.example.scrollablecompose.uiScreen` merupakan deklarasi nama package uiScreen.
2. Syntax `class ChocolateViewModelFactory(private val param: String) : ViewModelProvider.Factory {` merupakan program untuk membuat ViewModelFactory kustom.
3. Syntax `override fun <Choco : ViewModel> create(modelClass: Class<Choco>): Choco {` merupakan program Override fungsi create() untuk menghasilkan instance ViewModel.
4. Syntax `if (modelClass.isAssignableFrom(ChocolateViewModel::class.java)) {return ChocolateViewModel(param) as Choco}` merupakan program untuk mengecek apakah modelClass adalah ChocolateViewModel, lalu membuat instance-nya.
5. Syntax `class ChocolateViewModel(private val param: String) : ViewModel() {` merupakan program untuk menyimpan dan mengelola data list coklat.
6. Syntax `private val _chocolateList = MutableStateFlow<List<Chocolate>>(emptyList())val chocolateList: StateFlow<List<Chocolate>> = _chocolateList` merupakan program mutable (bisa diubah) tapi hanya bisa diakses internal.
7. Syntax `Image()` merupakan program untuk menampilkan gambar dari resource berdasarkan ID dari objek chocolate.
8. Syntax `private fun loadChocolates() {_chocolateList.value = ChocolateProvider.chocolateListTimber.d("Chocolate`

```
list loaded: ${_chocolateList.value.size}
items") } merupakan program untuk mengambil data dari ChocolateProvider dan
simpan ke _chocolateList.
```

9. Syntax `fun getChocolateById(id: Int): Chocolate? {return
_chocolateList.value.find { it.no == id }}` merupakan
program untuk mendapatkan 1 objek chocolate berdasarkan nomor. Berguna untuk
menampilkan detail berdasarkan ID.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

R. Pembahasan

Karena RecyclerView masih kompatibel saat ini, terlebih keterbatasan LazyColumn untuk kasus-kasus spesifik, seperti pagination yang kompleks, data binding tingkat lanjut, multi-view types, dan item click listeners. Kemudian RecyclerView juga dapat memberikan lebih banyak kontrol terkait optimasi memori dengan fitur seperti ViewPool, yang memungkinkan daftar item tampilan yang sudah ada untuk meningkatkan kinerja. Selain itu, perpindahan dari XML ke Jetpack Compose juga memerlukan waktu dan perubahan besar dalam struktur aplikasi apalagi aplikasi yang dari awal dikembangkan dengan XML yang membutuhkan pemahaman dan penguasaan baru untuk bisa berpindah menggunakan Jetpack Compose.

S. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/rizkiirr/PraktikumMobile/tree/main/PRAKTIKUM%20MOBILE/Modu>

14

MODUL 5: Connect to the Internet

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
<https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

T. Source Code

Jetpack Compose:

MainActivity.kt

Table 38. MainActivity.kt

1	<code>package com.example.movie</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import androidx.activity.ComponentActivity</code>
5	<code>import androidx.activity.compose.setContent</code>

6	<code>import androidx.compose.foundation.layout.Box</code>
7	<code>import androidx.compose.foundation.layout.fillMaxSize</code>
8	<code>import androidx.compose.foundation.layout.padding</code>
9	<code>import androidx.compose.material3.*</code>
10	<code>import androidx.compose.runtime.collectAsState</code>
11	<code>import androidx.compose.runtime.getValue</code>
12	<code>import androidx.compose.ui.Alignment</code>
13	<code>import androidx.compose.ui.Modifier</code>
14	<code>import androidx.compose.ui.text.style.TextAlign</code>
15	<code>import androidx.compose.ui.unit.dp</code>
16	<code>import androidx.lifecycle.viewmodel.compose.viewModel</code>
17	<code>import androidx.navigation.NavType</code>
18	<code>import androidx.navigation.compose.NavHost</code>
19	<code>import androidx.navigation.compose.composable</code>
20	<code>import androidx.navigation.compose.rememberNavController</code>
21	<code>import androidx.navigation.navArgument</code>
22	<code>import androidx.room.Room</code>
23	<code>import com.example.movie.data.local.MovieDatabase</code>
24	<code>import com.example.movie.data.remote.ApiService</code>
25	<code>import com.example.movie.data.repository.MovieRepositoryImpl</code>
26	<code>import</code> <code>com.example.movie.domain.usecase.GetPopularMoviesUseCase</code>
27	<code>import com.example.movie.ui.screen.MovieDetailScreen</code>
28	<code>import com.example.movie.ui.screen.MovieListScreen</code>
29	<code>import com.example.movie.ui.theme.MovieTheme</code>
30	<code>import com.example.movie.ui.viewmodel.MovieViewModel</code>
31	<code>import com.example.movie.ui.viewmodel.MovieViewModelFactory</code>
32	<code>import</code> <code>com.jakewharton.retrofit2.converter.kotlinx.serialization.as</code> <code>ConverterFactory</code>
33	<code>import kotlinx.serialization.json.Json</code>
34	<code>import okhttp3.MediaType.Companion.toMediaType</code>
35	<code>import okhttp3.OkHttpClient</code>

```

36 import okhttp3.logging.HttpLoggingInterceptor
37 import retrofit2.Retrofit
38
39 class MainActivity : ComponentActivity() {
40     @OptIn(ExperimentalMaterial3Api::class)
41     override fun onCreate(savedInstanceState: Bundle?) {
42         super.onCreate(savedInstanceState)
43
44         val loggingInterceptor =
45             HttpLoggingInterceptor().apply { level =
46                 HttpLoggingInterceptor.Level.BODY }
47
48         val okHttpClient =
49             OkHttpClient.Builder().addInterceptor(loggingInterceptor).bu
50             ild()
51
52         val apiService = Retrofit.Builder()
53             .baseUrl("https://api.themoviedb.org/3/")
54             .client(okHttpClient)
55             .addConverterFactory(Json { ignoreUnknownKeys =
56                 true }.asConverterFactory("application/json".toMediaType()))
57             .build()
58             .create(ApiService::class.java)
59
60         val movieDatabase = Room.databaseBuilder(
61             applicationContext,
62             MovieDatabase::class.java, "movie_database"
63         ).build()
64
65         val movieRepository = MovieRepositoryImpl(apiService,
66             movieDatabase)
67
68         val getPopularMoviesUseCase =
69             GetPopularMoviesUseCase(movieRepository)
70
71         val viewModelFactory =
72             MovieViewModelFactory(getPopularMoviesUseCase)
73
74     }
75 }

```

```

62         setContent {
63             MovieTheme {
64                 val navController = rememberNavController()
65                 val viewModel: MovieViewModel =
viewModel(factory = viewModelFactory)
66                 val state by viewModel.state.collectAsState()

67                 NavHost(
68                     navController = navController,
69                     startDestination = "movie_list"
70                 ) {
71                     composable("movie_list") {
72                         Scaffold(
74                             modifier =
Modifier.fillMaxSize(),
75                             topBar = {
77                                 TopAppBar(
78                                     title = { Text("Popular
Movies") }
80                                 )
82                             }
84                         ) { innerPadding ->
85                             Box(
86                                 modifier = Modifier
88                                     .padding(innerPadding)
89                                     .fillMaxSize(),
90                                 contentAlignment =
Alignment.Center
91                             ) {
92                                 if (state.isLoading) {
93
94                                     CircularProgressIndicator()
95                                 }
96

```

```

97     state.error?.let { error ->
98         Text(
99             text = "Terjadi
100 Kesalahan:\n$error",
101             modifier =
102             Modifier.padding(16.dp),
103             textAlign =
104             TextAlign.Center
105         )
106     }
107     if (!state.isLoading &&
108         state.error == null) {
109         MovieListScreen(
110             movies =
111             state.movies,
112             onClick = {
113                 movieId ->
114                 navController.navigate("movie_detail/$movieId")
115             }
116         )
117     }
118     composable(
119         route = "movie_detail/{movieId}",
120         arguments =
121         listOf(navArgument("movieId") { type = NavType.IntType })
122     ) { backStackEntry ->
123         val movieId =
124         backStackEntry.arguments?.getInt("movieId")

```

123	val selectedMovie = state.movies.find
	{ it.id == movieId }
124	
125	if (selectedMovie != null) {
126	MovieDetailScreen(
127	movie = selectedMovie,
128	onNavigateBack = {
129	navController.popBackStack()
130	}
131	)
132	} else {
133	Box(modifier =
	Modifier.fillMaxSize(), contentAlignment = Alignment.Center)
134	{
135	Text("Film tidak ditemukan.")
136	}
137	}
138	}
139	}
140	}
141	}
142	}
143	}

MovieDao.kt

Table 39. MovieDao.kt

1	package com.example.movie.data.local
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import kotlinx.coroutines.flow.Flow

8	
9	@Dao
10	interface MovieDao {
11	
12	@Insert(onConflict = OnConflictStrategy.REPLACE)
13	suspend fun insertAll(movies: List<MovieEntity>)
14	
15	@Query("SELECT * FROM movies")
16	fun getAllMovies(): Flow<List<MovieEntity>>
17	
18	@Query("DELETE FROM movies")
19	suspend fun clearAll()
20	}

MovieDatabase.kt

Table 40. MovieDatabase.kt

1	package com.example.movie.data.local
2	
3	import androidx.room.Database
4	import androidx.room.RoomDatabase
5	
6	@Database(
7	entities = [MovieEntity::class],
8	version = 1,
9	exportSchema = false
10	)
11	abstract class MovieDatabase : RoomDatabase() {
12	
13	abstract fun movieDao(): MovieDao
14	}

MovieEntity.kt

Table 41. MovieEntity.kt

1	package com.example.movie.data.local
---	--------------------------------------

2	
3	<code>import androidx.room.Entity</code>
4	<code>import androidx.room.PrimaryKey</code>
5	
6	<code>@Entity(tableName = "movies")</code>
7	<code>data class MovieEntity(</code>
8	<code> @PrimaryKey</code>
9	<code> val id: Int,</code>
10	<code> val title: String,</code>
11	<code> val overview: String,</code>
12	<code> val posterUrl: String,</code>
13	<code> val releaseDate: String</code>
14	<code>)</code>

MovieListResponse.kt

Table 42. *MovieListResponse.kt*

1	<code>package com.example.movie.data.remote.datatransferobjects</code>
2	
3	<code>import kotlinx.serialization.SerialName</code>
4	<code>import kotlinx.serialization.Serializable</code>
5	
6	<code>@Serializable</code>
7	<code>data class MovieListResponse(</code>
8	<code> @SerialName("results")</code>
9	<code> val results: List<MovieResult></code>
10	<code>)</code>
11	
12	<code>@Serializable</code>
13	<code>data class MovieResult(</code>
14	<code> @SerialName("id")</code>
15	<code> val id: Int,</code>
16	<code> @SerialName("title")</code>
17	<code> val title: String,</code>
18	<code> @SerialName("overview")</code>

19	val overview: String,
20	@SerializedName("poster_path")
21	val posterPath: String,
22	@SerializedName("release_date")
23	val releaseDate: String
24	)

ApiService.kt

Table 43. ApiService.kt

1	package com.example.movie.data.remote
2	
3	import com.example.movie.data.remote.datatransferobjects.MovieListR esponse
4	import retrofit2.http.GET
5	import retrofit2.http.Query
6	
7	interface ApiService {
8	@GET("movie/popular")
9	suspend fun getPopularMovies(10 @Query("api_key") apiKey: String 11): MovieListResponse
12	}

MovieRepositoryImpl.kt

Table 44. MovieRepositoryImpl.kt

1	package com.example.movie.data.repository
2	
3	import android.util.Log
4	import com.example.movie.data.local.MovieDatabase
5	import com.example.movie.data.remote.ApiService
6	import com.example.movie.data.toDomain
7	import com.example.movie.data.toEntity

```

8 import com.example.movie.domain.model.Movie
9 import com.example.movie.domain.repository.MovieRepository
10 import kotlinx.coroutines.flow.Flow
11 import kotlinx.coroutines.flow.first
12 import kotlinx.coroutines.flow.flow
13
14 class MovieRepositoryImpl(
15     private val apiService: ApiService,
16     private val db: MovieDatabase
17 ) : MovieRepository {
18
19     private val dao = db.movieDao()
20
21     override fun getPopularMovies():
22     Flow<Result<List<Movie>>> = flow {
23         val cachedMovies = dao.getAllMovies().first().map {
24             it.toDomain() }
25         emit(Result.success(cachedMovies))
26
27         try {
28             val response = apiService.getPopularMovies(apiKey
29             = "33e3d3edb5f09019aeba353631ab45a9")
30             Log.d("MOVIE_APP_DEBUG", "API Response: Received
31             ${response.results.size} movies from network.")
32             val movieEntities = response.results.map {
33                 it.toEntity() }
34             dao.clearAll()
35             dao.insertAll(movieEntities)
36
37             val newCachedMovies =
38             dao.getAllMovies().first().map { it.toDomain() }
39             emit(Result.success(newCachedMovies))
40

```

35	} catch (e: Exception) {
36	Log.e("MOVIE_APP_DEBUG", "Gagal mengambil data dari API. Error: ", e)
37	
38	emit(Result.failure(e))
39	}
40	}
41	}

Mapper.kt

Table 45. Mapper.kt

1	package com.example.movie.data
2	
3	import com.example.movie.data.local.MovieEntity
4	import com.example.movie.data.remote.datatransferobjects.MovieResult
5	import com.example.movie.domain.model.Movie
6	
7	fun MovieResult.toEntity(): MovieEntity {
8	return MovieEntity(id = this.id, title = this.title, overview = this.overview, posterUrl = "https://image.tmdb.org/t/p/w500\${this.posterPath}", releaseDate = this.releaseDate)
9	
10	
11	
12	
13	
14	)
15	}
16	
17	fun MovieEntity.toDomain(): Movie {
18	return Movie(id = this.id, title = this.title, overview = this.overview, posterUrl = this.posterUrl, releaseDate = this.releaseDate)
19	
20	

21	overview = this.overview,
22	posterUrl = this.posterUrl,
23	releaseDate = this.releaseDate
24	)
25	}

Movie.kt

Table 46. Movie.kt

1	package com.example.movie.domain.model
2	
3	data class Movie(
4	val id: Int,
5	val title: String,
6	val overview: String,
7	val posterUrl: String,
8	val releaseDate: String
9	)

MovieRepository.kt

Table 47. MovieRepository.kt

1	package com.example.movie.domain.repository
2	
3	import com.example.movie.domain.model.Movie
4	import kotlinx.coroutines.flow.Flow
5	
6	interface MovieRepository {
7	fun getPopularMovies(): Flow<Result<List<Movie>>>
8	}

GetPopularMoviesUseCase.kt

Table 48. GetPopularMoviesUseCase.kt

1	package com.example.movie.domain.usecase
2	
3	import com.example.movie.domain.model.Movie

4	import com.example.movie.domain.repository.MovieRepository
5	import kotlinx.coroutines.flow.Flow
6	
7	class GetPopularMoviesUseCase(
8	private val repository: MovieRepository
9	) {
10	
11	operator fun invoke(): Flow<Result<List<Movie>>> {
12	return repository.getPopularMovies()
13	}
14	}

MovieItem.kt

Table 49. MovieItem.kt

1	package com.example.movie.ui.component
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.material3.Button
8	import androidx.compose.material3.Card
9	import androidx.compose.material3.MaterialTheme
10	import androidx.compose.material3.Text
11	import androidx.compose.runtime.Composable
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.draw.clip
15	import androidx.compose.ui.layout.ContentScale
16	import androidx.compose.ui.platform.LocalContext
17	import androidx.compose.ui.unit.dp
18	import coil.compose.AsyncImage
19	import com.example.movie.domain.model.Movie
20	import java.net.URLEncoder

```

21
22 @Composable
23 fun MovieItem(
24     movie: Movie,
25     onDetailClick: (Int) -> Unit
26 ) {
27     val context = LocalContext.current
28
29     Card(
30         modifier = Modifier
31             .padding(8.dp)
32             .fillMaxWidth(),
33         shape = RoundedCornerShape(16.dp)
34     ) {
35         Row(
36             modifier = Modifier.padding(8.dp),
37             verticalAlignment = Alignment.CenterVertically
38         ) {
39             AsyncImage(
40                 model = movie.posterUrl,
41                 contentDescription = "Poster film"
42                 ${movie.title}",
43                 contentScale = ContentScale.Crop,
44                 modifier = Modifier
45                     .width(100.dp)
46                     .height(140.dp)
47                     .clip(RoundedCornerShape(12.dp))
48             )
49
50             Spacer(modifier = Modifier.width(12.dp))
51
52             Column(
53                 modifier = Modifier
54                     .padding(16.dp)

```

53	.weight(1f)	
54	) {	
55	Text(text = movie.title, style =	
	MaterialTheme.typography.titleMedium)	
56	Spacer(modifier = Modifier.height(4.dp))	
57	Text(text = "Rilis: \${movie.releaseDate}")	
58	Spacer(modifier = Modifier.height(4.dp))	
59	Text(	
60	text = movie.overview,	
61	style =	
	MaterialTheme.typography.bodySmall,	
62	maxLines = 3,	
63	modifier = Modifier.padding(top = 4.dp)	
64	)	
65		
66	Row(modifier = Modifier.padding(top = 8.dp))	
67	{	
68	Button(onClick = {	
69	try {	
70	val encodedTitle =	
	URLEncoder.encode(movie.title, "UTF-8")	
71	val url =	
	"https://www.imdb.com/find?q=\$encodedTitle"	
72	val intent =	
	Intent(Intent.ACTION_VIEW, Uri.parse(url))	
	context.startActivity(intent)	
73	} catch (e: Exception) {	
74	e.printStackTrace()	
75	}	
76	}) {	
77	Text("IMDb")	
78	}	
79	Spacer(modifier =	
80	Modifier.padding(horizontal = 4.dp))	

81	Button (onClick = {
	onDetailClick (movie.id) }) {
82	Text ("Detail")
83	}
84	}
85	}
86	}
87	}
88	}

MovieDetailScreen.kt

Table 50. MovieDetailScreen.kt

1	package com.example.movie.ui.screen
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.rememberScrollState
5	import androidx.compose.foundation.verticalScroll
6	import androidx.compose.material.icons.Icons
7	import
	androidx.compose.material.icons.automirrored.filled.ArrowBack
	k
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.draw.clip
12	import androidx.compose.ui.layout.ContentScale
13	import androidx.compose.ui.text.font.FontWeight
14	import androidx.compose.ui.unit.dp
15	import androidx.compose.ui.unit.sp
16	import coil.compose.AsyncImage
17	import com.example.movie.domain.model.Movie
18	
19	@OptIn (ExperimentalMaterial3Api::class)
20	@Composable

```

21 fun MovieDetailScreen(
22     movie: Movie,
23     onBackPressed: () -> Unit
24 ) {
25     Scaffold(
26         topBar = {
27             TopAppBar(
28                 title = { Text(text = "Detail Film") },
29                 navigationIcon = {
30                     IconButton(onClick = onBackPressed) {
31                         Icon(
32                             imageVector =
33                             Icons.AutoMirrored.Filled.ArrowBack,
34                             contentDescription = "Kembali"
35                         )
36                     }
37                 }
38             )
39         } { innerPadding ->
40             Column(
41                 modifier = Modifier
42                     .padding(innerPadding)
43                     .padding(16.dp)
44                     .verticalScroll(rememberScrollState())
45             ) {
46                 AsyncImage(
47
48                     model = movie.posterUrl,
49                     contentDescription = "Poster ${movie.title}",
50                     contentScale = ContentScale.Crop,
51                     modifier = Modifier
52                         .fillMaxWidth()
53                         .height(400.dp)

```

54	<code>.clip(MaterialTheme.shapes.large)</code>
55	<code>)</code>
56	<code>Spacer(modifier = Modifier.height(16.dp))</code>
57	<code>Text(</code>
58	<code>text = movie.title,</code>
59	<code>style</code> =
	<code>MaterialTheme.typography.headlineMedium,</code>
60	<code>fontWeight = FontWeight.Bold</code>
61	<code>)</code>
62	<code>Spacer(modifier = Modifier.height(8.dp))</code>
63	<code>Text(</code>
64	<code>text = "Rilis: \${movie.releaseDate}",</code>
65	<code>style = MaterialTheme.typography.bodyMedium,</code>
66	<code>fontSize = 16.sp</code>
67	<code>)</code>
68	<code>Spacer(modifier = Modifier.height(16.dp))</code>
69	<code>Text(</code>
70	<code>text = movie.overview,</code>
71	<code>style = MaterialTheme.typography.bodyLarge,</code>
72	<code>lineHeight = 24.sp</code>
73	<code>)</code>
74	<code>}</code>
75	<code>}</code>
76	<code>}</code>

MovieListScreen.kt

Table 51. MovieListScreen.kt

1	<code>package com.example.movie.ui.screen</code>
2	
3	<code>import androidx.compose.foundation.layout.PaddingValues</code>
4	<code>import androidx.compose.foundation.lazy.LazyColumn</code>
5	<code>import androidx.compose.foundation.lazy.items</code>
6	<code>import androidx.compose.runtime.Composable</code>
7	<code>import androidx.compose.ui.unit.dp</code>

8	import com.example.movie.domain.model.Movie
9	import com.example.movie.ui.component.MovieItem
10	
11	@Composable
12	fun MovieListScreen(
13	movies: List<Movie>,
14	onDetailClick: (Int) -> Unit
15	) {
16	LazyColumn(
17	contentPadding = PaddingValues(vertical = 8.dp)
18	) {
19	items(items = movies, key = { it.id }) { movie ->
20	MovieItem(
21	movie = movie,
22	onDetailClick = onDetailClick
23	)
24	}
25	}
26	}

MovieListState.kt

Table 52. MovieListState.kt

1	package com.example.movie.ui.screen
2	
3	import com.example.movie.domain.model.Movie
4	
5	data class MovieListState(
6	val isLoading: Boolean = false,
7	val movies: List<Movie> = emptyList(),
8	val error: String? = null,
9	)

MovieViewModel.kt

Table 53. *MovieViewModel.kt*

1	package com.example.movie.ui.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import
	com.example.movie.domain.usecase.GetPopularMoviesUseCase
6	import com.example.movie.ui.screen.MovieListState
7	import kotlinx.coroutines.flow.SharingStarted
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.map
10	import kotlinx.coroutines.flow.stateIn
11	
12	class MovieViewModel(
13	private val getPopularMoviesUseCase:
	GetPopularMoviesUseCase
	) : ViewModel() {
14	
15	val state: StateFlow<MovieListState> =
	getPopularMoviesUseCase()
16	.map { result ->
17	result.fold(
18	onSuccess = { movies ->
19	MovieListState(
20	isLoading = false,
21	movies = movies
22	)
23	},
24	onFailure = { error ->
25	MovieListState(
26	isLoading = false,
27	error = error.message
28	)
29	}

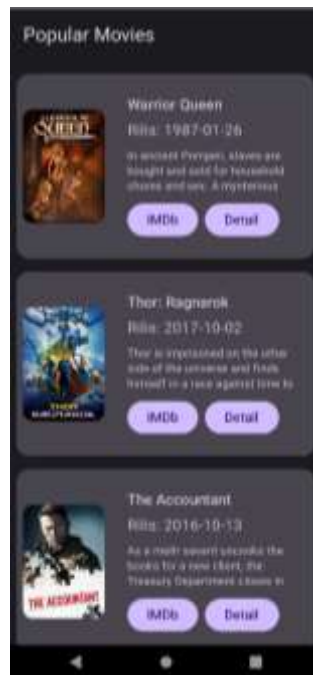
30	)
31	}
32	.stateIn(
33	scope = viewModelScope,
34	started = SharingStarted.WhileSubscribed(5000L),
35	initialValue = MovieListState(isLoading = true)
36	)
37	}

MovieViewModelFactory.kt

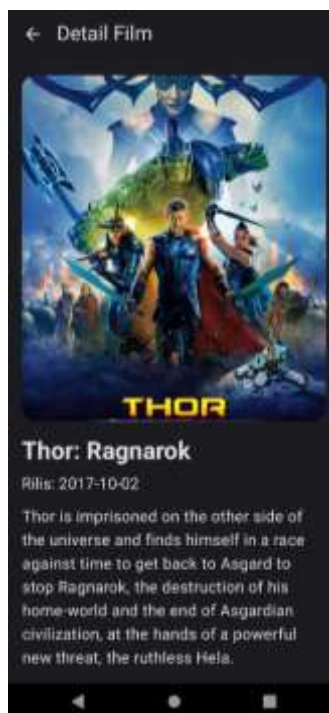
Table 54. MovieViewModelFactory.kt

1	package com.example.movie.ui.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
	com.example.movie.domain.usecase.GetPopularMoviesUseCase
6	
7	class MovieViewModelFactory(
8	private val getPopularMoviesUseCase:
	GetPopularMoviesUseCase
	) : ViewModelProvider.Factory {
9	override fun <T : ViewModel> create(modelClass: Class<T>):
	T {
10	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java)) {
11	@Suppress("UNCHECKED_CAST")
12	return MovieViewModel(getPopularMoviesUseCase) as
	T
13	}
14	throw IllegalArgumentException("Unknown ViewModel
	class")
15	}
16	}

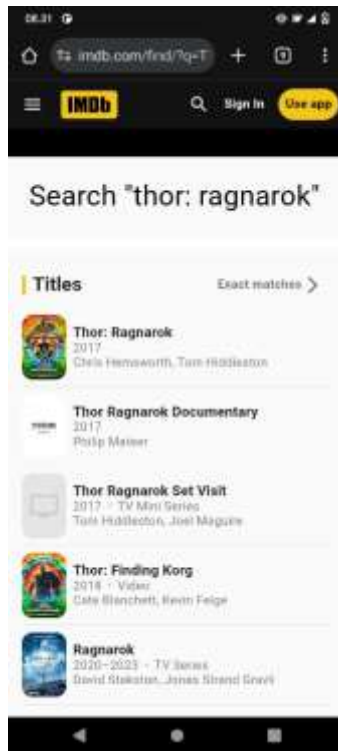
U. Output Program



Gambar 29. UI List



Gambar 30. UI Detail



Gambar 31. UI IMDb

V. Pembahasan

MainActivity.kt:

1. Syntax `package com.example.movie` merupakan deklarasi nama *package* untuk file Kotlin ini.
2. Syntax `class MainActivity : AppCompatActivity()` merupakan deklarasi kelas bernama `MainActivity` yang mewarisi (extends) kelas `ComponentActivity`, yang merupakan kelas dasar untuk aplikasi yang menggunakan Jetpack Compose.
3. Syntax `override fun onCreate(savedInstanceState: Bundle?)` merupakan *lifecycle method* pertama yang dipanggil saat *Activity* dibuat. Semua logika inisialisasi aplikasi ditempatkan di dalam method ini.
4. Syntax `super.onCreate(savedInstanceState)` adalah pemanggilan method `onCreate()` dari kelas induknya. Ini adalah panggilan wajib untuk memastikan *Activity* dapat berjalan dengan benar.

5. Syntax `val loggingInterceptor ... val apiService = Retrofit.Builder()...` adalah blok kode untuk mengkonfigurasi Retrofit, yaitu *library* untuk melakukan panggilan API (HTTP Client).
6. `HttpLoggingInterceptor` digunakan untuk mencatat (*log*) semua detail request dan response API ke Logcat, yang sangat berguna untuk proses *debugging*.
7. `OkHttpClient` dikonfigurasi untuk menggunakan *interceptor* tersebut.
8. `Retrofit.Builder()` membangun *instance* Retrofit dengan menentukan URL dasar API, `okHttpClient` yang sudah dibuat, dan *converter factory* dari Kotlinx Serialization untuk mengubah data JSON dari API menjadi objek data Kotlin secara otomatis.
9. Syntax `val movieDatabase = Room.databaseBuilder(...)` berfungsi untuk membuat *instance* dari Room Database. Room adalah *library* dari Jetpack untuk mengelola database SQLite lokal secara lebih mudah dan aman. Database ini diberi nama "movie_database".
10. Syntax `val movieRepository = ..., val getPopularMoviesUseCase = ..., val viewModelFactory = ...` adalah serangkaian program untuk melakukan Dependency Injection secara manual.
11. `MovieRepositoryImpl` dibuat dengan memberikan `apiService` (untuk data remote) dan `movieDatabase` (untuk data lokal).
12. `GetPopularMoviesUseCase` dibuat dengan `movieRepository` sebagai dependensinya.
13. `MovieViewModelFactory` dibuat dengan `getPopularMoviesUseCase`. Factory ini nantinya akan digunakan untuk membuat `MovieViewModel` dengan dependensi yang sudah disiapkan.
14. Syntax `setContent { ... }` merupakan fungsi utama dari Jetpack Compose yang digunakan untuk mendefinisikan dan membangun seluruh tampilan antarmuka pengguna (UI) secara deklaratif di dalam *Activity*.
15. Syntax `MovieTheme { ... }` adalah sebuah *Composable function* yang menerapkan tema visual (seperti warna, tipografi, dan bentuk) yang telah didefinisikan di `ui.theme` ke semua komponen *Composable* yang ada di dalamnya.
16. Syntax `val navController = rememberNavController()` berfungsi untuk membuat dan mengingat sebuah `NavController`. Controller ini bertanggung jawab untuk mengelola navigasi antar layar (*Composable*) di dalam aplikasi.

17. Syntax `val viewModel: MovieViewModel = viewModel(factory = viewModelFactory)` digunakan untuk mendapatkan *instance* dari `MovieViewModel`. Dengan menggunakan `viewModel()`, Jetpack Compose akan secara otomatis mempertahankan *instance* `ViewModel` ini selama perubahan konfigurasi (misalnya rotasi layar), dan `viewModelFactory` digunakan untuk memastikan `ViewModel` dibuat dengan dependensi yang benar.
18. Syntax `val state by viewModel.state.collectAsState()` berfungsi untuk mengobservasi *state* (sebuah `StateFlow`) dari `ViewModel`. `collectAsState()` mengubah `Flow` menjadi `State` yang dapat dibaca oleh Compose. Setiap kali ada perubahan pada *state* di `ViewModel`, UI akan secara otomatis diperbarui (*recompose*).
19. Syntax `NavHost(...)` adalah komponen utama dari Jetpack Navigation Compose. Komponen ini berfungsi sebagai wadah yang akan menampilkan *Composable* tujuan (*destination*) sesuai dengan *route* saat ini yang dikelola oleh `NavController`. `startDestination` menetapkan layar pertama yang akan ditampilkan.
20. Syntax `composable("movie_list") { ... }` mendefinisikan sebuah layar atau *destination* dalam `NavHost` dengan *route* bernama "movie_list". Konten di dalam blok lambda ini adalah UI untuk layar daftar film.
21. Syntax `Scaffold(...)` adalah komponen *layout* dari Material 3 yang menyediakan struktur standar untuk layar, seperti `topBar` (bilah aplikasi atas), *floating action button*, dan area konten utama.
22. Syntax `if (state.isLoading) { ... }, state.error?.let { ... }` dan `if (!state.isLoading && state.error == null) { ... }` adalah blok logika untuk menangani tiga kondisi UI yang berbeda berdasarkan *state* dari `ViewModel`:
23. Jika `isLoading` bernilai `true`, tampilkan `CircularProgressIndicator`.
24. Jika `error` tidak `null`, tampilkan pesan kesalahan.
25. Jika tidak sedang *loading* dan tidak ada *error*, tampilkan `MovieListScreen` dengan data film.
26. Syntax `onDetailClick = { movieId -> navController.navigate("movie_detail/$movieId") }` adalah sebuah *lambda* yang diteruskan ke `MovieListScreen`. Ketika sebuah item film diklik, fungsi ini akan

- dieksekusi untuk menavigasikan pengguna ke layar detail dengan memanggil `navController.navigate()`. `movieId` dari film yang diklik disisipkan ke dalam *route*.
27. Syntax `composable("movie_detail/{movieId}", ...)` mendefinisikan layar detail film. `{movieId}` adalah sebuah *placeholder* untuk argumen yang akan diterima dari *route*. `arguments = listOf(...)` secara eksplisit mendefinisikan bahwa `movieId` adalah sebuah argumen dengan tipe data `Int`.
28. Syntax `val movieId = backStackEntry.arguments?.getInt("movieId")` berfungsi untuk mengambil nilai `movieId` yang dikirimkan melalui navigasi dari layar sebelumnya.
29. Syntax `val selectedMovie = state.movies.find { it.id == movieId }` mencari objek film yang sesuai di dalam daftar `state.movies` berdasarkan `movieId` yang telah diambil.
30. Syntax `if (selectedMovie != null) { ... } else { ... }` adalah pengecekan untuk memastikan film ditemukan. Jika `selectedMovie` ada, maka `MovieDetailScreen` akan ditampilkan. Jika tidak, pesan "Film tidak ditemukan" yang akan muncul.
31. Syntax `onNavigateBack = { navController.popBackStack() }` adalah sebuah *lambda* yang diteruskan ke `MovieDetailScreen`. Ketika tombol kembali di layar detail ditekan, `navController.popBackStack()` dipanggil untuk kembali ke layar sebelumnya (layar daftar film).

MovieDao.kt:

1. Syntax `suspend fun insertAll(movies: List<MovieEntity>)` adalah deklarasi fungsi untuk memasukkan daftar film ke dalam database.
2. `suspend`: *Keyword* ini menandakan bahwa fungsi ini adalah *coroutine function*. Ini memungkinkan operasi database (yang berpotensi berjalan lama) dieksekusi di *background thread* tanpa memblokir UI.
3. `movies: List<MovieEntity>`: Parameter ini adalah daftar objek `MovieEntity` yang akan disimpan ke dalam tabel.
4. Syntax `@Query("SELECT * FROM movies")` adalah anotasi yang berisi *query* SQL. *Query* ini berarti "ambil semua kolom (*) dari tabel movies". Room akan memverifikasi validitas *query* SQL ini pada saat kompilasi.
5. Syntax `fun getAllMovies(): Flow<List<MovieEntity>>` adalah deklarasi fungsi untuk mengambil semua data film dari database.

6. Flow<List<MovieEntity>>: Fungsi ini mengembalikan Flow, yang merupakan fitur dari Kotlin Coroutines. Keunggulannya adalah Flow bersifat reaktif. Setiap kali ada perubahan data di tabel movies (misalnya, setelah insertAll dipanggil), Flow ini akan secara otomatis memancarkan (*emit*) daftar data terbaru ke pengamatnya (*collector*). Ini memungkinkan UI diperbarui secara *real-time*.
7. Syntax @Query("DELETE FROM movies") adalah anotasi yang berisi *query* SQL untuk menghapus semua *record* atau baris data dari tabel movies.
8. Syntax suspend fun clearAll() adalah deklarasi fungsi untuk mengosongkan tabel movies. Kata kunci suspend digunakan agar operasi penghapusan ini juga berjalan di *background thread*.

MovieDatabase.kt:

1. Syntax package com.example.movie.data.local merupakan deklarasi nama *package* untuk file ini, menandakan bahwa file ini adalah bagian dari komponen data lokal aplikasi.
2. Syntax @Database(...) adalah anotasi utama dari Room untuk mengkonfigurasi database.
3. Syntax abstract class MovieDatabase : RoomDatabase() mendeklarasikan kelas database.
4. Syntax abstract fun movieDao(): MovieDao adalah sebuah fungsi abstrak di dalam kelas database.

MovieEntity.kt :

1. Syntax package com.example.movie.data.local merupakan deklarasi nama *package* untuk file ini, yang menunjukkan lokasinya di dalam struktur proyek sebagai bagian dari komponen data lokal.
2. Syntax @Entity(tableName = "movies") adalah anotasi dari Room yang menandai kelas MovieEntity sebagai sebuah tabel database.
3. tableName = "movies": Properti ini secara eksplisit menentukan nama tabel yang akan dibuat di dalam database SQLite. Jika tidak ditentukan, Room akan menggunakan nama kelas sebagai nama tabel secara default.

4. Syntax `data class MovieEntity(...)` mendeklarasikan sebuah data class Kotlin. data class sangat ideal untuk digunakan sebagai *entity* karena secara otomatis menyediakan fungsi-fungsi standar seperti `equals()`, `hashCode()`, dan `toString()`, yang berguna untuk mengelola objek data.
5. Syntax `@PrimaryKey` adalah anotasi yang menandai properti `id` sebagai kunci utama (primary key) dari tabel `movies`. *Primary key* adalah kolom yang nilainya harus unik untuk setiap baris, berfungsi sebagai pengidentifikasi unik untuk setiap *record*.
6. Syntax `val id: Int, val title: String, ...` adalah deklarasi properti-properti dari `MovieEntity`. Masing-masing properti ini akan dipetakan menjadi sebuah kolom di dalam tabel `movies` dengan tipe data yang sesuai.

MovieListResponse.kt:

1. Syntax `package com.example.movie.data.remote.datatransferobjects` merupakan deklarasi nama *package*, yang menunjukkan bahwa file ini berfungsi untuk transfer data dari sumber *remote* (API).
2. Syntax `@Serializable` adalah anotasi dari library Kotlinx Serialization. Anotasi ini memberitahu *compiler* bahwa kelas yang ditandai dapat diubah ke dan dari format lain, seperti JSON. Ini adalah langkah wajib agar proses deserialisasi otomatis dapat berjalan.
3. Syntax `data class MovieListResponse(...)` mendeklarasikan sebuah DTO yang merepresentasikan struktur level atas dari response JSON API. Sesuai dengan response dari The Movie DB, objek JSON utama memiliki sebuah *field* bernama "results".
4. Syntax `@SerializedName("results")` adalah anotasi yang memetakan nama *field* di dalam JSON ("results") ke nama properti di dalam kelas Kotlin (`results`). Ini sangat penting jika nama *field* di JSON tidak sesuai dengan konvensi penamaan di Kotlin (misalnya, `snake_case` di JSON vs. `camelCase` di Kotlin).
5. Syntax `val results: List<MovieResult>` adalah properti yang akan menampung daftar film. Tipe datanya adalah `List` dari `MovieResult`, yang berarti *field* "results" di JSON adalah sebuah *array* dari objek film.

6. Syntax `data class MovieResult(...)` mendeklarasikan DTO kedua yang merepresentasikan satu objek film di dalam *array* "results".
7. Properti di dalam `MovieResult`: Setiap properti di sini dipetakan ke sebuah *field* di dalam objek JSON film menggunakan anotasi `@SerializedName`.

ApiService.kt:

1. Syntax `package com.example.movie.data.remote` merupakan deklarasi nama *package*, yang mengindikasikan bahwa file ini adalah bagian dari komponen data *remote* (jaringan).
2. Syntax `import ...` berfungsi untuk mengimpor kelas dan anotasi yang dibutuhkan, yaitu `MovieListResponse` (DTO) dan anotasi dari Retrofit seperti `GET` dan `Query`.
3. Syntax `interface ApiService` mendeklarasikan sebuah *interface* Kotlin. Dalam Retrofit, *interface* digunakan untuk mendefinisikan sekumpulan panggilan API.
4. Syntax `@GET("movie/popular")` adalah anotasi dari Retrofit yang menandai fungsi `getPopularMovies` sebagai sebuah HTTP GET request.
5. Syntax `suspend fun getPopularMovies(...)` adalah deklarasi fungsi yang akan melakukan panggilan jaringan.
6. Syntax `@Query("api_key") apiKey: String` adalah anotasi Retrofit yang menambahkan sebuah *query parameter* ke dalam URL.
7. Syntax `MovieListResponse` menentukan tipe data yang akan dikembalikan oleh fungsi ini. Retrofit, bersama dengan *converter factory* (Kotlinx Serialization), akan secara otomatis mengubah (deserialize) respons JSON dari API menjadi sebuah objek `MovieListResponse`.

MovieRepositoryImpl.kt:

1. Syntax `package com.example.movie.data.repository` merupakan deklarasi nama *package*, menandakan bahwa file ini adalah bagian dari *layer* repositori data.
2. Syntax `class MovieRepositoryImpl(...) : MovieRepository` mendeklarasikan sebuah kelas yang mengimplementasikan *interface* `MovieRepository`.

3. Syntax `private val dao = db.movieDao()` adalah inisialisasi properti `dao` dengan mendapatkan *instance* `MovieDao` dari `MovieDatabase`. Ini adalah cara untuk mendapatkan akses ke fungsi-fungsi *query* database.
4. Syntax `override fun getPopularMovies(): Flow<Result<List<Movie>>> = flow { ... }` adalah implementasi dari fungsi yang didefinisikan di *interface* `MovieRepository`.
5. Syntax `val cachedMovies = dao.getAllMovies().first().map { it.toDomain() }` adalah langkah pertama di dalam *flow*.
6. Syntax `emit(Result.success(cachedMovies))` berfungsi untuk memancarkan (mengirim) data dari cache yang berhasil diambil. Ini memungkinkan UI untuk menampilkan data dengan cepat tanpa harus menunggu panggilan jaringan selesai.
7. Syntax `try { ... } catch (e: Exception) { ... }` adalah blok untuk menangani kesalahan jaringan. Semua operasi yang berhubungan dengan API ditempatkan di dalam blok `try`.
8. Syntax `val response = apiService.getPopularMovies(...)` melakukan panggilan jaringan ke API untuk mendapatkan daftar film populer. Perlu dicatat, API Key ditulis langsung di sini (hardcoded), yang umumnya tidak disarankan untuk aplikasi produksi.
9. Syntax `val movieEntities = response.results.map { it.toEntity() }` mengubah (memetakan) setiap objek `MovieResult` (DTO dari API) menjadi objek `MovieEntity` yang siap untuk disimpan ke database.
10. Syntax `dao.clearAll()` dan `dao.insertAll(movieEntities)` adalah proses untuk memperbarui cache. Pertama, semua data lama di tabel dihapus, kemudian data baru dari API dimasukkan.
11. Syntax `val newCachedMovies = dao.getAllMovies().first().map { it.toDomain() }` mengambil kembali data yang baru saja dimasukkan ke database dan mengubahnya menjadi model domain.
12. Syntax `emit(Result.success(newCachedMovies))` memancarkan kembali data yang sudah diperbarui dari API. Ini akan membuat UI secara otomatis menampilkan data yang paling segar.

13. Syntax `emit(Result.failure(e))` dieksekusi di dalam blok `catch` jika terjadi kesalahan selama panggilan API. Ini mengirimkan informasi kesalahan ke lapisan UI, sehingga pengguna dapat diberi tahu tentang masalah tersebut.
14. Syntax `Log.d(...)` dan `Log.e(...)` digunakan untuk mencatat informasi debug dan kesalahan ke Logcat, yang sangat membantu selama proses pengembangan dan pemecahan masalah.

Mapper.kt:

1. Syntax `package com.example.movie.data` merupakan deklarasi nama *package*, yang menempatkan file ini di dalam *layer* data aplikasi.
2. Syntax `fun MovieResult.toEntity(): MovieEntity` mendeklarasikan sebuah fungsi ekstensi untuk kelas `MovieResult`.
3. Syntax `fun MovieEntity.toDomain(): Movie` mendeklarasikan sebuah fungsi ekstensi untuk kelas `MovieEntity`.

Movie.kt

1. Syntax `package com.example.movie.domain.model` merupakan deklarasi nama *package*. Ini menempatkan file di dalam *layer* domain, sub-paket model, yang merupakan lokasi standar untuk kelas-kelas model inti dalam *Clean Architecture*.
2. Syntax `data class Movie(...)` mendeklarasikan sebuah data class Kotlin bernama `Movie`. Ini adalah representasi data final yang telah diproses dan siap digunakan oleh lapisan presentasi (UI).
3. Properti di dalam `Movie`:
 - `val id: Int`: ID unik dari film.
 - `val title: String`: Judul film.
 - `val overview: String`: Sinopsis atau ringkasan cerita film.
 - `val posterUrl: String`: URL lengkap yang sudah siap pakai untuk menampilkan gambar poster.
 - `val releaseDate: String`: Tanggal rilis film.

MovieRepository.kt:

1. Syntax `package com.example.movie.domain.repository` merupakan deklarasi nama *package*. Ini menempatkan *interface* di dalam *layer* domain, sub-paket repository, sesuai dengan prinsip *Clean Architecture*.
2. Syntax `import ...` berfungsi untuk mengimpor kelas model `Movie` dan `Flow` dari Kotlin Coroutines.
3. Syntax `interface MovieRepository` mendeklarasikan sebuah *interface*. *Interface* ini tidak berisi logika implementasi, melainkan hanya definisi fungsi yang harus disediakan oleh kelas mana pun yang mengimplementasikannya (dalam hal ini, `MovieRepositoryImpl`).
4. Syntax `fun getPopularMovies(): Flow<Result<List<Movie>>>>` mendefinisikan sebuah fungsi abstrak.

GetPopularMoviesUseCase.kt:

1. Syntax `package com.example.movie.domain.usecase` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini membantu mengatur kode dalam struktur hirarkis, menunjukkan bahwa ini adalah bagian dari domain usecase dalam aplikasi movie.
2. Syntax `import com.example.movie.domain.model.Movie` Baris ini mengimpor kelas `Movie` dari package `com.example.movie.domain.model`. Ini diperlukan agar kelas `Movie` dapat digunakan di dalam file ini untuk merepresentasikan objek film.
3. Syntax `import com.example.movie.domain.repository.MovieRepository` Ini mengimpor interface `MovieRepository` dari package `com.example.movie.domain.repository`. `MovieRepository` kemungkinan besar mendefinisikan kontrak untuk mendapatkan data film, dan akan digunakan di sini.
4. Syntax `import kotlinx.coroutines.flow.Flow` Baris ini mengimpor tipe `Flow` dari `kotlinx.coroutines.flow`. `Flow` adalah tipe reaktif dari Kotlin Coroutines yang digunakan untuk mengalirkan (stream) data secara asinkron, yang cocok untuk operasi jaringan atau database yang mungkin menghasilkan beberapa nilai dari waktu ke waktu.
5. Syntax `class GetPopularMoviesUseCase( private val repository: MovieRepository ) { ... }` Ini adalah deklarasi kelas `GetPopularMoviesUseCase`. Kelas ini adalah "use

case" atau "interactor", yang mewakili sebuah operasi bisnis tunggal: mendapatkan film-film populer. Ia memiliki constructor yang menerima sebuah instance dari MovieRepository. Kata kunci private val berarti repository adalah properti pribadi yang hanya bisa diakses di dalam kelas ini dan diinisialisasi saat objek dibuat.

6. Syntax operator fun invoke(): Flow<Result<List<Movie>>> { ... } Ini adalah deklarasi fungsi invoke() dengan kata kunci operator. Saat Anda menandai sebuah fungsi dengan operator fun invoke(), itu memungkinkan Anda memanggil instance kelas GetPopularMoviesUseCase seperti sebuah fungsi.
7. Syntax return repository.getPopularMovies() Di dalam fungsi invoke(), baris ini memanggil metode getPopularMovies() dari objek repository yang telah diinjeksikan (diberikan) ke dalam GetPopularMoviesUseCase. Hasil dari pemanggilan ini, yang diharapkan berupa Flow<Result<List<Movie>>>, kemudian dikembalikan oleh use case. Ini menunjukkan bahwa use case ini delegasikan tugas pengambilan data film populer ke MovieRepository.

MovieItem.kt:

1. Syntax package com.example.movie.ui.component Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa komponen UI ini merupakan bagian dari lapisan antarmuka pengguna (ui) dan khususnya bagian dari component yang dapat digunakan kembali dalam aplikasi.
2. Syntax @Composable fun MovieItem(movie: Movie, onDetailClick: (Int) -> Unit) { ... } Ini adalah deklarasi fungsi Composable MovieItem. Fungsi ini dirancang untuk menampilkan satu item film dalam daftar.
3. Syntax val context = LocalContext.current Baris ini mengambil Context Android saat ini, yang akan digunakan untuk memulai Intent (misalnya, membuka browser).
4. Syntax Card(modifier = Modifier .padding(8.dp) .fillMaxWidth(), shape = RoundedCornerShape(16.dp)) { ... } Ini adalah Composable Card yang menyediakan wadah dengan elevasi dan sudut membulat.
5. Syntax Row(modifier = Modifier.padding(8.dp), verticalAlignment = Alignment.CenterVertically) { ... } Ini adalah Composable Row, yang mengatur turunan-turunannya secara horizontal.

6. Syntax `AsyncImage( model = movie.posterUrl, contentDescription = "Poster film ${movie.title}", contentScale = ContentScale.Crop, modifier = Modifier .width(100.dp) .height(140.dp) .clip(RoundedCornerShape(12.dp)) )` Ini adalah Composable `AsyncImage` yang digunakan untuk menampilkan poster film.
7. Syntax `Spacer(modifier = Modifier.width(12.dp))` Ini adalah Composable `Spacer` yang berfungsi sebagai ruang kosong horizontal antara poster film dan kolom teks.
8. Syntax `Column( modifier = Modifier .padding(16.dp) .weight(1f) ) { ... }` Ini adalah Composable `Column`, yang mengatur turunan-turunannya secara vertikal.
9. Syntax `Text(text = movie.title, style = MaterialTheme.typography.titleMedium)` Ini adalah Composable `Text` yang menampilkan judul film. style diatur menggunakan `MaterialTheme.typography.titleMedium` untuk konsistensi tema.
10. Syntax `Text(text = "Rilis: ${movie.releaseDate}")` Ini adalah Composable `Text` yang menampilkan tanggal rilis film.
11. Syntax `Text( text = movie.overview, style = MaterialTheme.typography.bodySmall, maxLines = 3, modifier = Modifier.padding(top = 4.dp) )` Ini adalah Composable `Text` yang menampilkan ringkasan (overview) film.
12. Syntax `Row(modifier = Modifier.padding(top = 8.dp)) { ... }` Ini adalah Composable `Row` lain yang digunakan untuk menempatkan dua tombol (IMDb dan Detail) secara horizontal.
13. Syntax `Button(onClick = { ... }) { Text("IMDb") }` Ini adalah Composable `Button` untuk membuka halaman IMDb film.
14. Syntax `Spacer(modifier = Modifier.padding(horizontal = 4.dp))` Ini adalah Composable `Spacer` yang berfungsi sebagai ruang kosong horizontal kecil antara tombol IMDb dan Detail.
15. Syntax `Button(onClick = { onDetailClick(movie.id) }) { Text("Detail") }` Ini adalah Composable `Button` untuk melihat detail film lebih lanjut.

MovieDetailScreen.kt:

1. Syntax `package com.example.movie.ui.screen` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini merupakan bagian dari

lapisan antarmuka pengguna (ui) dan secara spesifik adalah sebuah screen (layar) dalam aplikasi.

2. Syntax `@OptIn(ExperimentalMaterial3Api::class)` Ini adalah anotasi opt-in yang menunjukkan bahwa kode di bawahnya menggunakan API yang masih bersifat eksperimental dalam Material3. Ini diperlukan untuk TopAppBar saat ini.
3. Syntax `@Composable fun MovieDetailScreen( movie: Movie, onBackPressed: () -> Unit ) { ... }` Ini adalah deklarasi fungsi Composable MovieDetailScreen. Fungsi ini dirancang untuk menampilkan detail lengkap sebuah film.
4. Syntax `Scaffold( topBar = { ... } ) { innerPadding -> ... }` Ini adalah Composable Scaffold, yang menyediakan struktur layout dasar untuk layar Material Design, seperti TopAppBar, FloatingActionButton, dan area konten utama.
5. Syntax `TopAppBar( title = { Text(text = "Detail Film") }, navigationIcon = { ... } )` Ini adalah Composable TopAppBar yang berfungsi sebagai bilah judul di bagian atas layar.
6. Syntax `IconButton(onClick = onBackPressed) { ... }` Ini adalah Composable IconButton yang ditempatkan di dalam navigationIcon TopAppBar.
7. Syntax `Icon( imageVector = Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Kembali" )` Ini adalah Composable Icon yang menampilkan ikon panah kembali dari koleksi ikon Material Design. contentDescription disediakan untuk aksesibilitas.
8. Syntax `Column( modifier = Modifier .padding(innerPadding) .padding(16.dp) .verticalScroll(rememberScrollState()) ) { ... }` Ini adalah Composable Column yang berisi seluruh konten detail film.
9. Syntax `AsyncImage( model = movie.posterUrl, contentDescription = "Poster ${movie.title}", contentScale = ContentScale.Crop, modifier = Modifier .fillMaxWidth() .height(400.dp) .clip(MaterialTheme.shapes.large) )` Ini adalah Composable AsyncImage untuk menampilkan poster film dengan ukuran yang lebih besar di layar detail.
10. Syntax `Spacer(modifier = Modifier.height(16.dp))` Ini adalah Composable Spacer yang berfungsi sebagai ruang kosong vertikal.

11. Syntax `Text( text = movie.title, style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.Bold )` Ini adalah Composable Text yang menampilkan judul film. `style` dan `fontWeight` digunakan untuk mengatur tampilan judul.
12. Syntax `Text( text = "Rilis: ${movie.releaseDate}", style = MaterialTheme.typography.bodyMedium, fontSize = 16.sp )` Ini adalah Composable Text yang menampilkan tanggal rilis film. `fontSize` diatur secara eksplisit.
13. Syntax `Text( text = movie.overview, style = MaterialTheme.typography.bodyLarge, lineHeight = 24.sp )` Ini adalah Composable Text yang menampilkan ringkasan (overview) film. `lineHeight` diatur untuk meningkatkan keterbacaan teks yang panjang.

MovieListScreen.kt:

1. Syntax `package com.example.movie.ui.screen` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini merupakan bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik adalah sebuah screen (layar) yang menampilkan daftar.
2. Syntax `@Composable fun MovieListScreen( movies: List<Movie>, onDetailClick: (Int) -> Unit ) { ... }` Ini adalah deklarasi fungsi Composable `MovieListScreen`. Fungsi ini dirancang untuk menampilkan daftar film.
3. Syntax `LazyColumn( contentPadding = PaddingValues(vertical = 8.dp) ) { ... }` Ini adalah Composable `LazyColumn`, container utama untuk menampilkan daftar film.
4. Syntax `items( items = movies, key = { it.id } ) { movie -> ... }` Ini adalah fungsi `items` yang digunakan di dalam `LazyColumn` untuk mengiterasi daftar `movies` dan membuat `MovieItem` untuk setiap film.
5. Syntax `MovieItem( movie = movie, onDetailClick = onDetailClick )` Di dalam lambda `items`, Composable `MovieItem` dipanggil.

MovieListState.kt:

1. Syntax `package com.example.movie.ui.screen` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini adalah bagian dari

lapisan antarmuka pengguna (ui) dan secara spesifik terkait dengan screen (layar), kemungkinan besar layar daftar film.

2. Syntax `import com.example.movie.domain.model.Movie` Ini mengimpor kelas `Movie` dari package `com.example.movie.domain.model`. Kelas `Movie` adalah model data yang merepresentasikan sebuah film, dan akan digunakan dalam `MovieListState` untuk menampung daftar film.
3. Syntax `data class MovieListState( val isLoading: Boolean = false, val movies: List<Movie> = emptyList(), val error: String? = null, )` Ini adalah deklarasi data class `MovieListState`. Dalam pengembangan Android modern, terutama dengan pendekatan MVI (Model-View-Intent) atau MVVM (Model-View-ViewModel), state (keadaan) sebuah layar sering kali direpresentasikan oleh sebuah *data class* tunggal. `MovieListState` ini dirancang untuk menampung semua informasi yang relevan dengan kondisi UI layar daftar film.
4. `val isLoading: Boolean = false`: Ini adalah properti yang menunjukkan status loading layar.
5. `val movies: List<Movie> = emptyList()`: Ini adalah properti yang menampung daftar objek `Movie`.
6. `val error: String? = null`: Ini adalah properti yang menampung pesan kesalahan jika terjadi masalah saat memuat data.

MovieViewModel.kt:

1. Syntax `package com.example.movie.ui.viewmodel` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini adalah bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik merupakan sebuah `viewmodel`, yang bertugas mengelola data dan logika bisnis untuk sebuah layar UI.
2. Syntax `class MovieViewModel( private val getPopularMoviesUseCase: GetPopularMoviesUseCase ) : ViewModel() { ... }` Ini adalah deklarasi kelas `MovieViewModel`. Kelas ini mewarisi dari `ViewModel` Android.
3. Syntax `val state: StateFlow<MovieListState> = getPopularMoviesUseCase()` Ini adalah deklarasi properti state yang bertipe `StateFlow<MovieListState>`. Properti

ini yang akan diobservasi oleh UI (Composable) untuk mendapatkan pembaruan tentang keadaan layar daftar film.

4. Syntax `.map { result -> ... }` Fungsi `map` ini mengubah `Flow<Result<List<Movie>>>` yang dipancarkan oleh *use case* menjadi `MovieListState`.
5. Syntax `.stateIn( scope = viewModelScope, started = SharingStarted.WhileSubscribed(5000L), initialValue = MovieListState(isLoading = true) )` Fungsi `stateIn` ini mengubah `Flow` biasa menjadi `StateFlow` yang dapat dibagikan.

MovieViewModelFactory.kt:

1. Syntax `package com.example.movie.ui.viewmodel` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini adalah bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik terkait dengan `viewModel`, dan dalam kasus ini, bertanggung jawab untuk membuat instance `ViewModel`.
2. Syntax `class MovieViewModelFactory( private val getPopularMoviesUseCase: GetPopularMoviesUseCase ) : ViewModelProvider.Factory { ... }` Ini adalah deklarasi kelas `MovieViewModelFactory`. Kelas ini adalah implementasi dari interface `ViewModelProvider.Factory`.
3. Syntax `override fun <T : ViewModel> create(modelClass: Class<T>): T { ... }` Ini adalah implementasi dari fungsi `create` yang wajib di-override dari interface `ViewModelProvider.Factory`. Fungsi ini bertanggung jawab untuk membuat dan mengembalikan instance `ViewModel` yang diminta.
4. Syntax `if (modelClass.isAssignableFrom(MovieViewModel::class.java)) { ... }` Ini adalah kondisi pengecekan untuk memastikan bahwa `Factory` ini hanya membuat instance dari `MovieViewModel`.
5. Syntax `@Suppress("UNCHECKED_CAST")` Ini adalah anotasi yang memberitahu kompiler Kotlin untuk menekan peringatan `"UNCHECKED_CAST"`. Peringatan ini muncul karena kita melakukan *casting* dari `MovieViewModel` ke tipe generik `T` yang tidak diketahui secara pasti oleh kompiler pada waktu kompilasi, tetapi aman dalam konteks ini karena kita sudah melakukan pengecekan `isAssignableFrom`.

6. Syntax `return MovieViewModel(getPopularMoviesUseCase)` as `T` Jika `modelClass` adalah `MovieViewModel`, baris ini membuat instance baru dari `MovieViewModel`, menyuntikkan `getPopularMoviesUseCase` yang diterima oleh Factory ini ke dalam konstruktor `MovieViewModel`, lalu mengembalikannya sebagai tipe `T`.
7. Syntax `throw IllegalArgumentException("Unknown ViewModel class")` Jika `modelClass` yang diminta bukan `MovieViewModel`, ini akan melemparkan `IllegalArgumentException`. Ini menandakan bahwa Factory ini tidak tahu bagaimana cara membuat `ViewModel` yang diminta.

Caching Strategy: Cache First, then Network

Saya menggunakan caching strategy ini karena saya ingin aplikasi ini mempunyai performa yang cepat untuk memberikan pengalaman pengguna yang baik, aplikasi masih dapat dijalankan meskipun tanpa terhubung ke internet, dan mengantisipasi untuk jaringan internet yang tidak stabil.

Ketika aplikasi membutuhkan data, pertama yang dilakukan dalam strategi ini adalah memeriksa apakah data sudah ada di cache lokal. Jika ada maka data akan langsung ditampilkan kepada pengguna. Namun, jika tidak ada, baru aplikasi akan melakukan permintaan data ke jaringan atau server untuk mendapatkan data tersebut. Sehingga, karena cara kerja ini lah saya memilih strategi ini yang sesuai dengan tujuan pembuatan aplikasi ini.

W. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rizkiirr/Praktikum_Mobile